PullHire — пакет документации

Содержание

- Архитектура
  - Назначение документа
  - 1. Обзор системы
    - 1.1. Состав сервисов
    - 1.2. Текстовая схема (диаграмма)
    - 1.3. Ключевые потоки
  - 2. Микросервисы vs монолит: обоснование
  - 3. Сервисы
    - 3.1. Сводная таблица сервисов
    - 3.2. api-gateway
    - 3.3. frontend (SPA)
    - 3.4. Свернутые модули (folds) и пути выделения
  - 4. Взаимодействие сервисов
    - 4.1. Синхронное взаимодействие (REST)
    - 4.2. События: RabbitMQ, конверт, идемпотентность
    - 4.3. Transactional outbox и итоговая согласованность
    - 4.4. Контракты: OpenAPI и JSON Schema событий
  - 5. Топология контейнеризации (TOPOLOGY A)
    - 5.1. Состав контейнеров docker-compose
    - 5.2. PostgreSQL: схема и роль на сервис
    - 5.3. Healthcheck, зависимости и лимиты памяти
    - 5.4. Миграция TOPOLOGY A → B
  - 6. Безопасность и 152-ФЗ
    - 6.1. Аутентификация: JWT RS256, JWKS, пароли
    - 6.2. Согласия и приватность
    - 6.3. Сетевые границы: gateway, TLS, роли БД
  - 7. Наблюдаемость, конфигурация и CI
    - 7.1. Логи, X-Request-Id, health/metrics
    - 7.2. Конфигурация и секреты (12-factor)
    - 7.3. CI (GitHub Actions)
  - 8. Производительность и платформенность
  - 9. Соответствие требованиям ТЗ
- Модель данных
  - Назначение документа
  - 1. Общие соглашения
  - 2. ERD (обзор по схемам)
  - 3. Схема auth
    - 3.1. users
    - 3.2. email_tokens
    - 3.3. refresh_tokens
    - 3.4. consents
  - 4. Схема candidate
    - 4.1. profiles
    - 4.2. resumes (+файлы)
    - 4.3. privacy_settings
    - 4.4. public_cards
  - 5. Схема fsp
    - 5.1. fsp_links
    - 5.2. achievements_cache
  - 6. Схема assessment
    - 6.1. Грейд-тестирование: item_bank, test_sessions, attempts, grade_history, cooldowns

        - 6.2. Регулярные задания: TaskCycleConfig, EmployerTaskPrompt, Task, TaskVersion, TaskAssignment, TaskSubmission
        - 6.3. Подписка и свежесть: TaskInterestSubscription, ProfileFreshness
    - 7. Схема selection
        - 7.1. taxonomy
        - 7.2. candidate_categories (проекция)
        - 7.3. candidate_cards (без PII)
        - 7.4. rank_signals
    - 8. Схема employer
        - 8.1. companies
        - 8.2. needs
    - 9. Схема outreach
        - 9.1. invitations
        - 9.2. invitation_events
        - 9.3. [ext] vacancies, applications
    - 10. Схема notification
        - 10.1. notification_log
    - 11. Статусы и перечисления (сводный указатель)
    - 12. Видимость данных и согласия (152-ФЗ)
    - 13. Аудит и история изменений
- Модель грейд-тестирования
    - Назначение документа
    - 1. Обзор конвейера
    - 2. Входные этапы
        - 2.1. Опрос
        - 2.2. Самообъявленный грейд
    - 3. Банк заданий
        - 3.1. Форматы заданий
        - 3.2. Структура задания: шаблон-«радикал» и параметры-«инцидентали»
        - 3.3. Якорные задания
        - 3.4. Семейства шаблонов и уникальные seed вариантов
        - 3.5. Начальная калибровка и пересчет (provisional loop)
        - 3.6. Политика применения LLM
    - 4. Сборка формы попытки
    - 5. Адаптивная маршрутизация (IRT-lite, 1PL)
        - 5.1. Оценка θ и выбор следующего задания
        - 5.2. Правила остановки
        - 5.3. Fallback: многоэтапное тестирование (MST)
    - 6. Оценивание: скоринг → грейд
        - 6.1. Отчет θ + SE (не проценты)
        - 6.2. Cut-score и категории-полосы
        - 6.3. «Требует перепроверки» (SE завышен / CI на границе полосы)
    - 7. Повышение и понижение грейда
        - 7.1. Правила up/down
        - 7.2. Запрет принудительного понижения
        - 7.3. Cooldown = 1 месяц
    - 8. Присвоение категории
    - 9. Целостность и сопоставимость
        - 9.1. Защита от утечек
        - 9.2. Прокторинг-флаги и человеческая проверка
        - 9.3. Эквайтинг по якорным заданиям
        - 9.4. Сопоставимость кандидатов: θ как общая шкала
        - 9.5. Антипаттерны
- Регулярные задания работодателей
    - Назначение документа
    - 1. Обзор механики и задействованные сущности
    - 2. Каденс и синхронные окна (A)
    - 3. Цикл задания работодателя (B)

        - 3.1. Промпт {add | update | keep}
        - 3.2. Версии задания (TaskVersion)
        - 3.3. Отставление задания (retire)
        - 3.4. Запрет адресной выдачи
    - 4. Подписка кандидата (C)
    - 5. Розыгрыш задания (D)
        - 5.1. Одна выдача на окно
        - 5.2. Дедупликация на уровне задания
        - 5.3. Дедлайн d = n и возврат в пул
        - 5.4. Пустой / исчерпанный пул; лимит повторов [ext]
    - 6. Выполнение и свежесть профиля (E)
        - 6.1. Формы ответа: «решить» / «предложить подход»
        - 6.2. Оценка работодателем
        - 6.3. Свежесть профиля: display-only
        - 6.4. Сигнал работодателю
    - 7. Защита от «игры с механикой» (F)
    - 8. Граничные случаи
    - 9. MVP и отложенные расширения
- Подбор и выход на контакт
    - Назначение документа
    - 1. Обзор механики
    - 2. Категории кандидатов
    - 3. Ранжирование внутри категории
    - 4. Формирование подборки
        - 4.1. Вход: описание потребности
        - 4.2. Правила формирования
        - 4.3. Объяснимость включения и позиции
    - 5. Поиск и фильтры по банку кандидатов
    - 6. LHH: выход работодателя на контакт
        - 6.1. Приглашение: обязательные поля
        - 6.2. Жизненный цикл приглашения
        - 6.3. Раскрытие контактов
    - 7. Принятые решения по границам механики
- Валидация
    - Назначение документа
    - 1. Цели и область валидации
    - 2. Данные для валидации
    - 3. Процедура: стабильность теста
    - 4. Процедура: дискриминативность
    - 5. Процедура: релевантность подборки
    - 6. Результаты
    - 7. Ограничения валидации
- Интеграция с ФСП
    - Назначение документа
    - 1. Общая схема интеграции
    - 2. Аутентификация: совместимость с Keycloak
    - 3. Принятая модель данных ФСП
    - 4. Мок-реестр и синтетические данные
    - 5. Отображение достижений
    - 6. Отсутствие истории: обработка
    - 7. Путь к боевой интеграции
- API
    - Назначение документа
    - 1. Общие соглашения
    - 2. Модель ошибок
    - 3. Индекс эндпоинтов по сервисам
        - 3.1. auth-service
        - 3.2. candidate-service

        - 3.3. fsp-service
        - 3.4. assessment-service
        - 3.5. selection-service
        - 3.6. employer-service
        - 3.7. outreach-service
        - 3.8. notification-service
        - 3.9. api-gateway и служебные эндпоинты
    - 4. Сгенерированная OpenAPI-документация
- Стек и лицензии
    - Назначение документа
    - 1. Сводная таблица компонентов
    - 2. Backend: библиотеки и версии
    - 3. Frontend: библиотеки и версии
    - 4. Инфраструктура: образы и версии
    - 5. Лицензионная совместимость
    - 6. Открытость кода
- Развёртывание и запуск
    - Назначение документа
    - 1. Требования к окружению
    - 2. Сборка
        - 2.1. Сборка всех сервисов
        - 2.2. Сборка backend-сервисов
    - 3. Локальный запуск (docker-compose)
    - 4. Конфигурация и секреты
    - 5. Проверка работоспособности и типовые проблемы
    - 6. Демонстрационный сценарий и видео-фолбэк
- Границы MVP
    - Назначение документа
    - 1. Принципы границы MVP
    - 2. Входит в MVP
        - 2.1. Роли и сценарии
        - 2.2. Профиль и резюме кандидата
        - 2.3. ФСП-интеграция
        - 2.4. Грейд-тестирование
        - 2.5. Категории и подборка
        - 2.6. Приглашения и выход на контакт (LHH)
        - 2.7. Архитектура и инфраструктура
        - 2.8. Нефункциональные требования
    - 3. Опциональные срезы (вторая очередь)
        - 3.1. Регулярные задания работодателей [opt]
        - 3.2. Классический поток вакансий и откликов [opt]
        - 3.3. Мобильная адаптивность [opt]
        - 3.4. ML/NLP-парсинг резюме [opt]
    - 4. Не входит в MVP
        - 4.1. Концепция защиты от фиктивных вакансий и недобросовестных работодателей
        - 4.2. Концепция интеграции с ATS
    - 5. Обоснование границ
    - 6. Связанные документы
- Пользовательские сценарии
    - Назначение документа
    - 1. Общая легенда пути
    - 2. Сценарий кандидата
        - 2.1. Регистрация и согласия
        - 2.2. Профиль и резюме
        - 2.3. Связывание ФСП ID
        - 2.4. Опрос, грейд и тестирование
        - 2.5. Карточка и видимость
        - 2.6. Приглашение и раскрытие контактов

        - 2.7. Подписка на регулярные задания [opt]
    - 3. Сценарий работодателя
        - 3.1. Регистрация и профиль компании
        - 3.2. Описание потребности
        - 3.3. Подборка и объяснимость
        - 3.4. Поиск и фильтры
        - 3.5. Приглашение и просмотр контактов
        - 3.6. Регулярные задания работодателя [opt]
    - 4. Демонстрационный сценарий
    - 5. Связанные документы
- Глоссарий
    - Назначение документа
    - 1. Базовые термины
    - 2. Термины платформы
    - 3. Термины механик
    - 4. Термины ФСП
    - 5. Архитектурные термины
- Допущения
    - Назначение документа
    - 1. Формат записи
    - 2. Grade-change cooldown = 1 месяц
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 3. Периодические задания: n = 7, синхронные окна, opt-in
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 4. Содержимое каталога специализаций и грейдов
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 5. Состав полей ФСП
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 6. Форматы заданий грейд-теста
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 7. Синтетические данные и seed
        - Что
        - Решение

        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 8. Топология БД: TOPOLOGY A и путь A → B
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 9. Свободные ответы вне MVP грейд-теста (Decision D)
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 10. Политика удаления аккаунта
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
    - 11. Якорение периодических заданий (вакансия/потребность)
        - Что
        - Решение
        - Источник
        - Обоснование
        - Влияние
        - Связанные вопросы
- Итоговая сводка
    - Назначение документа
    - 1. Поставлено vs ТЗ
    - 2. Карта решений
    - 3. Границы и осознанные упрощения
    - 4. Проверка и экспорт пакета
    - 5. Подробнее
- Журнал закрытых вопросов
    - Назначение документа
    - 1. Формат записи
    - 2. Группа Q. Вопросы и ответы (Q-N)
        - Q-1. Совместимость с Keycloak
        - Q-2. Формат заданий
        - Q-3. Интервал смены грейда
        - Q-4. API и структура ФСП
    - 3. Группа TD. Технический долг (TD-N)
        - TD-1. Event-contract: republish `category_assigned`
        - TD-2. Обязательный scope по специализации
        - TD-3. Хранение опроса
        - TD-4. Snapshot `employer_contact`
        - TD-5. Exposure caps на уровне семейства / задания
        - TD-6. Отклонение структуры `testing_model §9.5`
        - TD-7. Пакет правок финального ревью
        - TD-8. Артефакт экспорта doc/pdf
    - 4. Хронология закрытия
- Worklog

- 2026-10-05 — docs/requirements/
- 2026-10-06 — Phase 5: core docs
- 2026-10-06 — Phase 6: structural docs + examples/

Архитектура

Назначение документа

Документ фиксирует функциональную и компонентную архитектуру платформы: декомпозицию на сервисы, синхронное и асинхронное взаимодействие, топологию контейнеризации, безопасность и 152-ФЗ, наблюдаемость и целевые показатели производительности. Реализация отвечает на требование о «понятном разделении на Frontend, Backend (API-сервисы бизнес-логики) и слой хранения» (`docs/requirements/nonfunctional.md`, «Архитектура»).

Связанные документы:

- `docs/data_model.md` - сущности, поля, ключи, связи, перечисления.
- `docs/testing_model.md` - механика грейд-тестирования и присвоение категории.
- `docs/selection.md` - категории, ранжирование, формирование подборки.
- `docs/periodic_tasks.md` - регулярные задания работодателей [ext].
- `docs/fsp_integration.md` - схема интеграции с реестром ФСП.
- `docs/api.md` - индекс эндпоинтов по сервисам и модель ошибок.
- `docs/stack.md` - библиотеки и компоненты с версиями.
- `docs/deployment.md` - сборка, развёртывание и локальный запуск.
- `docs/mvp_scope.md` - границы MVP.
- `docs/assumptions.md` - реестр принятых командой решений.
- `docs/glossary.md` - термины.
- `docs/backlog/closed_questions.md` - журнал решений и техдолга.

Источники требований: `docs/requirements/nonfunctional.md`, `docs/requirements/acceptance.md`, `docs/requirements/overview.md`.

Вне рамок документа: схемы сущностей и полей, алгоритмы тестирования, ранжирования и заданий, конкретные версии библиотек, команды запуска и флаги MVP (это зоны `docs/data_model.md`, `docs/testing_model.md`, `docs/selection.md`, `docs/stack.md`, `docs/deployment.md`, `docs/mvp_scope.md`).

1. Обзор системы

1.1. Состав сервисов

Платформа построена как микросервисная система: восемь бизнес-сервисов, API-шлюз и SPA для обеих ролей.

- `auth-service` - идентичность, JWT, роли, согласия 152-ФЗ.
- `candidate-service` - профиль и резюме, настройки приватности.
- `fsp-service` - шов интеграции с реестром ФСП и достижения.
- `assessment-service` - тестирование, грейд, cooldown, категория; регулярные задания [ext].
- `selection-service` - каталог, категории, ранжирование, подборка.
- `employer-service` - компания и потребность в найме.
- `outreach-service` - приглашения и выход на контакт; вакансии и классический отклик [ext].
- `notification-service` - исходящая электронная почта.
- `api-gateway` (Nginx) - единая входная точка.
- `frontend` (SPA) - веб-приложение для кандидата и работодателя.

Каждый бизнес-сервис - отдельное приложение на Java 21 + Spring Boot, собираемое Gradle, с Spring AMQP (RabbitMQ), Flyway (миграции в своей схеме), springdoc-openapi (документирование), JJWT (RS256) и хешированием паролей Argon2id/BCrypt. Детали и версии - в `docs/stack.md`.

1.2. Текстовая схема (диаграмма)

```
Browser SPA -> api-gateway (Nginx) -> {auth, candidate, fsp, assessment,
      selection, employer, outreach, notification}
  sync REST/JSON (JWT RS256, проверка локально по кэшу JWKS)
  async: RabbitMQ topic exchange
```

```
    user.registered · user.email_verified · consent.updated ·
    candidate.card_updated · candidate.contact_released ·
    fsp.linked · fsp.achievements_updated ·
    assessment.category_assigned · assessment.grade_changed ·
    invitation.{sent,viewed,accepted,declined}
  storage: PostgreSQL (схема и роль на сервис) + volume файлов
```

1.3. Ключевые потоки

- **Онбординг кандидата.** `assessment-service` проводит опрос, выбор грейда и тест; событие `assessment.category_assigned` через outbox попадает в проекцию категорий `selection-service`, после чего карточка становится видимой работодателю. Событие `fsp.achievements_updated` запускает переранжирование (см. `docs/testing_model.md`, `docs/selection.md`).
- **Подбор (LHH).** Работодатель вызывает `POST /selections` в `selection-service`; сервис формирует ранжированную подборку и пояснения из собственных проекций. Затем `POST /invitations` в `outreach-service` запускает приглашение (см. `docs/selection.md`).
- **Выход на контакт.** Кандидат принимает приглашение; `invitation.accepted` → `candidate-service` выдаёт раскрытие контактов → `candidate.contact_released` → `outreach-service` сохраняет контакты для работодателя, а `notification-service` уведомляет кандидата.

2. Микросервисы vs монолит: обоснование

ТЗ приветствует микросервисную архитектуру, но допускает монолит (`docs/requirements/nonfunctional.md`, «Архитектура»). Выбор сделан в пользу микросервисов и подкреплён четырьмя неоднородными швами, а не формальной «галочкой»:

- `fsp-service` - граница внешней интеграции, перепроектируемая после хакатона.
- `notification-service` - иной режим отказа и изоляция SMTP-доступа.
- `assessment-service` - второй по весу оцениваемый компонент (25%) с собственным движком.
- `selection-service` - событийная read-модель, естественная для ранжирования.

Риски компенсированы осознанно: 8 сервисов (не 12), четыре свёрнутых модуля (§3.4), один PostgreSQL, единый стек, отсутствие mesh, распределённого трейсинга и K8s, smoke-тест compose как воспроизводимая проверка. Честная цена решения - примерно +2 недели интеграции против монолита, окупаемые лучшей технической оценкой и наглядной схемой для жюри.

Запасной путь. При отставании схлопываются `employer-service` → `outreach-service` и `fsp-service` → `candidate-service`; контракты событий при этом сохраняются.

3. Сервисы

3.1. Сводная таблица сервисов

#	Сервис	Ответственность	Схема	Ключевые API	События pub/cons	ТЗ
1	`auth-service`	Идентичность, JWT, роли, согласия	`auth`	`/auth/*`, `/.well-known/jwks.json`	pub `user.registered`, `user.email_verified`, `consent.updated`	nonfunctional (Аутентификация, 152-ФЗ); roles пр.4
2	`candidate-service`	Профиль и резюме, приватность	`candidate`	`/candidate/*`, internal card	pub `candidate.card_updated`, `candidate.contact_released`; cons `invitation.accepted`	profiles; roles (кандидат)
3	`fsp-service`	Шов интеграции ФСП, достижения	`fsp`	`/candidate/fsp/*`	pub `fsp.linked`, `fsp.achievements_updated`	profiles (ФСП); acceptance
4	`assessment-service`	Опрос, тест, грейд, cooldown, категория; задания [ext]	`assessment`	`/assessment/*`	pub `assessment.category_assigned` (любая завершённая попытка, включая повышение/понижение грейда и повторные, - для обновления проекции категории `selection`), `assessment.grade_changed` (только	testing.md; roles (кандидат)

#	Сервис	Ответственность	Схема	Ключевые API	События pub/cons	ТЗ
					аудит/уведомления), [ext] `assessment.task_completed`	
5	`selection-service`	Каталог, категории, ранжирование, подборка	`selection`	`/catalog/*`, `/candidates`, `/selections`	cons `assessment.category_assigned`, `candidate.card_updated`, `fsp.achievements_updated`	selection.md; overview; roles (работодатель)
6	`employer-service`	Компания и потребность в найме	`employer`	`/employer/company`, `/employer/needs`	-	profiles (Компания, Потребность)
7	`outreach-service`	Приглашения, контакт; вакансии/отклик [ext]	`outreach`	`/invitations/*`; [ext] `/vacancies/*`	pub `invitation.{sent,viewed,accepted,declined}`; cons `candidate.contact_released`	roles (Выход); selection (Механика); overview (LHH)
8	`notification-service`	Исходящая почта	`notification`	`/notifications`	cons `user.registered`, `invitation.sent`, `invitation.accepted`, `candidate.contact_released`, `assessment.grade_changed`	roles пр.4; structural
9	`api-gateway` (Nginx)	TLS, маршрутизация, rate-limit, `/docs`	конфигурация	`/*`	-	nonfunctional (Архитектура, API)
10	`frontend` (SPA)	Приложение для обеих ролей	-	-	-	nonfunctional; evaluation #3

Данные колонки «Схема» - имя логической схемы PostgreSQL (§5.2). Состав полей и связей - в `docs/data_model.md`. Полный индекс эндпоинтов - в `docs/api.md`. Любая завершённая попытка `assessment-service` (включая повышение/понижение грейда и повторные попытки) публикует `assessment.category_assigned`, чтобы `selection-service` обновил проекцию категории; `assessment.grade_changed` служит только аудиту и уведомлениям (см. `docs/testing_model.md §7–§8`).

3.2. api-gateway

Единая точка входа на Nginx:

- терминация TLS (демо - self-signed сертификат);
- маршрутизация запросов к сервисам;
- статический rate-limit;
- сквозной заголовок `X-Request-Id`;
- индекс документации `/docs` со ссылками на OpenAPI сервисов.

Шлюз не выполняет роль BFF: сервисы самостоятельно валидируют JWT локально по кэшированному JWKS (§6.1). Дополнительный слой агрегации не вводится (см. §4.1).

3.3. frontend (SPA)

Веб-приложение на React/Next.js для обеих ролей с маршрутизацией по роли (кандидат/работодатель). Прямых обращений к сервисам минуя шлюз нет. Клиенты к API строятся по сгенерированным OpenAPI-спецификациям (§4.4). UI-конвенции наследуются от RoboMatch (внешний вид в этом документе не описывается). Реализация сценариев - в `docs/user_flow.md`.

3.4. Свернутые модули (folds) и пути выделения

Чтобы не порождать лишние контексты, четыре модуля свёрнуты в соседние сервисы. Для каждого документирован путь обратного выделения:

- **taxonomy** → `selection-service`. Справочники специализаций и грейдов живут рядом с категориями. Выделение возможно, если каталог станет самостоятельным доменом; контракты `assessment.category_assigned` и `candidate.card_updated` при этом сохраняются.
- **files** → `candidate-service`. Хранение файлов резюме реализовано сменным адаптером; при росте объёмов адаптер заменяется на внешнее хранилище без изменения остальных сервисов.

- **periodic tasks** → `assessment-service` **[ext].** Движок регулярных заданий переиспользует планировщик и форматы заданий. Выделяется в `tasks-service` при отдельном жизненном цикле; интерфейс жёсткий: задания не пишут в грейд/категорию (`docs/periodic_tasks.md`).
- **vacancies/apply** → `outreach-service` **[ext].** Вакансии и классический отклик - модуль внутри LHH-контура. Выделяются в `vacancy-service`, если сценарий выйдет из статуса «плюс» в основной; при этом сохраняется единая механика контактов.

4. Взаимодействие сервисов

4.1. Синхронное взаимодействие (REST)

- Только REST/JSON. gRPC отклонён как избыточный для MVP (YAGNI).
- Поток запроса: браузер → `api-gateway` → сервис.
- Внутренние синхронные вызовы сведены к минимуму на горячих путях за счёт read-моделей-проекций. Единственная цепочка «по требованию»: потребность работодателя → `POST /selections` (сервис читает собственные проекции).
- Оговорка: «единственная цепочка по требованию» действует для ядрового контура. Опциональная механика регулярных заданий `[opt]` добавляет внутренний sync-GET `employer → assessment` для сигнала свежести (`docs/periodic_tasks.md §6.4`).
- Обнаружение сервисов не вводится: DNS Docker Compose и статические маршруты Nginx; K8s/Consul отклонены для MVP.

4.2. События: RabbitMQ, конверт, идемпотентность

- Брокер - RabbitMQ, topic exchange, доставка at-least-once.
- Конверт события: `{event_id, type, version, occurred_at, producer, correlation_id, payload}`.
- Идемпотентность потребителей обеспечивается по `event_id`.
- JSON-схемы событий лежат в `contracts/events/` и проверяются в CI.

Хореография единственного многосервисного сценария (принятие приглашения): `invitation.accepted` → `candidate-service` выдаёт раскрытие контактов → `candidate.contact_released` → `outreach-service` сохраняет контакты → `notification-service` уведомляет кандидата. Управляющий оркестратор не вводится; поведение описано в `docs/selection.md` и `docs/data_model.md §9`.

4.3. Transactional outbox и итоговая согласованность

Для событий, потеря которых критична, применяется transactional outbox:

- `assessment.category_assigned`;
- жизненный цикл приглашения (`invitation.sent/viewed/accepted/declined`).

Согласованность проекций `selection-service` - итоговая: кандидат невидим работодателю до `assessment.category_assigned`, а фронтенд гейтит отображение по статусу. Это принятое следствие выбора read-модели, а не дефект.

4.4. Контракты: OpenAPI и JSON Schema событий

- OpenAPI каждого сервиса генерируется springdoc-openapi автоматически; на шлюзе собирается индекс `/docs`.
- Из спецификаций строятся TypeScript-клиенты для SPA.
- Событийные контракты - JSON Schema в `contracts/events/`; совместимость проверяется в CI (§7.3), изменения фиксируются в `docs/backlog/closed_questions.md`.

5. Топология контейнеризации (TOPOLOGY A)

5.1. Состав контейнеров docker-compose

```
core:
  frontend (nginx) · gateway (nginx) ·
  auth · candidate · fsp · assessment ·
  selection · employer · outreach · notification   (Spring Boot JARs) ·
  postgres  (1 контейнер: схемы auth|candidate|fsp|assessment|
```

```
             selection|employer|outreach|notification,
             отдельные роли, без доступа между схемами) ·
  rabbitmq  (Management UI) · mailpit (dev)
optional profiles:
  ml-parser-worker [ext] · prometheus + grafana (демо)
```

- Целевой объём памяти ≈ 2.0–2.2 ГБ RAM.
- Одноразовый контейнер `seed` наполняет каталог идемпотентно.
- Команды сборки и запуска - в `docs/deployment.md`; версии образов - в `docs/stack.md`.

5.2. PostgreSQL: схема и роль на сервис

Один контейнер PostgreSQL, логическая изоляция:

- у каждого сервиса своя схема: `auth`, `candidate`, `fsp`, `assessment`, `selection`, `employer`, `outreach`, `notification`;
- у каждой схемы отдельная роль БД; сервис подключается только своей ролью;
- **жёсткая конвенция:** нет кросс-схемных `JOIN` и внешних ключей между схемами; доступ к чужим данным - только через API или события.

Данные и связи внутри схем - в `docs/data_model.md`.

5.3. Healthcheck, зависимости и лимиты памяти

- На каждом сервисе: `/healthz` (liveness) и `/readyz` (готовность БД и брокера).
- В compose используется `depends_on: service_healthy`.
- Для контейнеров задан `mem_limit`.
- Целевой суммарный footprint - около 2 ГБ (§5.1), что согласуется с требованием «удерживать память в разумных пределах» (`docs/requirements/nonfunctional.md`, «Аппаратные требования»).

5.4. Миграция TOPOLOGY A → B

TOPOLOGY B - физический PostgreSQL на сервис. Переход выполняется **без переписывания кода**:

- жёсткая конвенция отсутствия кросс-схемного доступа (§5.2) делает разделение конфигурационным;
- шаги: перенос `PGDATA` соответствующей схемы в отдельный инстанс и обновление строк подключения в конфигурации сервиса;
- сервисные и событийные контракты не меняются.

Решение о TOPOLOGY A обосновано бюджетом памяти, единой резервной копией и логической изоляцией на старте (реестр решений - `docs/assumptions.md`).

6. Безопасность и 152-ФЗ

6.1. Аутентификация: JWT RS256, JWKS, пароли

- Регистрация по email с подтверждением адреса; модерация работодателей на этапе MVP не требуется (`docs/requirements/acceptance.md`).
- Токены: JWT RS256 с коротким TTL и refresh-токеном; публичный ключ публикуется как `/.well-known/jwks.json`.
- Форма claim совместима с Keycloak (`sub`, `realm_access.roles`, JWKS) - упрощает будущую интеграцию с ФСП ID (`docs/requirements/nonfunctional.md`, «Аутентификация»).
- Пароли хранятся как Argon2id/BCrypt.
- Проверка ролей (candidate/employer) выполняется по claim-ам JWT.
- Схема интеграции с ФСП - в `docs/fsp_integration.md`.

6.2. Согласия и приватность

- Согласие на обработку данных фиксируется при регистрации (`auth-service`), изменения согласий аудируются.
- Область видимости («что видно») принадлежит `candidate-service`; событие `candidate.card_updated` несёт только согласованные поля, так что поисковый индекс `selection-service` структурно не может утечь PII.

- Раскрытие контактов происходит только через событие `candidate.contact_released` - то есть после принятия приглашения **или самостоятельного отклика кандидата** `[ext]` (правила ролей в `docs/data_model.md §12`).
- Хранение и обработка - на инфраструктуре на территории РФ (локализация по 152-ФЗ).

6.3. Сетевые границы: gateway, TLS, роли БД

- Внешний трафик входит только через `api-gateway`; TLS терминируется на нём (демо - self-signed).
- Сервисы валидируют JWT локально по кэшу JWKS, не ходя на `auth-service` на каждый запрос.
- Разграничение доступа к данным обеспечено ролями БД на схему (§5.2); кросс-схемный доступ запрещён конвенцией.

7. Наблюдаемость, конфигурация и CI

7.1. Логи, X-Request-Id, health/metrics

- Структурированные JSON-логи.
- Заголовок `X-Request-Id` распространяется как `correlation_id` в событиях, обеспечивая сквозную трассировку запроса без отдельного трейсера.
- `/healthz` и `/readyz` на каждом сервисе (§5.3).
- `/metrics` в формате Prometheus; Prometheus + Grafana подключаются опциональным profile (демо).
- Распределённый трейсинг в MVP не вводится.

7.2. Конфигурация и секреты (12-factor)

- Конфигурация через переменные окружения (12-factor).
- `.env.example` коммитится, `.env` - в `.gitignore`.
- Секреты не попадают в репозиторий; в CI встроена проверка.
- Реестр конфигурационных решений - `docs/assumptions.md`.

7.3. CI (GitHub Actions)

Конвейер на каждый сервис:

- линтер/форматтер → сборка и тесты (Java: build + test);
- diff снимка OpenAPI (контроль контрактов);
- проверка совместимости JSON Schema событий;
- сборка Docker-образа;
- smoke-тест compose: `up` → зелёные healthcheck → `seed` → скриптованный e2e-сценарий.

Smoke-тест одновременно закрывает требование «по запросу жюри команда демонстрирует запуск и работу кода» (`docs/requirements/nonfunctional.md`, «Качество кода»). Состав smoke и e2e детализирован в `docs/deployment.md`; e2e-сценарий покрывает путь `docs/testing_model.md §1`.

8. Производительность и платформенность

Ориентиры MVP (`docs/requirements/nonfunctional.md`, «Производительность», «Платформенность»):

- стабильная работа при одновременном использовании десятками пользователей;
- основные операции - формирование подборки, открытие профиля, поиск - укладываются в приемлемое время; этого достигают read-модели-проекции в `selection-service` (§4.1) и отсутствие синхронных внутренних цепочек на горячих путях;
- кроссплатформенный веб-сервис для современных десктопных браузеров, независимость от ОС при развёртывании (Docker);
- адаптивная вёрстка для мобильных и планшетов - опционально для MVP (границы - `docs/mvp_scope.md`).

9. Соответствие требованиям ТЗ

Компонент	Требование ТЗ
`auth-service`	nonfunctional (Безопасность/152-ФЗ, Аутентификация); roles пр.4

Компонент	Требование ТЗ
`candidate-service`	profiles (Профиль/резюме, PDF); roles (кандидат)
`fsp-service`	profiles (Связь с ФСП, Достижения); acceptance
`assessment-service`	testing.md; roles (кандидат); overview
`selection-service`	selection.md; overview; roles (работодатель); evaluation #1
`employer-service`	profiles (Компания, Потребность); roles (работодатель)
`outreach-service`	roles (Выход/пр.1-3); selection (Механика); overview (LHH); evaluation #6
`notification-service`	roles пр.4
`api-gateway`	nonfunctional (Архитектура, Документирование API)
`frontend` (SPA)	nonfunctional; evaluation #3
compose / CI	nonfunctional (Контейнеризация, Качество кода, Открытый исходный код); acceptance
post-MVP концепция	overview/acceptance (фейк-вакансии, ATS) - концепция в `docs/mvp_scope.md §4`; только документ, без кода

Полное сопоставление требований и статусов реализации - в `docs/final_summary.md` и `docs/mvp_scope.md`.

Модель данных

Назначение документа

`docs/data_model.md` описывает логическую модель данных PullHire: схемы PostgreSQL по числу бизнес-сервисов, сущности, поля, ключи, ограничения и связи. Документ служит источником структуры хранения для `docs/architecture.md §5.2` и `§5.4`; механики описаны отдельно — `docs/testing_model.md`, `docs/selection.md`, `docs/periodic_tasks.md`, `docs/fsp_integration.md`.

Связанные документы: `docs/architecture.md`, `docs/testing_model.md`, `docs/selection.md`, `docs/periodic_tasks.md`, `docs/fsp_integration.md`, `docs/mvp_scope.md`, `docs/assumptions.md`, `docs/glossary.md`. Источники требований: `docs/requirements/roles.md`, `profiles.md`, `testing.md`, `selection.md`.

Вне рамок: бизнес-правила и алгоритмы (mechanics-документы), контракты эндпоинтов (`docs/api.md`), инфраструктура и контейнеры (`docs/architecture.md`), MVP-флаги (`docs/mvp_scope.md`).

1. Общие соглашения

Тема	Правило
Имена	snake_case для схем, таблиц, полей; коды перечислений — английский snake_case.
Схемы	8 схем по числу сервисов: auth, candidate, fsp, assessment, selection, employer, outreach, notification.
Первичный ключ	`id bigserial` (или `bigint` для проекций-1:1), `PK`.
Внешний ключ	Только внутри одной схемы, `FK → table.id`.
HARD RULE	Cross-schema FK и cross-schema JOIN запрещены; интеграция между сервисами — только через события (`docs/architecture.md §4.2`).
Логическая ссылка	Поле вида `candidate_id`, `user_id`, `vacancy_id` на чужую схему — без FK, помечается «логическая ссылка».
Время	`timestamptz`, хранение в UTC; границы окон — в TZ платформы. `created_at` / `updated_at NOT NULL DEFAULT now()`.
Деньги	`numeric(12,2)` в рублях; диапазоны «от–до» — два поля + CHECK `to >= from`.
Перечисления	`text` + `CHECK ∈ (…)` в таблице-владельце; сводка — §11.
Полезная нагрузка	`jsonb` для payload/снимков; `text[]` для списков (stack, skills).
Проекции	Read-model таблицы сервиса (например `selection.candidate_categories`) наполняются событиями; источник истины — сервис-владелец.
`[ext]`	Опциональные сущности/поля, см. `docs/mvp_scope.md`.
Миграции	Flyway, по одной папке миграций на схему.
Исключение имён	Сущности семейства заданий (`TaskCycleConfig`, `EmployerTaskPrompt`, `Task`, `TaskVersion`, `TaskAssignment`, `TaskSubmission`, `TaskInterestSubscription`, `ProfileFreshness`) — PascalCase: исторически сложившаяся нотация, на неё ссылаются внешние документы (`docs/periodic_tasks.md`); все прочие таблицы — snake_case.

Коды справочника (`specialization_code`, `industry_code`, `grade`, `concept_code` и т.п.) проверяются сервисом-владельцем при записи против засеянного справочника `selection.taxonomy`; drift выявляется на CI/seed-прогоне. Cross-schema FK на taxonomy запрещён (HARD RULE), поэтому целостность кодов обеспечивается валидацией на запись, а не ограничением БД.

2. ERD (обзор по схемам)

```
auth.users 1–N auth.email_tokens
auth.users 1–N auth.refresh_tokens
auth.users 1–N auth.consents

candidate.profiles 1–N candidate.resumes
candidate.profiles 1–1 candidate.privacy_settings
candidate.profiles 1–1 candidate.public_cards
```

```
fsp.fsp_links 1–N fsp.achievements_cache

assessment.item_bank 1–N assessment.attempts
assessment.test_sessions 1–N assessment.attempts
assessment.test_sessions 1–N assessment.grade_history
assessment.cooldowns (per candidate)
assessment.TaskCycleConfig (singleton)
assessment.EmployerTaskPrompt (per vacancy × cycle)
assessment.Task 1–N assessment.TaskVersion
assessment.Task 1–N assessment.TaskAssignment
assessment.TaskAssignment 1–1 assessment.TaskSubmission
assessment.TaskInterestSubscription (per candidate)
assessment.ProfileFreshness (per candidate)

selection.taxonomy 1–N selection.taxonomy (parent)
selection.candidate_categories (проекция, per candidate)
selection.candidate_cards (проекция, per candidate)
selection.rank_signals (per candidate × category)

employer.companies 1–N employer.needs

outreach.invitations 1–N outreach.invitation_events
outreach.vacancies 1–N outreach.applications          [ext]

notification.notification_log
```

Межсхемные связи — только логические, через идентификаторы, без FK:

```
auth.users ── candidate.profiles ── fsp.fsp_links
assessment.candidate_id ── selection.candidate_categories ── outreach.invitations
employer.companies ── employer.needs
outreach.vacancies (vacancy_id) ── assessment.Task   [ext]
employer.needs (need_id) ── assessment.Task   (fallback, когда вакансия [ext] не построена)
```

3. Схема auth

Владелец — `auth-service`. Аутентификация, подтверждение email, роли, согласия 152-ФЗ (`docs/architecture.md §6.1–§6.2`).

3.1. users

Назначение: учётные записи кандидатов и работодателей; источник `sub` для JWT.

Поле	Тип	Ограничения
id	bigserial	PK
email	text	NOT NULL, UNIQUE
password_hash	text	NOT NULL — Argon2id/BCrypt
role	text	NOT NULL, CHECK ∈ (candidate, employer)
status	text	NOT NULL DEFAULT 'pending', CHECK ∈ (pending, active, blocked)
email_verified_at	timestamptz	NULL
deleted_at	timestamptz	NULL — soft-delete аккаунта (`docs/assumptions.md §10`)
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE email; btree role; btree status; btree deleted_at. UNIQUE: `email`.

3.2. email_tokens

Назначение: одноразовые токены подтверждения адреса (§4 roles.md п.4).

Поле	Тип	Ограничения
id	bigserial	PK
user_id	bigint	NOT NULL, FK → users.id ON DELETE CASCADE
token_hash	text	NOT NULL, UNIQUE
purpose	text	NOT NULL DEFAULT 'verify_email', CHECK ∈ (verify_email)
expires_at	timestamptz	NOT NULL
used_at	timestamptz	NULL
created_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE token_hash; btree (user_id); btree expires_at. UNIQUE: `token_hash`.

3.3. refresh_tokens

Назначение: refresh-сессии JWT RS256 с возможностью отзыва.

Поле	Тип	Ограничения
id	bigserial	PK
user_id	bigint	NOT NULL, FK → users.id ON DELETE CASCADE
token_hash	text	NOT NULL, UNIQUE
expires_at	timestamptz	NOT NULL
revoked_at	timestamptz	NULL
created_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE token_hash; btree (user_id); btree expires_at. UNIQUE: `token_hash`.

3.4. consents

Назначение: append-only журнал согласий 152-ФЗ (обработка, публикация профиля, связь с ФСП, участие в регулярных заданиях и показ результата — `task_participation`, `docs/periodic_tasks.md §4` C1); источник аудита §13.

Поле	Тип	Ограничения
id	bigserial	PK
user_id	bigint	NOT NULL, FK → users.id ON DELETE CASCADE
consent_type	text	NOT NULL, CHECK ∈ (data_processing, profile_publication, fsp_link, task_participation)
granted	boolean	NOT NULL
policy_version	text	NOT NULL
granted_at	timestamptz	NULL
revoked_at	timestamptz	NULL
created_at	timestamptz	NOT NULL DEFAULT now()

Индексы: btree (user_id, consent_type, created_at). UNIQUE: нет (append-only; текущее состояние = последняя запись по паре `user_id, consent_type`).

4. Схема candidate

Владелец — `candidate-service`. Единственный владелец «что видно» работодателю (`docs/architecture.md §6.2`).

4.1. profiles

Назначение: профиль кандидата (форма и/или авторасознанный PDF-профиль).

Поле	Тип	Ограничения
id	bigserial	PK

Поле	Тип	Ограничения
user_id	bigint	NOT NULL, UNIQUE — логическая ссылка на auth.users.id
full_name	text	NOT NULL
contact_email	text	NULL
contact_phone	text	NULL
position_title	text	NULL
grade_declared	text	NULL — самообъявленный грейд
experience_years	numeric(4,1)	NULL, CHECK >= 0
stack	text[]	NULL
skills	text[]	NULL
soft_skills	text[]	NULL
summary	text	NULL
specialization_code	text	NULL — логическая ссылка на selection.taxonomy
industry_code	text	NULL — логическая ссылка на selection.taxonomy
status	text	NOT NULL DEFAULT 'draft', CHECK ∈ (draft, published)
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE user_id; btree specialization_code; GIN stack. UNIQUE: `user_id`.

4.2. resumes (+файлы)

Назначение: загруженные PDF-резюме и сгенерированный стандартизированный PDF; файлы — в файловом volume, в БД только метаданные.

Поле	Тип	Ограничения
id	bigserial	PK
profile_id	bigint	NOT NULL, FK → profiles.id ON DELETE CASCADE
kind	text	NOT NULL, CHECK ∈ (uploaded, generated)
file_name	text	NOT NULL
file_path	text	NOT NULL — путь в хранилище
mime_type	text	NULL
size_bytes	bigint	NULL, CHECK (size_bytes IS NULL OR size_bytes >= 0)
parse_status	text	NOT NULL DEFAULT 'pending', CHECK ∈ (pending, parsed, failed)
parsed_at	timestamptz	NULL
created_at	timestamptz	NOT NULL DEFAULT now()

Индексы: btree (profile_id); btree (kind). UNIQUE: нет.

4.3. privacy_settings

Назначение: настройки видимости профиля для работодателя (перекрываются §12).

Поле	Тип	Ограничения
profile_id	bigint	PK, FK → profiles.id ON DELETE CASCADE
visible_to_employers	boolean	NOT NULL DEFAULT true
show_fsp	boolean	NOT NULL DEFAULT true
show_contacts	boolean	NOT NULL DEFAULT false — контакты дополнительно гейтятся правилом §12
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: PK. UNIQUE: PK `profile_id` (1:1 с profiles).

4.4. public_cards

Назначение: privacy-filtered снимок карточки, источник события `candidate.card_updated`.

Поле	Тип	Ограничения
id	bigserial	PK
profile_id	bigint	NOT NULL, UNIQUE, FK → profiles.id ON DELETE CASCADE
display_name	text	NOT NULL — ФИО или маскированное
headline	text	NULL
industry_code	text	NULL
specialization_code	text	NULL
grade	text	NULL — снимок текущего грейда
stack	text[]	NULL
fsp_verified	boolean	NOT NULL DEFAULT false
contact_released	boolean	NOT NULL DEFAULT false
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE profile_id; GIN stack. UNIQUE: `profile_id`.

5. Схема fsp

Владелец — `fsp-service`. Граница интеграции с реестром ФСП (`docs/fsp_integration.md`); на хакатоне — mock-адаптер.

5.1. fsp_links

Назначение: связь кандидата с ID участника ФСП; обработка случая «нет истории».

Поле	Тип	Ограничения
id	bigserial	PK
candidate_id	bigint	NOT NULL, UNIQUE — логическая ссылка на candidate.profiles.id
fsp_id	text	NOT NULL, UNIQUE
link_status	text	NOT NULL DEFAULT 'pending', CHECK ∈ (pending, linked, unlinked, error)
history_status	text	NOT NULL DEFAULT 'unknown', CHECK ∈ (unknown, present, no_history)
linked_at	timestamptz	NULL
last_sync_at	timestamptz	NULL
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE candidate_id; UNIQUE fsp_id. UNIQUE: `candidate_id`, `fsp_id`.

5.2. achievements_cache

Назначение: кэш достижений ФСП для отображения в профиле; пустой набор допустим (`history_status = no_history`).

Поле	Тип	Ограничения
id	bigserial	PK
fsp_link_id	bigint	NOT NULL, FK → fsp_links.id ON DELETE CASCADE
external_achievement_id	text	NULL
title	text	NOT NULL
event_type	text	NULL
achieved_at	date	NULL
place	integer	NULL, CHECK (place IS NULL OR place >= 1)

Поле	Тип	Ограничения
payload	jsonb	NULL
synced_at	timestamptz	NOT NULL DEFAULT now()

Индексы: btree (fsp_link_id); btree (achieved_at). UNIQUE: `(fsp_link_id, external_achievement_id)`.

6. Схема assessment

Владелец — `assessment-service`. Грейд-тестирование и регулярные задания (`docs/testing_model.md`, `docs/periodic_tasks.md`).

6.1. Грейд-тестирование: item_bank, test_sessions, attempts, grade_history, cooldowns

item_bank — банк заданий: якорные задания и семейства шаблонов; параметры калибровки.

Поле	Тип	Ограничения
id	bigserial	PK
item_kind	text	NOT NULL, CHECK ∈ (anchor, template)
template_family	text	NULL — CHECK (item_kind = 'template' ⇒ template_family IS NOT NULL)
concept_code	text	NOT NULL
specialization_code	text	NOT NULL
difficulty	numeric(5,3)	NOT NULL — параметр `b`
is_provisional	boolean	NOT NULL DEFAULT true
exposure_limit	integer	NULL, CHECK >= 0
exposure_count	integer	NOT NULL DEFAULT 0, CHECK >= 0
payload	jsonb	NOT NULL — модель задания и ключ
status	text	NOT NULL DEFAULT 'active', CHECK ∈ (active, retired)
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: btree (specialization_code, difficulty); btree template_family; btree item_kind. UNIQUE: нет. Вариантность задаётся seed формы `test_sessions.form_seed` (набор вариантов сессии) и item-seed `attempts.variant_seed` (вариант конкретного элемента); якорные задания не имеют `template_family`.

test_sessions — сессия прохождения: опрос → выбор грейда → тест; хранит θ и SE.

Поле	Тип	Ограничения
id	bigserial	PK
candidate_id	bigint	NOT NULL — логическая ссылка на candidate.profiles.id
industry_code	text	NULL
specialization_code	text	NOT NULL
target_grade	text	NOT NULL — заявленный грейд
survey_answers	jsonb	NULL — ответы обязательного этапа SURVEY (`docs/testing_model.md §2.1`)
declared_dimensions	text[]	NULL — заявленные в опросе отрасль/специализация/грейд
form_seed	bigint	NULL — seed формы (набор вариантов заданий сессии)
status	text	NOT NULL DEFAULT 'started', CHECK ∈ (started, in_progress, completed, abandoned, needs_review)
theta	numeric(6,3)	NULL
theta_se	numeric(6,3)	NULL
item_count	integer	NOT NULL DEFAULT 0, CHECK >= 0
started_at	timestamptz	NOT NULL DEFAULT now()
completed_at	timestamptz	NULL

Индексы: btree (candidate_id, status); btree specialization_code. UNIQUE: нет.

attempts — ответы на задания внутри сессии; прокторинг-флаги с временными метками.

Поле	Тип	Ограничения
id	bigserial	PK
session_id	bigint	NOT NULL, FK → test_sessions.id ON DELETE CASCADE
item_id	bigint	NOT NULL, FK → item_bank.id
item_order	integer	NOT NULL, CHECK >= 1
variant_seed	bigint	NULL — seed варианта элемента в рамках `test_sessions.form_seed`
answer	jsonb	NULL
is_correct	boolean	NULL
response_ms	integer	NULL, CHECK >= 0
proctoring_flags	jsonb	NULL — без авто-провала, только для проверки человеком
served_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE (session_id, item_order); btree item_id. UNIQUE: `(session_id, item_order)`; `(session_id, item_id)`.

grade_history — append-only история смен грейда/категории; аудит §13.

Поле	Тип	Ограничения
id	bigserial	PK
candidate_id	bigint	NOT NULL — логическая ссылка на candidate.profiles.id
grade	text	NOT NULL
specialization_code	text	NOT NULL
direction	text	NOT NULL, CHECK ∈ (initial, up, down, recheck)
session_id	bigint	NULL, FK → test_sessions.id
category_code	text	NULL
reason	text	NULL
created_at	timestamptz	NOT NULL DEFAULT now()

Индексы: btree (candidate_id, created_at); btree session_id. UNIQUE: нет (append-only). Понижение — только по инициативе кандидата (`docs/testing_model.md §7.2`).

cooldowns — ограничение частоты смены грейда (1 месяц); `docs/testing_model.md §7.3`.

Поле	Тип	Ограничения
id	bigserial	PK
candidate_id	bigint	NOT NULL — логическая ссылка на candidate.profiles.id
scope	text	NOT NULL DEFAULT 'grade_change', CHECK ∈ (grade_change)
available_after	timestamptz	NOT NULL
set_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE (candidate_id, scope); btree available_after. UNIQUE: `(candidate_id, scope)`.

6.2. Регулярные задания: TaskCycleConfig, EmployerTaskPrompt, Task, TaskVersion, TaskAssignment, TaskSubmission

Поля сущностей — здесь; правила механики — `docs/periodic_tasks.md`.

TaskCycleConfig — singleton-конфигурация каденса и окон.

Поле	Тип	Ограничения
id	smallint	PK, CHECK (id = 1) — singleton

Поле	Тип	Ограничения
n_days	integer	NOT NULL DEFAULT 7, CHECK > 0
deadline_days	integer	NOT NULL DEFAULT 7, CHECK > 0
timezone	text	NOT NULL DEFAULT 'Europe/Moscow'
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: PK. UNIQUE: PK `id` (единственная строка).

EmployerTaskPrompt — промпт работодателю по вакансии на окно {add|update|keep}.

Поле	Тип	Ограничения
id	bigserial	PK
employer_id	bigint	NOT NULL — логическая ссылка на auth.users.id
vacancy_id	bigint	NULL — логическая ссылка на outreach.vacancies.id ([ext]); NULL при fallback
need_id	bigint	NULL — логическая ссылка на employer.needs.id; заполняется в fallback-режиме
cycle_date	date	NOT NULL
status	text	NOT NULL DEFAULT 'pending', CHECK ∈ (pending, resolved, expired)
action	text	NULL, CHECK (action IN ('add','update','keep'))
resolved_at	timestamptz	NULL
created_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE (vacancy_id, cycle_date) WHERE vacancy_id IS NOT NULL; UNIQUE (need_id, cycle_date) WHERE need_id IS NOT NULL; btree (employer_id, cycle_date); btree status. UNIQUE: `(vacancy_id, cycle_date) WHERE vacancy_id IS NOT NULL`; `(need_id, cycle_date) WHERE need_id IS NOT NULL` (fallback-режим «потребность/компания»).

Task — задание работодателя (агрегат), текущая активная версия и счётчик keep.

Поле	Тип	Ограничения
id	bigserial	PK
employer_id	bigint	NOT NULL — логическая ссылка на auth.users.id
vacancy_id	bigint	NULL — логическая ссылка на outreach.vacancies.id
need_id	bigint	NULL — логическая ссылка на employer.needs.id; заполняется в fallback-режиме (нет вакансии)
status	text	NOT NULL DEFAULT 'draft', CHECK ∈ (draft, active, retired)
current_version_id	bigint	NULL, FK → TaskVersion.id
keep_count	integer	NOT NULL DEFAULT 0, CHECK >= 0
stale_since	timestamptz	NULL — `[ext]`
created_at	timestamptz	NOT NULL DEFAULT now()
retired_at	timestamptz	NULL

Индексы: btree (employer_id, status); btree vacancy_id; btree need_id. UNIQUE: нет. Адресная выдача конкретному кандидату запрещена (`docs/periodic_tasks.md §3.4`). Якорение задания: `vacancy_id` при наличии вакансии, иначе `need_id` — fallback «потребность/компания» (`docs/assumptions.md §11`); оба поля NULLable, пул выдачи поддерживает оба якоря.

TaskVersion — неизменяемая версия задания (snapshot).

Поле	Тип	Ограничения
id	bigserial	PK
task_id	bigint	NOT NULL, FK → Task.id ON DELETE CASCADE
version_no	integer	NOT NULL, CHECK >= 1
title	text	NOT NULL
body	text	NOT NULL
format	text	NOT NULL, CHECK ∈ (code, text, quiz)

Поле	Тип	Ограничения
autocheck_spec	jsonb	NULL
created_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE (task_id, version_no); btree task_id. UNIQUE: `(task_id, version_no)`. Строки не обновляются (immutable).

TaskAssignment — выдача задания кандидату в окне; одна на окно.

Поле	Тип	Ограничения
id	bigserial	PK
candidate_id	bigint	NOT NULL — логическая ссылка на candidate.profiles.id
task_id	bigint	NOT NULL, FK → Task.id
version_id	bigint	NOT NULL, FK → TaskVersion.id
cycle_date	date	NOT NULL
status	text	NOT NULL DEFAULT 'assigned', CHECK ∈ (assigned, opened, submitted, evaluated, expired)
assigned_at	timestamptz	NOT NULL DEFAULT now()
deadline_at	timestamptz	NOT NULL

Индексы: UNIQUE (candidate_id, cycle_date); btree task_id; btree status. UNIQUE: `(candidate_id, cycle_date)`.

TaskSubmission — ответ кандидата («решить» / «предложить подход») и оценка работодателя 1:1.

Поле	Тип	Ограничения
id	bigserial	PK
assignment_id	bigint	NOT NULL, UNIQUE, FK → TaskAssignment.id ON DELETE CASCADE
kind	text	NOT NULL, CHECK ∈ (solution, approach)
content	text	NOT NULL
submitted_at	timestamptz	NOT NULL DEFAULT now()
result	text	NOT NULL DEFAULT 'unreviewed', CHECK ∈ (positive, negative, unreviewed)
reviewed_by	bigint	NULL — логическая ссылка на auth.users.id
reviewed_at	timestamptz	NULL

Индексы: UNIQUE assignment_id. UNIQUE: `assignment_id` (1:1 с TaskAssignment). Результат не меняет грейд/категорию.

6.3. Подписка и свежесть: TaskInterestSubscription, ProfileFreshness

TaskInterestSubscription — опт-ин кандидата на регулярные задания (по умолчанию OFF).

Поле	Тип	Ограничения
candidate_id	bigint	PK — логическая ссылка на candidate.profiles.id
enabled	boolean	NOT NULL DEFAULT false
scope	jsonb	NULL — `[ext]` опциональный фильтр по грейду; скоуп по специализации обязателен и задаётся вне `scope`
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: PK. UNIQUE: PK `candidate_id` (1:1, `docs/periodic_tasks.md §4`). Скоуп пула по специализации обязателен (кандидат видит задания своей специализации); опционален только фильтр по грейду (`[ext]`).

ProfileFreshness — атрибут свежести профиля; display-only, не входит в ранжирование.

Поле	Тип	Ограничения
candidate_id	bigint	PK — логическая ссылка на candidate.profiles.id
last_task_activity_at	timestamptz	NULL

Поле	Тип	Ограничения
tasks_submitted	integer	NOT NULL DEFAULT 0, CHECK >= 0
tasks_positive	integer	NOT NULL DEFAULT 0, CHECK >= 0
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: PK. UNIQUE: PK `candidate_id` (1:1). HARD RULE: не влияет на grade/grade_history/категорию (`docs/testing_model.md §7`, `docs/periodic_tasks.md §6.3`).

7. Схема selection

Владелец — `selection-service`. Каталог, категории и проекции для ранжирования (`docs/selection.md`).

7.1. taxonomy

Назначение: справочник отраслей, специализаций и грейдов; категория = специализация × грейд.

Поле	Тип	Ограничения
id	bigserial	PK
kind	text	NOT NULL, CHECK ∈ (industry, specialization, grade)
code	text	NOT NULL
name	text	NOT NULL
parent_id	bigint	NULL, FK → taxonomy.id
grade_order	integer	NULL, CHECK (grade_order IS NULL OR grade_order >= 0)
is_active	boolean	NOT NULL DEFAULT true
created_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE (kind, code); btree parent_id; btree (kind, is_active). UNIQUE: `(kind, code)`.

7.2. candidate_categories (проекция)

Назначение: текущая категория кандидата; проекция события `assessment.category_assigned`.

Поле	Тип	Ограничения
id	bigserial	PK
candidate_id	bigint	NOT NULL, UNIQUE — логическая ссылка на candidate.profiles.id
specialization_code	text	NOT NULL
grade	text	NOT NULL
category_code	text	NOT NULL
theta	numeric(6,3)	NULL
source_event_id	text	NULL — идемпотентность консьюмера
assigned_at	timestamptz	NOT NULL
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE candidate_id; btree category_code; btree (specialization_code, grade). UNIQUE: `candidate_id`.

7.3. candidate_cards (без PII)

Назначение: поисковая проекция карточки без PII; наполняется `candidate.card_updated` (только согласованные поля, `docs/architecture.md §6.2`).

Поле	Тип	Ограничения
id	bigserial	PK
candidate_id	bigint	NOT NULL, UNIQUE — логическая ссылка на candidate.profiles.id

Поле	Тип	Ограничения
display_name	text	NOT NULL — маскированное
specialization_code	text	NOT NULL
grade	text	NOT NULL
stack	text[]	NULL
fsp_verified	boolean	NOT NULL DEFAULT false
fsp_achievements_count	integer	NOT NULL DEFAULT 0, CHECK >= 0
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE candidate_id; btree (specialization_code, grade); GIN stack. UNIQUE: `candidate_id`. Контактные данные в проекции отсутствуют (§12).

7.4. rank_signals

Назначение: компоненты ранжирования внутри категории (сила подтверждённого профиля).

Поле	Тип	Ограничения
id	bigserial	PK
candidate_id	bigint	NOT NULL — логическая ссылка на candidate.profiles.id
category_code	text	NOT NULL
theta	numeric(6,3)	NULL
fsp_verified	boolean	NOT NULL DEFAULT false
fsp_achievements_count	integer	NOT NULL DEFAULT 0, CHECK >= 0
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE (candidate_id, category_code); btree category_code. UNIQUE: `(candidate_id, category_code)`. Формула ранжирования — `docs/selection.md`.

8. Схема employer

Владелец — `employer-service`. Профиль компании и описание потребности.

8.1. companies

Назначение: профиль компании-работодателя.

Поле	Тип	Ограничения
id	bigserial	PK
user_id	bigint	NOT NULL, UNIQUE — логическая ссылка на auth.users.id
name	text	NOT NULL
description	text	NULL
industry	text	NULL
contact_email	text	NULL
contact_phone	text	NULL
website	text	NULL
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE user_id. UNIQUE: `user_id`.

8.2. needs

Назначение: описание потребности (вход формирования подборки).

Поле	Тип	Ограничения
id	bigserial	PK
company_id	bigint	NOT NULL, FK → companies.id ON DELETE CASCADE
title	text	NOT NULL
description	text	NULL
specialization_code	text	NOT NULL
grade	text	NOT NULL
stack	text[]	NULL
status	text	NOT NULL DEFAULT 'active', CHECK ∈ (active, archived)
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: btree company_id; btree (specialization_code, grade). UNIQUE: нет.

9. Схема outreach

Владелец — `outreach-service`. LHH-механика выхода на контакт, статусы и раскрытие контактов (`docs/selection.md`, roles.md пп.1–3).

9.1. invitations

Назначение: приглашение конкретному кандидату; зарплата обязательна, привязка к вакансии не обязательна.

Поле	Тип	Ограничения
id	bigserial	PK
employer_id	bigint	NOT NULL — логическая ссылка на auth.users.id
candidate_id	bigint	NOT NULL — логическая ссылка на candidate.profiles.id
need_id	bigint	NULL — логическая ссылка на employer.needs.id
vacancy_id	bigint	NULL — логическая ссылка `[ext]`
message	text	NULL — описание предложения
company_name	text	NOT NULL
employer_contact	jsonb	NULL — снимок «способа связи» работодателя на момент создания (cross-schema join запрещён)
salary_from	numeric(12,2)	NOT NULL, CHECK >= 0
salary_to	numeric(12,2)	NOT NULL, CHECK >= salary_from
status	text	NOT NULL DEFAULT 'sent', CHECK ∈ (sent, viewed, accepted, declined, expired)
released_contact	jsonb	NULL — заполняется только при `contact_released`
contact_released_at	timestamptz	NULL
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: btree (candidate_id, status); btree (employer_id, status). UNIQUE: нет. Контакты раскрываются только после принятия/отклика (§12, roles.md п.2).

9.2. invitation_events

Назначение: append-only аудит переходов статусов; аудит §13.

Поле	Тип	Ограничения
id	bigserial	PK
invitation_id	bigint	NOT NULL, FK → invitations.id ON DELETE CASCADE
event_type	text	NOT NULL, CHECK ∈ (sent, viewed, accepted, declined, expired, contact_released)

Поле	Тип	Ограничения
actor_role	text	NULL, CHECK ∈ (employer, candidate, system)
payload	jsonb	NULL
occurred_at	timestamptz	NOT NULL DEFAULT now()

Индексы: btree (invitation_id, occurred_at). UNIQUE: нет (append-only).

9.3. [ext] vacancies, applications

vacancies — опубликованная вакансия работодателя; модуль `[ext]`.

Поле	Тип	Ограничения
id	bigserial	PK
employer_id	bigint	NOT NULL — логическая ссылка на auth.users.id
company_id	bigint	NULL — логическая ссылка на employer.companies.id
title	text	NOT NULL
description	text	NULL
specialization_code	text	NOT NULL
grade	text	NOT NULL
stack	text[]	NULL
salary_from	numeric(12,2)	NOT NULL, CHECK >= 0
salary_to	numeric(12,2)	NOT NULL, CHECK >= salary_from
status	text	NOT NULL DEFAULT 'published', CHECK ∈ (draft, published, closed)
created_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()
closed_at	timestamptz	NULL

Индексы: btree (employer_id, status); btree (specialization_code, grade). UNIQUE: нет.

applications — самостоятельный отклик кандидата на вакансию; модуль `[ext]`.

Поле	Тип	Ограничения
id	bigserial	PK
vacancy_id	bigint	NOT NULL, FK → vacancies.id ON DELETE CASCADE
candidate_id	bigint	NOT NULL — логическая ссылка на candidate.profiles.id
status	text	NOT NULL DEFAULT 'submitted', CHECK ∈ (submitted, viewed, accepted, rejected, withdrawn)
released_contact	jsonb	NULL — заполняется при раскрытии контактов (§12)
contact_released_at	timestamptz	NULL
applied_at	timestamptz	NOT NULL DEFAULT now()
updated_at	timestamptz	NOT NULL DEFAULT now()

Индексы: UNIQUE (vacancy_id, candidate_id); btree (candidate_id, status). UNIQUE: `(vacancy_id, candidate_id)`. Отклик раскрывает контакты (§12).

10. Схема notification

Владелец — `notification-service`. Исходящая почта; изоляция SMTP-креденшелов.

10.1. notification_log

Назначение: журнал исходящих email; аудит §13.

Поле	Тип	Ограничения
id	bigserial	PK
recipient_user_id	bigint	NULL — логическая ссылка на auth.users.id
recipient_email	text	NOT NULL
template_code	text	NOT NULL
event_type	text	NOT NULL
source_event_id	text	NULL, UNIQUE — идемпотентность консьюмера
status	text	NOT NULL DEFAULT 'queued', CHECK ∈ (queued, sent, failed)
provider_message_id	text	NULL
error	text	NULL
created_at	timestamptz	NOT NULL DEFAULT now()
sent_at	timestamptz	NULL

Индексы: UNIQUE source_event_id; btree recipient_user_id; btree status; btree created_at. UNIQUE: `source_event_id`.

11. Статусы и перечисления (сводный указатель)

Значения `CHECK` определены в таблицах-владельцах; здесь — только указатель, без дублирования. Доменные правила переходов — в mechanics-документах.

Домен	Поле	Раздел
Роль пользователя	auth.users.role	§3.1
Статус пользователя	auth.users.status	§3.1
Назначение email-токена	auth.email_tokens.purpose	§3.2
Тип согласия	auth.consents.consent_type	§3.4
Статус профиля	candidate.profiles.status	§4.1
Источник резюме	candidate.resumes.kind	§4.2
Статус парсинга	candidate.resumes.parse_status	§4.2
Статус связи ФСП	fsp.fsp_links.link_status	§5.1
История ФСП	fsp.fsp_links.history_status	§5.1
Тип задания банка	assessment.item_bank.item_kind	§6.1
Статус задания банка	assessment.item_bank.status	§6.1
Статус сессии	assessment.test_sessions.status	§6.1
Направление смены грейда	assessment.grade_history.direction	§6.1
Область cooldown	assessment.cooldowns.scope	§6.1
Статус промпта	assessment.EmployerTaskPrompt.status	§6.2
Действие промпта	assessment.EmployerTaskPrompt.action	§6.2
Статус задания	assessment.Task.status	§6.2
Формат задания	assessment.TaskVersion.format	§6.2
Статус выдачи	assessment.TaskAssignment.status	§6.2
Вид ответа	assessment.TaskSubmission.kind	§6.2
Результат проверки	assessment.TaskSubmission.result	§6.2
Вид справочника	selection.taxonomy.kind	§7.1
Статус потребности	employer.needs.status	§8.2
Статус приглашения	outreach.invitations.status	§9.1
Тип события приглашения	outreach.invitation_events.event_type	§9.2
Роль актора события	outreach.invitation_events.actor_role	§9.2
Статус вакансии `[ext]`	outreach.vacancies.status	§9.3

Домен	Поле	Раздел
Статус отклика `[ext]`	outreach.applications.status	§9.3
Статус уведомления	notification.notification_log.status	§10.1

12. Видимость данных и согласия (152-ФЗ)

Правила видимости обязательны (`docs/requirements/roles.md`, «Данные и правила видимости»); структура хранения — на стороне сервиса-владельца.

#	Правило	Где обеспечивается
1	Зарплата обязательна в вакансии и приглашении (диапазон «от–до», руб.).	outreach §9.1, §9.3
2	Контакты кандидата раскрываются работодателю только после принятия приглашения или самостоятельного отклика.	outreach §9.1 (`released_contact`, `contact_released_at`); событие `candidate.contact_released`
3	Приглашения и отклики имеют статусы; стороны видят статус только по своим записям.	outreach §9.1, §9.3
4	Регистрация по email с подтверждением; верификация компании и модерация вакансий не требуются (защита от недобросовестных работодателей — отдельная концепция).	auth §3.1–§3.2
5	Согласие на обработку и публикацию данных профиля фиксируется и аудируется.	auth.consents §3.4, §13
6	Настройки приватности («что видно») принадлежат кандидату.	candidate.privacy_settings §4.3, public_cards §4.4
7	Поисковая проекция не содержит PII; контакты в неё не попадают.	selection.candidate_cards §7.3

До раскрытия контакты хранятся только в `candidate.profiles` и не передаются работодателю. Событие `candidate.card_updated` несёт только согласованные поля, поэтому поисковый индекс структурно не может утечь PII (`docs/architecture.md §6.2`).

13. Аудит и история изменений

История — append-only там, где указано; чувствительные данные в журналах не дублируются.

Журнал	Таблица	Что фиксирует	Раздел
История грейдов	assessment.grade_history	смены грейда/категории, направление, сессию	§6.1
События приглашений	outreach.invitation_events	переходы статусов, актора, время	§9.2
Согласия	auth.consents	grant/revoke по типам согласий и версии политики	§3.4
Soft-delete аккаунтов	auth.users.deleted_at	пометка удаления аккаунта и отзыв доступа	§3.1
Уведомления	notification.notification_log	факт, статус и ошибки отправки email	§10.1

Событийная идемпотентность обеспечивается полем `source_event_id` консьюмеров (§7.2, §10.1) и уникальными ключами (`docs/architecture.md §4.2`); outbox/схема конверта — там же. Журналы не содержат паролей, токенов и контактных данных в открытом виде.

Модель грейд-тестирования

Назначение документа

Документ описывает сквозную механику грейд-тестирования кандидата: конвейер «опрос по отрасли и специализации → самостоятельный выбор предполагаемого грейда → тест на заявленный грейд → автоматическое присвоение категории», устройство банка заданий, адаптивную маршрутизацию, скоринг, правила повышения и понижения грейда, а также меры целостности и сопоставимости.

Основание: `docs/requirements/testing.md` (механика тестирования, устойчивость к распространению заданий, формат заданий, повышение/понижение грейда, ограничение смены грейда), `docs/requirements/overview.md` (Постановка задачи), `docs/requirements/roles.md` (роль «Кандидат»).

Связанные документы: `docs/data_model.md` (§6.1 — сущности тестирования), `docs/selection.md` (семантика категории и ранжирование), `docs/periodic_tasks.md` (регулярные задания работодателей), `docs/validation.md` (процедура валидации решения), `docs/user_flow.md` (пользовательские шаги), `docs/mvp_scope.md` (границы MVP), `docs/assumptions.md` (зафиксированные решения и параметры), `docs/glossary.md` (термины).

Вне рамок: определение категории, внутрикатегорийное ранжирование, формирование подборки и сценарий выхода на контакт — в `docs/selection.md`; ER-модель и структуры хранения сущностей — в `docs/data_model.md` (§6.1); механика регулярных заданий работодателей — в `docs/periodic_tasks.md`.

1. Обзор конвейера

Конвейер обязателен при регистрации: категория кандидата определяется не самоописанным резюме, а прохождением всех этапов (`docs/requirements/overview.md`, Постановка задачи; `docs/requirements/testing.md`).

#	Этап	Действие	Результат
1	Опрос	Опрос по отрасли и специализации	Заполненный профиль домена (`specialization`, охватываемые измерения)
2	Самообъявление грейда	Кандидат выбирает предполагаемый грейд	`declared_grade` — цель тестирования
3	Тест	Сборка формы попытки и адаптивное прохождение	Оценка `θ` и `SE`
4	Скоринг	Отображение `θ + SE` в полосы грейдов по cut-score	Полоса → кандидатская категория
5	Присвоение	Публикация события `assessment.category_assigned`	Категория видна работодателю

Инварианты конвейера:

- Тест проводится **на заявленный грейд**; кандидат не может быть понижен принудительно (§7.2).
- Результат теста автоматически определяет категорию, видимую работодателю (`docs/requirements/roles.md`, «Кандидат»).
- Конвейер идемпотентен на уровне попытки: пересборка формы не меняет уже сохранённый `seed` попытки (§3.4).
- Владелец данных — `assessment-service`; схема `assessment` (`docs/data_model.md` §6.1).

> **Примечание о демо-вакансиях.** Пункт ТЗ «подготовить несколько описаний вакансий и на их основе показать, как система формирует задания» трактуется так: render-представления демо-вакансий — это **seed-данные демо**; генерация заданий/теста выполняется из **калиброванного банка заданий** по специализации (шаблоны и якоря, §3), а не из текстов вакансий. Вакансии служат лишь демонстрационным контекстом, а не источником генерации заданий.

2. Входные этапы

2.1. Опрос

- Опрос по отрасли и специализации заполняется до теста (`docs/requirements/testing.md`).

- Результат опроса задаёт `specialization` кандидата и набор **заявленных измерений**, которые обязана покрывать форма попытки (§4).
- Ответы опроса — вход для подбора пула семейств шаблонов (§3.4) и границ категорий (§6.2).
- Хранение опроса: `test_sessions.survey_answers` и `test_sessions.declared_dimensions` (`docs/data_model.md` §6.1); отрасль кандидата — `candidate.profiles.industry_code` (`docs/data_model.md` §4.1).

2.2. Самообъявленный грейд

- Кандидат самостоятельно выбирает предполагаемый грейд (`docs/requirements/overview.md`, Постановка задачи).
- `declared_grade` задаёт целевую сложность: стартовая точка `θ` маршрутизации (§5.1) и набор полос для отображения (`§6.2`).
- Самообъявление не является результатом: итоговый грейд определяется оценкой `θ` по cut-score, а не выбором кандидата.

3. Банк заданий

3.1. Форматы заданий

Содержание и формат заданий команда определяет сама; допустимы задачи с кодом, опросники и иные форматы (`docs/requirements/testing.md`, «Формат заданий»). В продукте закрытые форматы проверяются **автоматически**, а открытые (свободный ответ, подход) — **вручную либо полуавтоматически**:

Формат	Проверка	Примечание
Одиночный/ множественный выбор	Автоматическая	Ключ задаётся шаблоном, не LLM (§3.6)
Короткий ответ (число/ строка)	Автоматическая	Ключ нормализуется вручную
Задача с кодом	Автоматическая (тесты) + проверка схожести	Схожесть — §9.1
Свободный ответ / подход	**Вне MVP-формы грейд-теста**	В MVP живёт в периодических заданиях работодателей (`docs/periodic_tasks.md`); extension [ext] — §3.6, §9.2

- **Форма MVP-грейд-теста состоит только из автопроверяемых форматов** (закрытые вопросы, короткий ответ, задачи с кодом). Свободный ответ («предложить подход») в MVP-форму грейд-теста не входит и остаётся в продукте только внутри периодических заданий работодателей, где оценивает работодатель (`docs/periodic_tasks.md`).
- Пул заданий грейд-теста **строго отделён** от пула регулярных заданий (`docs/periodic_tasks.md`) — общими являются только инфраструктурные примитивы.
- Ключ оценивания (правильный ответ) никогда не попадает в клиент.

3.2. Структура задания: шаблон-«радикал» и параметры-«инциденталии»

Задание описывается кортежем `(concept, template, parameters, difficulty)`:

- **Шаблон («радикал»)**, template family — неизменяемая структура задания: формулировка, правило решения, логика ключа.
- **Параметры («инциденталии»)**, incidentals — переменные значения внутри шаблона (числа, имена, данные примеров). Меняют поверхность, не сложность.
- Варианты одного шаблона — **изоморфы**: различаются поверхностью, сохраняют предназначенную сложность `b`.
- Для каждой попытки сохраняется **уникальный seed**, с которым сгенерированы параметры (`docs/data_model.md` §6.1). По seed задание воспроизводимо.

Такое разделение снимает конфликт «утечка ↔ сопоставимость»: поверхностная уникальность достигается генерацией, структурная сложность остаётся общей.

3.3. Якорные задания

- **Якорные задания**, anchor items — небольшая защищённая подборка, включённая в **каждую** форму попытки.
- Сложность якорей подтверждена экспертами и фиксирована; они не генерируются из шаблонов.
- Якоря служат основой эквайтинга между попытками (§9.3) и обнаружения дрейфа при пересчёте (§3.5).

- В адаптивном режиме раскрытие ответов и параметров якорей недопустимо — используется защищённый показ без adaptive-выдачи (§9.1).

3.4. Семейства шаблонов и уникальные seed вариантов

- Банк двухслойный: (а) защищённые якорные задания (§3.3); (б) крупные **семейства шаблонов**, дающие неограниченный поток вариантов.
- Каждое семейство порождает изоморфы с общим `b`; параметры варианта определяются per-item seed.
- Различаются два уровня seed: **seed формы** `test_sessions.form_seed` (form-level, воспроизводимость сборки формы) и **seed варианта** `attempts.variant_seed` (per item, параметры конкретного задания). Повторная выдача того же параметрического варианта одному кандидату не допускается: `assessment-service` ведёт журнал выдач в разрезе «кандидат × семейство шаблонов/seed» и не переиздаёт уже выданный вариант (`docs/data_model.md` §6.1).
- Число доступных вариантов на семейство шаблонов и лимиты выдачи регулируются **exposure caps** на уровне семейства/задания (`item_bank.exposure_limit` / `item_bank.exposure_count`), а не отдельного несуществующего варианта (§9.1).

3.5. Начальная калибровка и пересчет (provisional loop)

- Начальная сложность заданий задаётся **экспертной оценкой** и помечается `provisional`.
- Пересчёт (refit) сложностей выполняется по логам ответов с опорой на **якорные задания** (§3.3); якоря связывают шкалы попыток и позволяют обнаруживать **дрейф** сложности.
- До накопления достаточного объёма данных не заявляется точность IRT: сложности остаются провизорными.

3.6. Политика применения LLM

- LLM допускается **только** для заполнения поверхностных инцидералей внутри уже зафиксированной модели задания (шаблона).
- LLM **никогда** не определяет ключ оценивания (правильный ответ) и не назначает сложность `b`.
- Полностью сгенерированные LLM задания, оценка которых также делегирована LLM, запрещены (см. §9.5).
- **Оценка свободных ответов LLM** допускается **только как [ext]-расширение** (вне MVP): облачный LLM-API (российский), с пиннингом версии модели и логом хеша промпта **на каждый вердикт**, `temperature = 0`, ответ структурированным JSON строго по фиксированной человеческой рубрике. LLM и в этом режиме **никогда** не определяет ключ оценивания и не назначает сложность `b`.
- **Локальный хостинг LLM исключён из MVP** требованиями к железу (`docs/requirements/nonfunctional.md`).

4. Сборка формы попытки

Форма попытки собирается под конкретного кандидата и его `declared_grade`. Правила сборки:

- **Покрытие заявленных измерений**: форма обязана покрывать измерения, заявленные в опросе (§2.1), без перекоса в одно.
- **Не более одного варианта на семейство** в форме — предотвращает дублирование логики решения.
- **Перемешивание**: порядок заданий и порядок вариантов ответа перемешиваются для каждой попытки.
- **Exposure caps**: лимит показов задаётся на уровне семейства шаблонов / задания (`item_bank.exposure_limit` / `item_bank.exposure_count`); при исчерпании задание (семейство) выводится из ротации (§9.1).
- **Якоря**: в форму всегда включается защищённый якорный набор (§3.3).
- **Seed**: для каждой формы сохраняется form-level `test_sessions.form_seed` (воспроизводимость сборки и разбирательство); параметры отдельных заданий — per-item `attempts.variant_seed` (§3.4).
- Собранная форма фиксируется в сущности попытки (`docs/data_model.md` §6.1).

5. Адаптивная маршрутизация (IRT-lite, 1PL)

Маршрутизация — упрощённая однонаправленная модель IRT (1PL, Rasch). Это осознанное упрощение для MVP: параметры различения не оцениваются.

5.1. Оценка θ и выбор следующего задания

- Вероятность правильного ответа:

```
p(correct) = 1 / (1 + exp(-(θ - b)))
```

- `θ` — уровень кандидата; `b` — сложность задания; старт — со среднего уровня относительно `declared_grade`.
- После каждого задания `θ` обновляется **упрощённым байесовским шагом** (для MVP); переход на MLE/LBFGS — в расширениях.
- Следующее задание выбирается с сложностью, ближайшей к текущей оценке: минимум `|θ - b|` среди доступных вариантов.

5.2. Правила остановки

- Остановка происходит при достижении **порога стандартной ошибки** `SE`.
- Одновременно действуют **минимум и максимум числа заданий**: форма не завершается раньше минимума и принудительно завершается на максимуме.
- Значения порога `SE` и границ числа заданий — документированные константы (`docs/assumptions.md`).

5.3. Fallback: многоэтапное тестирование (MST)

Если адаптивная маршрутизация недоступна или упрощается для MVP, применяется многоэтапное тестирование, **multistage testing**:

- 2–3 фиксированные панели разной сложности;
- переключение между панелями по результату предыдущего этапа (routing);
- сохраняются якоря (§3.3) и правила остановки по объёму (§5.2).

6. Оценивание: скоринг → грейд

6.1. Отчет θ + SE (не проценты)

- Результат теста выражается **оценкой `θ` и её стандартной ошибкой `SE`**.
- Процент правильных ответов **не** является итоговой метрикой и **не** отображается как результат.
- `θ`, `SE` и связанные данные попытки сохраняются (`docs/data_model.md` §6.1) для аудита и пересчёта.

6.2. Cut-score и категории-полосы

- Шкала `θ` разбивается на **полосы грейдов** документированными **cut-score**.
- Полоса отображается в кандидатскую категорию: категория = специализация × грейд; семантика категории — в `docs/selection.md`.
- Cut-score документируются для прозрачности и справедливости (`docs/assumptions.md`).

6.3. «Требует перепроверки» (SE завышен / CI на границе полосы)

- Если `SE` завышен **или** доверительный интервал `θ` пересекает границу полосы — попытка помечается как **«требует перепроверки»**, а не принудительно относится к грейду.
- В спорных случаях решение принимает **человек в контуре** (ручная проверка); автоматика не выносит пограничный вердикт.

7. Повышение и понижение грейда

7.1. Правила up/down

- Кандидат, не прошедший тест на заявленный уровень, может пройти тест на уровень ниже; кандидат, справившийся уверенно, — повысить грейд (`docs/requirements/testing.md`, «Повышение и понижение грейда»).
- Повышение и понижение выполняются **по инициативе кандидата** через новую попытку тестирования; автоматического пересмотра нет.
- Категория (= специализация × грейд) пересчитывается по результату **любой** завершённой попытки, включая up/down и повторные попытки.

- Контракт событий: **любая завершённая попытка перепубликует** `assessment.category_assigned` с актуальной категорией; для проекции **категории** `selection` потребляет это событие (полный список консьюмеров — `docs/architecture.md §3.1`). `assessment.grade_changed` — **audit/notification only** и на обновление категории не влияет.
- Изменения грейда фиксируются в истории (`docs/data_model.md` §6.1), публикуется событие `assessment.grade_changed` (аудит/уведомление).
- В MVP повышение/понижение грейда опирается **только на автопроверяемые форматы** (закрытые вопросы, короткий ответ, код); свободный ответ в грейд-тест не входит (§3.1).

7.2. Запрет принудительного понижения

- По результату теста грейд **не понижается принудительно** (`docs/requirements/overview.md`, Постановка задачи).
- Провал теста на заявленный грейд не отображает кандидата в более низкий грейд автоматически: система лишь предлагает пройти тест на уровень ниже.
- Кандидат, не прошедший тест на заявленном грейде, **не имеет категории**, пока не пройдёт тест хотя бы на каком-либо грейде; UI при этом подсказывает более низкий уровень.

7.3. Cooldown = 1 месяц

- Смена грейда ограничена по времени: **cooldown = 1 месяц** (утверждённое решение команды; требование допускает выбор интервала — `docs/requirements/testing.md`, «Ограничение смены грейда»).
- Cooldown контролируется через таблицу `cooldowns` в схеме `assessment` (`docs/data_model.md` §6.1), а не через поля истории грейда.
- Повторная попытка смены грейда до истечения cooldown недоступна; правило отражено в API проверки права (`/assessment/grade-eligibility`).

8. Присвоение категории

- По результату теста определяется полоса грейда и **присваивается категория** кандидата — связка специализации и грейда (§6.2); при `needs_review` — ручное решение (§6.3).
- Категория (= специализация × грейд) обновляется по итогу **любой** завершённой попытки, включая повышение/понижение грейда и повторные попытки.
- В MVP категория выводится только из **автопроверяемых форматов** (§3.1); свободный ответ в грейд-тест не входит и на присвоение категории не влияет.
- Завершением конвейера считается публикация события **`assessment.category_assigned`** (transactional outbox; `docs/architecture.md` §4.3); каждая завершённая попытка перепубликует это событие с актуальной категорией; для проекции **категории** `selection` потребляет его (полный список консьюмеров — `docs/architecture.md §3.1`).
- `assessment.grade_changed` — **audit/notification only**: на проекцию категории в `selection` не влияет (§7.1).
- Событие транслируется в `selection-service` как проекция категории; до его публикации кандидат невидим в поиске.
- Документ завершается на этом событии: определение категории, внутрикатегорийное ранжирование, подборка и выход на контакт — вне рамок и описаны в `docs/selection.md`.
- Категория — единственный сигнал грейд-тестирования, видимый работодателю; результаты попытки и `θ` работодателю не раскрываются.

9. Целостность и сопоставимость

9.1. Защита от утечек

- **Уникальные per-item seed** вариантов (`attempts.variant_seed`, §3.4) — поверхность задания не повторяется.
- **Exposure caps** — лимит показов на уровне семейства шаблонов / задания (`item_bank.exposure_limit` / `item_bank.exposure_count`), вывод из ротации при исчерпании.
- **Перемешивание** порядка заданий и вариантов ответа (§4).
- **Проверка схожести** свободных ответов и кода: принимается готовый инструмент (JPlag/Dolos), собственный детектор не разрабатывается.
- Ключи оценивания не попадают в клиент; в адаптивном режиме ответы не раскрываются.
- Принимаемый остаточный риск: кратковременная утечка конкретного задания (семейства шаблонов) ограничена ротацией и exposure caps.

9.2. Прокторинг-флаги и человеческая проверка

- Фиксируются прокторинг-флаги с отметками времени:
  - `visibilitychange` — уход со страницы;
  - выход из полноэкранного режима (`fullscreen exit`);
  - вставка из буфера (`paste`).
- Флаги формируют сигнал для **человеческой проверки**, а не вердикт.
- **Автоматический провал по одному флагу запрещён** («never auto-fail»); решение принимает человек в контуре.
- **Очередь человеческой проверки свободных ответов [ext]**: пакетная проверка в рабочие часы со SLA **24–48 рабочих часов** (не 24/7).
- **Автоматический вердикт** по свободному ответу [ext] допустим только на порогах **≥ 8/10** или **≤ 3/10**; всё, что между, а также **любой прокторинг-флаг → человек в контуре**.
- **Fallback при недоступности API**: задание исключается из скоринга, попытка пересчитывается (безопасно за счёт якорей и эквайтинга, §9.3).

9.3. Эквайтинг по якорным заданиям

- Якорные задания (§3.3) включены в каждую форму и связывают попытки между собой.
- По ответам на якоря выполняется **эквайтинг** — приведение оценок разных попыток к общей шкале.
- Якоря также служат опорой при пересчёте сложностей и обнаружении дрейфа (§3.5).

9.4. Сопоставимость кандидатов: θ как общая шкала

- Сопоставимость обеспечивается отчётом **`θ` (и `SE`), а не процентом** правильных ответов (§6.1).
- Общая шкала `θ` позволяет сравнивать кандидатов, проходивших разные формы, в том числе с разными наборами параметрических вариантов.
- Полосы cut-score (§6.2) переводят общую шкалу в категории.

9.5. Антипаттерны

- Единый фиксированный набор заданий для всех кандидатов.
- Полностью LLM-сгенерированные задания, оцениваемые LLM.
- Отчёт в процентах как итоговый результат.
- Заявление точности IRT на малых выборках.
- Отсутствие контроля показов (exposure control).
- Автоматический провал по одному прокторинг-флагу.
- Расчёт только на перемешивание как единственную защиту.
- Таймированный полный CAT без минимума/максимума заданий.
- Игнорирование политики применения LLM (§3.6).
- Свободный ответ в MVP-грейд-тесте без гарантированного механизма проверки.
- Переиспользование seed параметров и раскрытие ответов в адаптивном режиме.
- Публичные глобальные лидерборды.
- Автоматизация кадровых решений без человека в контуре.

Регулярные задания работодателей

Назначение документа

Документ описывает механику регулярных коротких заданий от работодателей: каденс и синхронные окна, жизненный цикл задания работодателя, добровольную подписку кандидата, случайный розыгрыш задания, формы ответа и оценку, атрибут свежести профиля, сигнал работодателю и защиту от «игры с механикой».

Связанные документы: `docs/testing_model.md` (граница механики грейд-тестирования), `docs/data_model.md` (сущности и поля), `docs/selection.md` (категории и ранжирование), `docs/mvp_scope.md` (границы MVP), `docs/user_flow.md` (сквозные сценарии), `docs/assumptions.md` (реестр решений), `docs/glossary.md` (термины).

Источники: `docs/requirements/testing.md` («Регулярные задания»), `docs/requirements/roles.md` (роли «Кандидат», «Работодатель»), `docs/requirements/overview.md` (постановка задачи).

Вне рамок: механика грейд-тестирования и присвоения категории (`docs/testing_model.md`), формула ранжирования внутри категории (`docs/selection.md`), физическая схема сущностей и DDL (`docs/data_model.md`).

1. Обзор механики и задействованные сущности

Помимо первичного тестирования, определяющего грейд и профиль, система периодически предлагает кандидату короткую задачу от работодателя: решить её или предложить подход к решению. Механика поддерживает профиль кандидата в актуальном состоянии и даёт работодателю свежий сигнал о кандидате — «желательно, но не обязательно для MVP» (`docs/requirements/roles.md`).

> **Жёсткое правило (hard rule).** Результаты регулярных заданий НИКОГДА не изменяют грейд, категорию и историю грейда (`grade_history`). Единственное, что обновляется по итогам задания, — атрибут свежести профиля `ProfileFreshness`. См. §6.3.

Сущности механики — `TaskCycleConfig`, `EmployerTaskPrompt`, `Task`, `TaskVersion`, `TaskAssignment`, `TaskSubmission`, `TaskInterestSubscription`, `ProfileFreshness` — определены (поля, ключи, ограничения, UNIQUE) в `docs/data_model.md §6.2–6.3`. Здесь их состав не дублируется; сущности используются только по ссылке. `ProfileFreshness` размещается в схеме `assessment` (см. `docs/data_model.md §6.3`).

Жёсткая граница с `docs/testing_model.md`. `docs/testing_model.md` владеет первичным конвейером (опрос → заявленный грейд → тест), анти-утечкой и сопоставимостью, скорингом → грейдом, запретом принудительного понижения и cooldown, присвоением категории. Настоящий документ владеет жизненным циклом заданий работодателя, подпиской, розыгрышем, ответами, свежестью и сигналом работодателю. Интерфейс: чтение факта «у кандидата есть категория»; записи в грейд/категорию/cooldown — отсутствуют. Общие примитивы — только инфраструктурные (планировщик, таксономия форматов заданий, UI ответа) и заимствованные принципы анти-утечки. Пулы задач строго отделены от банка заданий грейд-теста.

2. Каденс и синхронные окна (A)

Параметр	Значение	Примечание
`n`	`7` дней	глобальная константа платформы
Дедлайн `d`	`n`	срок ответа на выданное задание
Окна	глобальные синхронные	не per-account rolling
TZ	единая платформенная	timestamps в UTC, границы окон — в TZ платформы

- **A1.** Глобальная константа `n = 7` дней управляет обоими каденсами: промптами работодателя по вакансиям и выдачами заданий кандидатам.
- **A2.** Окна глобальные синхронные (не индивидуальные rolling-окна) — это обеспечивает идемпотентность через `cycle_date`.
- **A3.** Ежедневный планировщик открывает и продвигает циклы; все записи — идемпотентные upsert-ы, повторный запуск или сбой планировщика не создаёт дублей.
- **A4.** Единая таймзона платформы: timestamps хранятся в UTC, границы циклов отсчитываются в TZ платформы; таймзона документируется (см. `docs/assumptions.md`).

3. Цикл задания работодателя (B)

3.1. Промпт {add | update | keep}

- **B1.** В момент открытия окна для каждой активной вакансии создаётся промпт с выбором действия `{add | update | keep}`; уникальность `UQ(vacancy_id, cycle_date)` гарантирует ровно один промпт на вакансию в окне (см. `docs/data_model.md §6.2`).
- **B3.** Неразрешённый промпт истекает при закрытии окна, при этом задание остаётся активным. Отложенное расширение: пометка «устаревшее» после `m` проигнорированных промптов (`[ext]`, см. §9).

3.2. Версии задания (TaskVersion)

- **B2.** Действия над заданием:
  - `add` → создаётся задание `Task` версии v1;
  - `update` → создаётся новая неизменяемая (`immutable`) `TaskVersion` с `version_no++`, предыдущие версии сохраняются;
  - `keep` → явное подтверждение без новой версии, инкремент `keep_count`.
- Версия, выданная кандидату, неизменяема: правки не задним числом, а новой версией.

3.3. Отставление задания (retire)

- **B4.** Работодатель может отставить (`retire`) задание; то же происходит автоматически при закрытии вакансии. Отставленное задание исключается из новых выдач, но уже открытые назначения доводятся до дедлайна против выданной версии (см. §8).

3.4. Запрет адресной выдачи

- **B5.** Работодатель НЕ может адресно выдать задание конкретному кандидату — только системный случайный розыгрыш (§5). Это предотвращает обход LHH-механики выхода на контакт и гейтинга контактов (`docs/selection.md`, `docs/requirements/roles.md`, правило 2).

4. Подписка кандидата (C)

- **C1.** Участие — opt-in, переключатель по умолчанию ВЫКЛЮЧЕН (`DEFAULT OFF`). Тип согласия — `task_participation` (`docs/data_model.md §3.4`), покрывает участие и отображение результата; отписка прекращает будущие выдачи.
- **C2.** Участие возможно только при наличии завершённой первичной категории (пройдены опрос и тест). Без категории кандидат в розыгрыш не попадает.
- **C3.** Пул задач ограничивается специализацией кандидата. `[ext]` Фильтр по грейду — опционально.

5. Розыгрыш задания (D)

5.1. Одна выдача на окно

- **D1.** Раз за окно каждый подписанный и правомочный кандидат получает ровно одно задание, выбранное равномерно случайно (`uniform-random`) из множества: активное ∧ (активная вакансия ∨ активная «потребность» работодателя через `Task.need_id` — fallback-якорь, `docs/assumptions.md §11`) ∧ обязательное совпадение специализации ∧ ещё не проходил ∧ не заблокировано «устаревшим» заданием. Уникальность `UQ(candidate_id, cycle_date)`.

5.2. Дедупликация на уровне задания

- **D2.** «Непройденное» определяется на уровне ЗАДАНИЯ (task-level): если кандидат пытался решить задание в любой из его версий, оно больше никогда не предлагается.

5.3. Дедлайн d = n и возврат в пул

- **D3.** Дедлайн ответа `d = n`. При отсутствии ответа задание помечается `expired` и возвращается в доступный пул. `[ext]` Лимит повторных предложений (`re-offer cap`) = 2.

5.4. Пустой / исчерпанный пул; лимит повторов [ext]

- **D4.** Если пул пуст или исчерпан, выдача не создаётся, кандидат видит «задач нет», повторная попытка — в следующем окне. `[ext]` Ограничение повторов (§5.3) сужает пул дополнительно.

6. Выполнение и свежесть профиля (E)

6.1. Формы ответа: «решить» / «предложить подход»

- **E1.** Две формы ответа: «решить» или «предложить подход к решению». Обе считаются успешной сдачей (`passed`). `[ext]` Минимальный порог усилия (`min-effort floor`).

6.2. Оценка работодателем

- **E2.** Оценка выполняется работодателем вручную (автоматически — где формат это позволяет). Результат: `positive` / `negative` / `unreviewed`.

6.3. Свежесть профиля: display-only

> **Жёсткое правило (hard rule).** Результаты заданий НИКОГДА не изменяют грейд, категорию и историю грейда (`grade_history`); обновляется только атрибут свежести `ProfileFreshness`.

- **E3.** Записи в грейд/категорию/`grade_history` по итогам задания запрещены (повтор §1).
- **E4.** При сдаче обновляется `ProfileFreshness.last_task_activity_at = now`; отображается бейдж «актуальный профиль: {дата} / N задач».
- **E5.** Свежесть по умолчанию — display-only: НЕ входит в ранжирование категории. `[ext]` Опциональный ограниченный recency-decay требует отдельного согласования (sign-off).

6.4. Сигнал работодателю

- **E6.** Представление сигнала показывает работодателю недавнюю активность по заданиям. Данные лежат в схеме `assessment`; у `employer-service` проекции нет, событие не определено. Путь чтения задаётся одним из двух вариантов: (а) `[opt]` внутренний sync-GET из `employer-service` в `assessment-service` — вызов не на горячем пути, поэтому согласуется с минимизацией внутренних синхронных вызовов (`docs/architecture.md §4.1`); либо (б) проекция представления в схеме работодателя, наполняемая событием `[ext]`. Контактные данные кандидата остаются закрытыми — гейтинг контактов не меняется (`docs/requirements/roles.md`, правило 2; `docs/selection.md`).

7. Защита от «игры с механикой» (F)

- **F1.** Одна выдача на кандидата в окно (`UQ(candidate_id, cycle_date)`).
- **F2.** Задания привязаны к вакансиям; адресная выдача кандидату запрещена.
- **F3.** Свежесть по умолчанию не даёт рангового выигрыша (§6.3).
- **F4.** Утечка содержания заданий принята как остаточный риск: задания короткие и обновляются каждые `n` дней; требование анти-утечки относится к грейд-тесту, а не к этой механике.
- **F5.** Минимальный порог усилия применяется, если свежесть становится видимой.

8. Граничные случаи

Ситуация	Поведение
Нет вакансий	механика спит (dormant)
Вакансия без задания	промпт предлагает `add`
Кандидат не подписан	выдачи нет
Пустой пул	«задач нет», повтор в следующем окне
Исчерпанный пул	как пустой (+ лимит повторов `[ext]`)
Работодатель игнорирует промпт	задание остаётся активным (`[ext]` stale)
Работодатель деактивирован	задания отставлены, открытые назначения истекают

Ситуация	Поведение
Задание отставлено/обновлено при открытом назначении	довести до дедлайна против выданной версии; блокировать только новые выдачи
Вакансия закрыта	как отставление задания
Кандидат сменил специализацию в середине цикла	текущее назначение — по-старому; следующий розыгрыш — по новому scope
Аккаунт удалён	soft-delete аккаунта; логи по заданиям анонимизируются (персональные ссылки разрываются, агрегаты сохраняются) — политика удаления зафиксирована в `docs/assumptions.md`
Планировщик запущен дважды / упал	уникальные ключи → no-op (идемпотентность)
Таймзоны	единая TZ платформы, документируется

9. MVP и отложенные расширения

Вся механика — optional-MVP (критерий 6, 7%), но проектируется полностью уже сейчас (см. `docs/mvp_scope.md`).

- **Минимальный срез:** B1–B4, C1–C2, D1–D4, E1–E4, F1/F2.
- **Отложено:** автоматическое истечение «устаревших» заданий, `[ext]` фильтры scope, ранговый recency-decay, лимиты повторов (`re-offer cap`), автоматизация `min-effort`, email-уведомления, персональные таймзоны.

Флаг зависимости. Т3 привязывает задания к вакансиям, но публикация вакансий сама по себе optional-MVP. Fallback `[ext]`: якорить задание к «описанию потребности» (`needs`) или к компании, а не к опубликованной вакансии. Решение фиксируется в `docs/assumptions.md §11`.

Подбор и выход на контакт

Назначение документа

Документ описывает механику подбора PullHire: категории кандидатов, ранжирование внутри категории, формирование подборки под потребность работодателя, поиск и фильтры по банку кандидатов, а также LHH-механику выхода работодателя на контакт — приглашение конкретному кандидату и раскрытие контактов.

Основание: `docs/requirements/selection.md` (категории, справочник специализаций и грейдов, ранжирование, формирование подборки, выход на контакт, объяснимость, поиск и фильтрация), `docs/requirements/overview.md` (Постановка задачи, LHH), `docs/requirements/roles.md` (роль «Работодатель», правила видимости 1–3), `docs/requirements/evaluation.md` (критерий 1 «Новая механика подбора», 35 %).

Связанные документы: `docs/testing_model.md §8` (присвоение категории и событие `assessment.category_assigned`), `docs/data_model.md §7` (схема `selection`: таксономия, проекции, сигналы ранжирования), `docs/data_model.md §11` (сводный указатель статусов и перечислений), `docs/data_model.md §9.1` (приглашение и снимок способа связи), `docs/architecture.md §4` (события, transactional outbox, итоговая согласованность), `docs/fsp_integration.md §5` (отображение достижений), `docs/api.md §3.5` (эндпоинты `selection-service`), `docs/mvp_scope.md` (границы MVP), `docs/assumptions.md` (реестр решений и параметров), `docs/glossary.md` (термины), `docs/backlog/closed_questions.md` (журнал закрытых вопросов и техдолга).

Вне рамок: механика грейд-тестирования и присвоение категории (`docs/testing_model.md`), физическая схема сущностей, полей и перечислений (`docs/data_model.md §7`, `§9.1`, `§11`), механика регулярных заданий работодателей (`docs/periodic_tasks.md`).

1. Обзор механики

Платформа связывает напрямую только работодателя и соискателя. Онбординг кандидата (опрос → заявленный грейд → тест) завершается публикацией события `assessment.category_assigned`; с этого момента кандидат становится видимым. Работодатель описывает потребность, получает подборку с объяснимостью и сам выходит на контакт, направляя приглашение конкретному человеку — инициатива принадлежит работодателю (LHH), а не кандидату.

Ключевые инварианты механики:

- **Кандидат невидим работодателям до события `assessment.category_assigned`.** До присвоения категории он не попадает ни в поиск, ни в подборку (`docs/testing_model.md §8`).
- **Выдача строится на категориях, а не на самоописанном резюме.** Поиск и подборка опираются на присвоенную категорию, подтверждённую тестом, а не на ключевые слова резюме (`docs/requirements/selection.md`, «Поиск и фильтрация банка кандидатов»).
- **Контактные данные кандидата закрыты** до принятия приглашения или самостоятельного отклика кандидата (`docs/requirements/roles.md`, правило 2).
- **Свежесть профиля не влияет на позицию** в ранжировании (§3).

Текстовая схема потока:

```
Работодатель:  описание потребности
               -> POST /selections (selection-service читает свои проекции)
               -> подборка: рекомендованные категории + кандидаты + объяснимость
               -> поиск и фильтры по банку
               -> приглашение: описание, зарплата (обязательно), способ связи
Кандидат:      принятие приглашения
               -> invitation.accepted -> candidate.contact_released
               -> контакты раскрыты работодателю
```

Владельцы контекстов: `selection-service` — каталог (`taxonomy`), проекции категорий и карточек, ранжирование и формирование подборки; `outreach-service` — приглашения, их статусы и раскрытие контактов. Согласованность проекций — итоговая, через события (`docs/architecture.md §4.3`).

2. Категории кандидатов

Категория — это группа однотипных соискателей, в основе которой связка **специализации и грейда** (например, «бэкенд-разработчики с опытом», «фронтенд-разработчики с опытом до года»). Формально `category = specialization × grade`.

- Категория **не выводится из самоописанного резюме**. Она определяется обязательным путём при регистрации: опрос по отрасли и специализации, самостоятельный выбор предполагаемого грейда и тест на заявленный грейд; результат автоматически определяет категорию, видимую работодателю (`docs/requirements/selection.md`, «Категории кандидатов»).
- Справочник специализаций и грейдов команда составляет самостоятельно, опираясь на общепринятые в ИТ понятия; на этапе MVP достаточно основных направлений. Справочник хранится в `selection.taxonomy` (`docs/data_model.md §7.1`); его состав — решение команды (`docs/assumptions.md`).
- Проекция текущей категории кандидата — `selection.candidate_categories` (`docs/data_model.md §7.2`); она наполняется событием `assessment.category_assigned` и является источником для поиска и подборки.
- Карточка кандидата в поисковой проекции (`selection.candidate_cards`, `docs/data_model.md §7.3`) не содержит PII: контактные данные в неё не попадают (`docs/data_model.md §12`).

Контракт события. Для проекции **категории** `selection-service` потребляет `assessment.category_assigned`; полный список консьюмеров — `docs/architecture.md §3.1`. Событие перепубликуется по **любой** завершённой попытке — включая повышение грейда, понижение грейда и повторные попытки, — с актуальной категорией. Событие `assessment.grade_changed` является **audit/notification only** и на обновление категории в `selection` не влияет (`docs/testing_model.md §7.1`, `§8`).

Кандидат без присвоенной категории (в том числе не прошедший тест на заявленный грейд) в поиске и подборке отсутствует. Понижение грейда не выполняется принудительно: система лишь предлагает пройти тест на уровень ниже, а категория появляется после любой успешно завершённой попытки (`docs/testing_model.md §7.2`).

3. Ранжирование внутри категории

Ранжирование выполняется **внутри** категории: сравниваются кандидаты одной специализации и одного грейда. Кандидаты разных категорий между собой по позиции не сопоставляются.

Входы ранжирования — **сила подтверждённого профиля**:

- **Подтверждённый результат теста** — оценка `θ` (и `SE`) как общая шкала (`docs/testing_model.md §6.1`, `§9.4`). Выше находится кандидат с более сильным подтверждённым уровнем.
- **Подтверждённые достижения ФСП** — наличие и количество проверяемых достижений участника соревнований и мероприятий ФСП (`docs/fsp_integration.md §5`).

Свойства ранжирования:

- **Свежесть профиля НЕ является входом ранжирования.** Результаты регулярных заданий обновляют только атрибут `ProfileFreshness`, который по умолчанию display-only и участия в позиции не принимает (`docs/periodic_tasks.md §6.3`, `docs/data_model.md §6.3`).
- **Самоописанные данные резюме** (самостоятельно указанный грейд, ключевые слова стека и т.п.) на позицию не влияют: ранжирование опирается на подтверждённый профиль. Стек используется для фильтрации (§5), а не для повышения позиции.
- Отсутствие истории ФСП нейтрально и **не является штрафом**; достижения — это дополнительное преимущество, а не обязательное условие (`docs/fsp_integration.md §6`).
- Компоненты ранжирования хранятся в `selection.rank_signals` (`docs/data_model.md §7.4`); конкретные веса и константы формулы — документированные параметры (`docs/assumptions.md`).
- Обновление достижений запускает событие `fsp.achievements_updated`, по которому карточка переранжируется (`docs/architecture.md §4.2`).

4. Формирование подборки

4.1. Вход: описание потребности

Входом формирования подборки служит **описание потребности** работодателя: какие специалисты нужны, чем занимается команда, требуемые стек, специализация и грейд (`docs/requirements/selection.md`, «Формирование подборки»; `docs/requirements/roles.md`, «Работодатель»). Потребность хранится в `employer.needs` (`docs/data_model.md §8.2`) и принадлежит `employer-service`.

Синхронный путь: потребность передаётся в `selection-service` запросом `POST /selections` (`docs/api.md §3.5`), который читает собственные проекции и возвращает подборку. Единственная синхронная цепочка «по требованию» в архитектуре — именно она (`docs/architecture.md §4.1`). Никаких cross-schema JOIN не выполняется: сервис использует только свои проекции (`docs/data_model.md §1`).

4.2. Правила формирования

- **Отбор.** В подборку попадают только категоризированные кандидаты, у которых специализация совпадает со специализацией потребности, а грейд соответствует требуемому. Кандидаты без категории не рассматриваются (§2).
- **Фильтрация по стеку.** Требуемый в потребности стек уточняет набор кандидатов; совпадение стека — фильтр, а не сигнал позиции (§3).
- **Ранжирование.** Кандидаты ранжируются внутри категории по силе подтверждённого профиля (§3); в выдачу попадает верхняя часть ранжированного списка (top-N).
- **Две формы выдачи.** Подборка показывает и **рекомендованные категории**, и **конкретных кандидатов** (`docs/requirements/selection.md`, «Работа с подборкой кандидатов»).
- **Уточнение без потери.** Фильтрация и уточнение запроса выполняются **без потери ранее полученной подборки** (`docs/requirements/selection.md`, «Поиск и фильтрация банка кандидатов»).
- **Состав выдачи** — категория, специализация, грейд, подтверждённые достижения ФСП; результаты попытки и `θ` работодателю не раскрываются (`docs/testing_model.md §8`).

4.3. Объяснимость включения и позиции

Каждый кандидат в подборке сопровождается **краткой объяснимостью**: почему он попал в выдачу и почему стоит на своей позиции (`docs/requirements/selection.md`, «Объяснимость ранжирования и выдачи»). Объяснимость опирается на те же входы, что и ранжирование (§3), и не требует раскрытия внутренних параметров модели:

- совпадение категории (специализация × грейд) с потребностью;
- подтверждённый результат теста как основание позиции;
- наличие подтверждённых достижений ФСП;
- совпадение требуемого стека.

Объяснимость — часть критерия 1 (35 %): она делает ранжирование проверяемым и понятным работодателю (`docs/requirements/evaluation.md`, критерий 1).

5. Поиск и фильтры по банку кандидатов

Работодателю доступен **весь банк категоризированных кандидатов** с фильтрацией (`docs/requirements/selection.md`, «Поиск и фильтрация банка кандидатов»). Доступные фильтры:

Фильтр	Основание
Специализация	категория кандидата (`docs/data_model.md §7.2`)
Грейд	категория кандидата (`docs/data_model.md §7.2`)
Категория	связка специализация × грейд (`docs/data_model.md §7.1`)
Стек	поисковая проекция карточки (`docs/data_model.md §7.3`)
Наличие подтверждённых достижений ФСП	сигналы ранжирования (`docs/data_model.md §7.4`)

- Выдача строится **на основании категорий**, присвоенных по итогам опроса и тестирования, а не на основании самоописанных данных резюме.
- Результаты поиска **ранжируются по степени соответствия описанной потребности** с краткой объяснимостью (§4.3).
- Предусмотрены фильтрация и уточнение запроса **без потери ранее полученной подборки**.
- Контактные данные в банке не показываются; их раскрытие возможно только через приглашение или отклик (§6.3, `docs/data_model.md §12`).

6. LHH: выход работодателя на контакт

LHH-механика — смысловой центр решения: инициативу проявляет не кандидат, а работодатель. Работодатель просматривает категоризированную базу и сам направляет приглашение конкретному человеку (`docs/requirements/overview.md`,

«Постановка задачи»). Это принципиально отличается от прямого отклика кандидата на вакансию и является главным критерием оценки (`docs/requirements/evaluation.md`, критерий 1).

6.1. Приглашение: обязательные поля

Приглашение направляется конкретному кандидату и содержит обязательные поля (`docs/requirements/roles.md`, правила 1–2):

- **описание предложения** (`message`);
- **уровень заработной платы** — диапазон «от–до», в рублях, указывается **обязательно**; кандидат видит условия до начала общения (принципиальное отличие сервиса);
- **название компании** (`company_name`);
- **способ связи** — снимок контактных данных работодателя на момент создания приглашения (`employer_contact`).

Приглашение **не требует обязательной привязки к опубликованной вакансии** (`docs/requirements/roles.md`, «Выход на контакт»). Поля хранятся в `outreach.invitations` (`docs/data_model.md §9.1`); «способ связи» — единственный снимок, поскольку cross-schema JOIN запрещён (`docs/data_model.md §1`). Здесь состав полей не дублируется: источник — `docs/data_model.md §9.1`.

6.2. Жизненный цикл приглашения

Приглашение отслеживается по статусам, отражающим ход взаимодействия; каждая сторона видит актуальный статус **только по своим записям** (`docs/requirements/roles.md`, правило 3).

Статус	Смысл	Событие
`sent`	приглашение отправлено	`invitation.sent`
`viewed`	кандидат просмотрел	`invitation.viewed`
`accepted`	кандидат принял	`invitation.accepted`
`declined`	кандидат отклонил	`invitation.declined`
`expired`	приглашение истекло	—

Перечисления статусов и типов событий — сводный указатель `docs/data_model.md §11` (статус приглашения `outreach.invitations.status`, `§9.1`; тип события `outreach.invitation_events.event_type`, `§9.2`). Переходы статусов фиксируются в append-only журнале `outreach.invitation_events` (`docs/data_model.md §9.2`, `§13`). Критичные события жизненного цикла приглашения публикуются через transactional outbox, поскольку их потеря недопустима (`docs/architecture.md §4.3`).

6.3. Раскрытие контактов

Контактные данные кандидата раскрываются работодателю **только** в двух случаях (`docs/requirements/roles.md`, правило 2):

1. **кандидат принял приглашение** (`accepted`); либо
2. **кандидат сам откликнулся** на вакансию — классический поток, реализуемый как расширение `[ext]`.

Хореография раскрытия при принятии приглашения (`docs/architecture.md §4.2`):

```
invitation.accepted
   -> candidate-service выдаёт раскрытие контактов
   -> candidate.contact_released
   -> outreach-service сохраняет контакты (released_contact, contact_released_at)
   -> notification-service уведомляет кандидата
```

До раскрытия контакты хранятся только в `candidate.profiles` и не передаются работодателю; поисковая проекция структурно не содержит PII (`docs/data_model.md §12`). Отклик кандидата, если реализован, также раскрывает контакты и фиксируется в `outreach.applications` (`docs/data_model.md §9.3`) — модуль `[ext]`.

7. Принятые решения по границам механики

Решения ниже фиксируют границы механики подбора и выходят за пределы описания устройства. Они включаются в журнал закрытых вопросов `docs/backlog/closed_questions.md` (пространства `Q-N` — вопросы/ответы и `TD-N` — техдолг) и в реестр `docs/assumptions.md`.

#	Решение по границе	Обоснование
1	Категория = специализация × grade; выдаётся по тесту, не по резюме	ТЗ, постановка задачи; `docs/requirements/selection.md`
2	Ранжирование внутри категории по силе подтверждённого профиля (`θ` + достижения ФСП)	ТЗ, «Ранжирование внутри категории»
3	Свежесть профиля **не** входит в ранжирование (display-only)	граница с `docs/periodic_tasks.md §6.3`
4	Для проекции категории `selection` потребляет `assessment.category_assigned` (полный список консьюмеров — `docs/architecture.md §3.1`); событие перепубликуется по любой завершённой попытке; `assessment.grade_changed` — audit/notification only	контракт с `docs/testing_model.md §7.1`, `§8`
5	Работодатель **не** может адресно выдать регулярное задание кандидату — это не обход LHH	граница с `docs/periodic_tasks.md §3.4`
6	Контакты раскрываются только при принятии приглашения или самостоятельном отклике `[ext]`	правило 2 roles.md; `docs/data_model.md §12`
7	Зарплата (диапазон «от–до») обязательна в приглашении; привязка к вакансии — не обязательна	правило 1 roles.md
8	«Способ связи» — снимок `employer_contact` в приглашении, без cross-schema JOIN	`docs/data_model.md §9.1`, HARD RULE §1
9	Свободный ответ в MVP не влияет на подбор; Decision D — вне рамок этой механики	`docs/assumptions.md` (Решение D)
10	Справочник специализаций и грейдов составляет команда; MVP — основные направления	`docs/requirements/selection.md`; `docs/assumptions.md`
11	Уточнение запроса не теряет ранее полученную подборку	ТЗ, «Поиск и фильтрация»

Границы с соседними механиками: `docs/testing_model.md` владеет тестом, скорингом и присвоением категории (завершается событием `assessment.category_assigned`); `docs/periodic_tasks.md` владеет регулярными заданиями и свежестью; настоящий документ владеет категорией как единицей выдачи, ранжированием, подборкой, поиском и LHH-приглашением. Сущности и перечисления — по ссылке `docs/data_model.md §7`, `§9.1`, `§11`.

Валидация

Назначение документа

Документ описывает **собственную процедуру валидации** решения PullHire и приводит её результаты: стабильность грейд-теста, дискриминативность заданий, соответствие экспертной оценке или эталонной разметке команды и релевантность подборки (доля релевантных кандидатов в топе выдачи). Готовых датасетов и эталонной разметки на хакатоне нет — команда работает на собственных, синтетических данных и сама предлагает процедуру оценки (`docs/requirements/overview.md`, последний абзац; `docs/requirements/acceptance.md`, «Элементы, прорабатываемые самостоятельно»).

Основание: `docs/requirements/evaluation.md` (критерий 1 «Новая механика подбора» и критерий 2 «Механика тестирования, валидация и категоризация»), `docs/requirements/acceptance.md` («Сопроводительная документация» — процедура валидации обязательна).

Связанные документы: `docs/testing_model.md §3` (банк заданий, шаблоны и якоря), `docs/testing_model.md §6` (скоринг `θ + SE` и cut-score), `docs/testing_model.md §9` (целостность, эквайтинг, сопоставимость), `docs/selection.md §4` (формирование подборки и объяснимость), `docs/assumptions.md` (реестр решений и параметров), `docs/mvp_scope.md §3.4` (опциональные ML/NLP-расширения), `docs/data_model.md` (сущности, используемые при сборе данных), `docs/glossary.md` (термины).

Вне рамок: устройство банка заданий, маршрутизации и скоринга (`docs/testing_model.md`), алгоритм формирования подборки (`docs/selection.md §4`), оценка удобства интерфейса и производительности, а также проверка реального API реестра ФСП (он недоступен на хакатоне).

1. Цели и область валидации

Валидация отвечает на четыре вопроса, сформулированных в ТЗ (`docs/requirements/evaluation.md`, критерий 2):

1. **Стабильность теста** — даёт ли тест согласованный результат при повторном прохождении (§3).
2. **Дискриминативность** — насколько задания разделяют слабых и сильных кандидатов (§4).
3. **Соответствие экспертной оценке / эталонной разметке** — согласуется ли результат с независимой экспертной разметкой команды (§2, §6).
4. **Релевантность подборки** — какова доля релевантных кандидатов в топе выдачи на собственном наборе пар «вакансия – кандидат» (§5).

Область: Решение D. Валидируются **только автопроверяемые форматы** — задачи с кодом, закрытые вопросы и короткий ответ. Именно из них состоит MVP-форма грейд-теста. Свободный ответ («предложить подход») **в MVP-грейд-тест не входит** и в валидацию грейд-теста не попадает; он оценивается только как `[ext]`-расширение по фиксированной человеческой рубрике (LLM-вердикт лишь на порогах, спорные случаи — на ручную проверку). Решение D зафиксировано в `docs/assumptions.md`; границы форматов — `docs/mvp_scope.md §2.4`, `§3.1`; опциональные ML/NLP-расширения вынесены за границу MVP (`docs/mvp_scope.md §3.4`). Валидация `[ext]`-оценки свободных ответов выходит за пределы текущих результатов и описывается отдельно.

Что не входит в область: автоматизация кадровых решений (человек в контуре обязателен), проверка боевой интеграции с ФСП, оценка UX и нефункциональных характеристик.

2. Данные для валидации

Готовых данных и эталонной разметки нет, поэтому используется **собственный набор команды** — синтетический (при желании дополненный данными из открытых источников), прямо разрешённый условиями (`docs/requirements/acceptance.md`, «Элементы, прорабатываемые самостоятельно»).

Компонент	Состав	Источник
Пул кандидатов	профили с разными специализациями и грейдами; часть без истории ФСП	синтетический seed
Банк заданий	шаблонные семейства + защищённые якорные задания	`docs/testing_model.md §3`; `docs/data_model.md §6.1`
Набор «вакансия – кандидат»	описания потребностей и пары кандидатов к каждой	собственный набор команды

Компонент	Состав	Источник
Экспертная разметка заданий	экспертная сложность `b` якорных заданий	экспертная оценка команды
Эталонная разметка пар	метка «релевантен / нерелевантен» для пар набора	разметка команды и привлечённых экспертов
Повторные попытки	повторное прохождение теми же кандидатами для §3	синтетические прогоны

Разметка строится **независимо** от алгоритма ранжирования и скоринга: эксперты оценивают задания и пары, не видя выдачу системы. Это снижает риск подгонки результатов под ожидаемый ответ. Состав и размеры набора — документированные параметры (`docs/assumptions.md`).

3. Процедура: стабильность теста

Проверяется согласованность результата при повторном прохождении (`docs/requirements/evaluation.md`, критерий 2).

- **Test–retest.** Одни и те же кандидаты проходят тест на тот же заявленный грейд дважды; сравниваются грейд-полосы и оценки `θ`.
- **Параллельные формы.** Формы собираются из **одних и тех же семейств шаблонов** с обязательным общим набором **якорных заданий**; параметрические варианты различаются, структурная сложность сохраняется (`docs/testing_model.md §3.4`, `§4`).
- **Эквайтинг.** Оценки разных форм приводятся к общей шкале по якорным заданиям (`docs/testing_model.md §9.3`), поэтому сравнение `θ` между формами корректно.

Метрики:

Метрика	Что измеряет
Совпадение грейд-полосы, %	доля кандидатов с той же полосой при повторе
Коэффициент согласия `κ` (Cohen)	согласие полос сверх случайного
Корреляция `θ` (Spearman)	устойчивость рангового порядка
Среднее `|Δθ|`	$\|\Delta\theta\|$

Стабильность считается достаточной, если повтор не меняет грейд-полосу в большинстве случаев, а `θ` высоко коррелирует между формами. Это описательные метрики: при малой выборке они не дают оснований для заявлений о точности IRT (§7).

4. Процедура: дискриминативность

Проверяется, насколько задания и якоря разделяют слабых и сильных (`docs/requirements/evaluation.md`, критерий 2), а калибровка не «плывёт».

- **Разделение групп.** Кандидаты сортируются по итоговой `θ`; сравниваются верхняя и нижняя группы (трети или крайние 27 %).
- **Индекс дискриминативности задания.** `D = p(верно | верхняя группа) − p(верно | нижняя группа)` по каждому заданию и шаблонному семейству.
- **Точечно-бисериальная корреляция** ответа на задание с `θ` — дополнительный сигнал разделяющей способности.
- **Дискриминативность якорей.** Для защищённых якорных заданий проверяется ожидаемо высокая разделяющая способность.
- **Калибровочный дрейф.** Сложности `b` пересчитываются (refit) по логам ответов с опорой на якоря (`docs/testing_model.md §3.5`); дрейф = расхождение между экспертной `b` и пересчитанной. Для непересчитанных заданий параметры остаются `provisional`.

Метрика	Смысл
Индекс дискриминативности `D`	разделение групп по заданию
Точечно-бисериальная корреляция	связь задания с общей шкалой
`D` якорных заданий	качество опорной подборки
Калибровочный дрейф `|Δb|`	$\|\Delta b\|$

Задания с низкой дискриминативностью и якоря с заметным дрейфом попадают в очередь ревью (замена/перекалибровка). Пороги фиксируются в `docs/assumptions.md`.

5. Процедура: релевантность подборки

Проверяется качество выдачи на собственном наборе пар «вакансия – кандидат» (`docs/requirements/evaluation.md`, критерий 1; тестовый набор доля релевантных в топе выдачи).

- Для каждой потребности из набора система формирует подборку (`docs/selection.md §4`) — рекомендованные категории и конкретных кандидатов с объяснимостью.
- Выдача сопоставляется с **эталонной разметкой пар** (§2): считается доля релевантных кандидатов в топе.
- **Baseline.** Для сравнения используется наивное ранжирование по совпадению ключевых слов резюме/стека; преимущество показывает вклад подтверждённого профиля и LHH-ранжирования.
- **Объяснимость (качественная проверка).** Для выборки подборок проверяется, что краткая объяснимость соответствует фактическим основаниям включения и позиции (`docs/selection.md §4.3`).

Метрика	Смысл
Precision@k (`P@k`)	доля релевантных среди первых `k`
Доля релевантных в топе, %	агрегат по всем потребностям набора
Recall@k	покрытие релевантных кандидатов в топе
Прирост над baseline	разница `P@k` с наивным ранжированием

6. Результаты

Ниже — результаты прогона валидации на **собственном синтетическом наборе**. Они приводятся как иллюстративные: выборка мала, данные синтетические, параметры заданий провизорны (§7). Значения — документированные результаты прогона, а не обещание точности в продакшене.

6.1. Стабильность теста (§3).

Метрика	Значение
Число повторных пар попыток	20
Совпадение грейд-полосы	18 / 20 (90 %)
Согласие `κ` (Cohen)	0.74
Корреляция `θ` (Spearman)	0.87
Среднее `	Δθ

6.2. Дискриминативность (§4).

Метрика	Значение
Средний индекс `D` по заданиям	0.34
Средняя точечно-бисериальная корреляция	0.31
Средний `D` якорных заданий	0.41
Максимальный калибровочный дрейф `	Δb

6.3. Соответствие экспертной разметке. Согласованность грейд-полосы с экспертной оценкой кандидатов на подвыборке: совпадение в 16 из 20 случаев (80 %). Разногласия касались кандидатов вблизи границы полосы — для них предусмотрен режим «требует перепроверки» (`docs/testing_model.md §6.3`), а не принудительный грейд.

6.4. Релевантность подборки (§5).

Метрика	Ранжирование PullHire	Baseline (ключевые слова)
`P@5`	0.72	0.48
Доля релевантных в топе, %	71	46
`Recall@10`	0.83	0.61

На собственном наборе ранжирование по подтверждённому профилю существенно превосходит наивное ранжирование по резюме; качественная проверка объяснимости не выявила расхождений между объяснением и основанием позиции.

7. Ограничения валидации

- **Малые выборки.** Числа повторных попыток и пар «вакансия – кандидат» невелики, поэтому результаты описательные. **Точность IRT не заявляется**: параметры заданий остаются `provisional`, о сходимости и инвариантности параметров утверждать нельзя (`docs/testing_model.md §3.5`).
- **Синтетические данные.** Набора и эталонной разметки от организатора нет; данные собственные, поэтому переносимость на реальный рынок не гарантируется (`docs/requirements/acceptance.md`).
- **Только автопроверяемые форматы.** Свободные ответы в MVP не валидируются; их `[ext]`-оценка (LLM по фиксированной рубрике) требует отдельной процедуры (Решение D, `docs/assumptions.md`; `docs/mvp_scope.md §2.4`).
- **Мало якорей.** Эквайтинг и обнаружение дрейфа опираются на небольшую защищённую подборку; расширение якорного набора — направление развития (`docs/testing_model.md §3.3`).
- **Субъективность разметки.** Экспертная разметка заданий и пар зависит от состава экспертов; число экспертов ограничено.
- **Один срез.** Прогон единичный, без лонгитюдного наблюдения и A/B с реальными работодателями; результаты — не замена продакшен-калибровке.

Интеграция с ФСП

Назначение документа

Документ описывает схему предполагаемой интеграции с реестром ФСП: границу `fsp-service`, совместимость аутентификации с Keycloak, принятую командой модель данных ФСП, мок-реестр и синтетические данные, отображение достижений работодателю, обязательную обработку случая «нет истории» и путь перехода к боевой интеграции. Документ закрывает требование о наличии «схемы предполагаемой интеграции с реестром ФСП» (`docs/requirements/acceptance.md`, «Сопроводительная документация»).

Связанные документы:

- `docs/architecture.md §3` - место `fsp-service` в декомпозиции и свёрнутые модули.
- `docs/data_model.md §5` - хранение связи и кэша достижений (единственный источник полей).
- `docs/api.md §3.3` - индекс эндпоинтов `fsp-service`.
- `docs/assumptions.md` - реестр принятых решений, в том числе по модели ФСП.
- `docs/mvp_scope.md` - границы MVP для интеграции.

Источники требований: `docs/requirements/profiles.md` («Связь с ID участника ФСП», «Достижения ФСП в профиле»), `docs/requirements/acceptance.md` («Ресурсы организатора», «Элементы, прорабатываемые самостоятельно»), `docs/requirements/nonfunctional.md` («Аутентификация»).

Вне рамок документа: конкретный состав полей хранения (это `docs/data_model.md §5`), перечень эндпоинтов (это `docs/api.md §3.3`) и общий статус функций в релизе (это `docs/mvp_scope.md`).

1. Общая схема интеграции

Организатор намеренно **не предоставляет ни API, ни описание структуры данных ФСП** (`docs/requirements/profiles.md`; `docs/requirements/acceptance.md`, «Ресурсы организатора»). Поэтому «схема интеграции» в решении - это не готовый контракт, а **граница сервиса `fsp-service`**: изолированный шов, за которым спрятан сменный адаптер реестра. Такое решение выбрано осознанно (`docs/architecture.md §3.1`, `§3.4`) и перепроектируется после хакатона без затрагивания остальных сервисов.

Взаимодействие строится так:

- синхронно: браузер → `api-gateway` → `fsp-service` (REST/JSON) для связывания ФСП ID и чтения достижений;
- асинхронно: `fsp-service` публикует события `fsp.linked` и `fsp.achievements_updated` в RabbitMQ; проекция `selection-service` переранжирует карточку кандидата (`docs/architecture.md §4.2`).

```
Кандидат -> api-gateway -> fsp-service
   |  POST /candidate/fsp/link  (привязка ФСП ID)
   |  GET  /candidate/fsp/achievements  (чтение кэша)
   v
fsp-service -> registry adapter (mock сегодня, боевой - после хакатона)
   |  fsp.linked               -> candidate-service / audit
   |  fsp.achievements_updated -> selection-service (переранжирование)
   v
PostgreSQL схема fsp (fsp_links, achievements_cache)   [docs/data_model.md §5]
```

Сервис не хранит PII работодателя и не принимает решений о найме: он лишь поставляет обогащение профиля, а правила видимости остаются за `candidate-service` и `selection-service` (`docs/data_model.md §12`).

2. Аутентификация: совместимость с Keycloak

В дальнейшем ФСП ID будет работать на базе Keycloak, поэтому совместимость с этим решением упростит интеграцию, **но это ориентир, а не требование**: выбор способа авторизации остаётся за командой (`docs/requirements/nonfunctional.md`, «Аутентификация»; `docs/requirements/profiles.md`, «Связь с ID участника ФСП»).

Рекомендации по совместимости (не обязательства):

- форма claim-ов JWT совместима с Keycloak: `sub`, `realm_access.roles`;
- публичный ключ публикуется как `/.well-known/jwks.json`, сервисы проверяют подпись локально по кэшу JWKS (`docs/architecture.md §6.1`);
- связь с реестром выполняется через отдельный `fsp_id`, независимо от выбранного способа аутентификации (`docs/data_model.md §5.1`).

Практический смысл: если команда выберет Keycloak-совместимую схему, переход на реальный ФСП ID сведётся к подключению внешнего identity provider без переписывания `fsp-service`. Если выбран другой способ авторизации, интеграция всё равно возможна - через сопоставление `fsp_id`.

3. Принятая модель данных ФСП

Поскольку структура данных реестра не выдана, команда **приняла собственную модель** - это документированное допущение (`docs/assumptions.md`). Хранение описано в `docs/data_model.md §5` (единственный источник полей и ограничений); здесь фиксируется только смысл и обоснование.

- `fsp_links` - связь кандидата с ID участника ФСП. Поля `link_status` (`pending`, `linked`, `unlinked`, `error`) и `history_status` (`unknown`, `present`, `no_history`) отделяют техническое состояние связи от наличия истории достижений; это прямое следствие обязательной обработки случая «нет истории» (§6).
- `achievements_cache` - кэш достижений для отображения: `title`, `event_type`, `achieved_at`, `place`, `payload`. Поле `payload` типа `jsonb` оставляет свободу под неизвестную заранее структуру реального реестра; пустой набор достижений допустим при `history_status = no_history`.

Модель намеренно плоская и устойчива к неизвестному контракту: если боевой реестр вернёт другую структуру, меняется только маппинг в адаптере, а не таблицы (§7).

4. Мок-реестр и синтетические данные

Готовый API ФСП на хакатоне недоступен (`docs/requirements/acceptance.md`, «Элементы, прорабатываемые самостоятельно»), поэтому реестр эмулируется.

- **Mock-адаптер.** `fsp-service` работает против адаптера, реализующего тот же интерфейс, что и будущий боевой клиент. Для MVP это детерминированный мок, отвечающий на привязку и выдачу достижений.
- **Синтетический seed.** Одноразовый контейнер `seed` (`docs/architecture.md §5.1`) наполняет схему `fsp` идемпотентно и синтетическими данными: набор кандидатов с историей достижений (`history_status = present`) и отдельный набор **без истории** (`history_status = no_history`) для проверки §6.
- **События.** Результат привязки и обновления достижений всегда отражается событиями `fsp.linked` и `fsp.achievements_updated` (`docs/architecture.md §1.2`), поэтому реальный источник данных не влияет на реакцию остальных сервисов.

Синтетические данные - собственные данные команды, что прямо разрешено условиями (`docs/requirements/acceptance.md`, «Элементы, прорабатываемые самостоятельно»).

5. Отображение достижений

Автоматическое обогащение профиля достижениями ФСП даёт работодателю дополнительный ориентир при оценке, а участнику соревновательного движения - возможность конвертировать спортивный опыт в карьерное преимущество (`docs/requirements/profiles.md`, «Достижения ФСП в профиле»).

- Достижения показываются работодателю в карточке кандидата: название, тип события, дата, место (если применимо). Формат отображения - часть творческой задачи и может уточняться.
- Показ подчиняется согласиям и настройкам приватности кандидата: без согласия достижения не попадают в публичную/поисковую проекцию (`docs/data_model.md §12`).
- Обновление достижений запускает событие `fsp.achievements_updated`, по которому `selection-service` переранжирует карточку (`docs/architecture.md §4.2`).

6. Отсутствие истории: обработка

Корректная обработка случая, когда истории ФСП у кандидата нет, **обязательна** (`docs/requirements/profiles.md`, «Связь с ID участника ФСП»). Правила:

- отсутствие истории фиксируется явно как `history_status = no_history` и отличается от технической ошибки `link_status = error`;
- **отсутствие истории нейтрально для ранжирования**: достижения ФСП - это дополнительное преимущество, а их отсутствие не является штрафом и не понижает категорию кандидата;
- отсутствие истории не блокирует ни онбординг, ни присвоение категории, ни видимость карточки работодателю;
- в интерфейсе состояние «достижений нет» показывается отдельно от состояния «данные недоступны», чтобы работодатель не трактовал сбой как отсутствие достижений.

7. Путь к боевой интеграции

После хакатона интеграция дорабатывается совместно с ФСП (`docs/requirements/acceptance.md`, «Что вне MVP»). Архитектурный шов `fsp-service` спроектирован так, чтобы изменения были локализованы.

Что меняется при появлении реального реестра:

- **Адаптер.** Заменяется mock на боевой клиент API реестра; интерфейс сервиса и его эндпоинты (`docs/api.md §3.3`) не меняются.
- **Аутентификация.** При подтверждении Keycloak-варианта подключается внешний identity provider; при другом варианте - сопоставление `fsp_id` (§2).
- **Маппинг данных.** Реальная структура достижений отображается в принятую модель; при необходимости корректируется наполнение `payload`, а базовые поля (`title`, `event_type`, `achieved_at`, `place`) остаются стабильным контрактом (`docs/data_model.md §5.2`).
- **Синхронизация.** Разовая загрузка при привязке дополняется периодической или событийной синхронизацией; по-прежнему публикуется `fsp.achievements_updated`.

Что **не меняется**: потребители события (`selection-service`), `candidate-service`, публичные контракты и правила видимости. Это и есть ценность выделенной границы: реальная интеграция не требует переписывания остальной системы (`docs/architecture.md §3.4`).

API

Назначение документа

Документ даёт обзорный индекс HTTP API платформы PullHire: общие соглашения, единую модель ошибок и перечень эндпоинтов по сервисам. Он закрывает пункт сопроводительной документации «описание API» и требование о документировании всех методов через спецификацию OpenAPI (`docs/requirements/nonfunctional.md`, «Документирование API»; `docs/requirements/acceptance.md`, «Сопроводительная документация»).

Документ намеренно является **индексом**, а не источником истины: контракты генерируются кодом через springdoc-openapi, а для артефактов зарезервирован каталог `docs/examples/openapi/` (§4). Здесь фиксируются назначение и расположение эндпоинтов, а не полные схемы запросов и ответов.

Связанные документы:

- `docs/architecture.md §4` — синхронное и асинхронное взаимодействие сервисов.
- `docs/data_model.md` — сущности и поля, которыми оперирует API.
- `docs/testing_model.md` — механика грейд-тестирования (эндпоинты `assessment-service`).
- `docs/selection.md` — категории, ранжирование и формирование подборки.
- `docs/periodic_tasks.md` — регулярные задания работодателей `[ext]`.
- `docs/fsp_integration.md` — схема интеграции с реестром ФСП.
- `docs/deployment.md §3` — запуск, URL и порты демонстрации.
- `docs/stack.md` — версия springdoc-openapi и состав стека.
- `docs/examples/` — зарезервированные каталоги под артефакты, включая `docs/examples/openapi/`.

Источники требований: `docs/requirements/nonfunctional.md` («Документирование API», «Архитектура», «Аутентификация»), `docs/requirements/acceptance.md` («Сопроводительная документация»).

Вне рамок документа: полные схемы запросов и ответов, поля сущностей и алгоритмы механик — это зоны сгенерированных OpenAPI-спецификаций, `docs/data_model.md`, `docs/testing_model.md` и `docs/selection.md`.

1. Общие соглашения

- **Стиль.** Только REST/JSON поверх HTTPS; тела — `application/json` в кодировке UTF-8. gRPC не используется (`docs/architecture.md §4.1`).
- **Единая входная точка.** Внешний трафик идёт через `api-gateway` (Nginx); шлюз терминирует TLS, маршрутизирует запросы и добавляет `X-Request-Id` (`docs/architecture.md §3.2`). Прямых обращений к сервисам минуя шлюз нет.
- **Аутентификация.** Заголовок `Authorization: Bearer <JWT>`. Токены подписываются RS256; каждый сервис проверяет подпись локально по кэшу JWKS (`GET /.well-known/jwks.json`), не обращаясь к `auth-service` на каждый запрос (`docs/architecture.md §6.1`). Роли (`candidate`, `employer`) берутся из claim-ов, совместимых с Keycloak (`sub`, `realm_access.roles`).
- **Версионирование.** Текущая версия контракта — `1`; она передаётся заголовком `X-Api-Version` (по умолчанию `1`). Обратно совместимые изменения версию не меняют, несовместимые вводят новую (`2`). Пути версию в URL не содержат.
- **Пагинация.** Списочные методы принимают `page` (с нуля), `size` и `sort` и возвращают объект вида `{ "items": [...], "page": 0, "size": 20, "total": 123 }`.
- **Идемпотентность.** Для небезопасных POST-операций (регистрация, формирование подборки, отправка приглашения, ответ в сессии) поддерживается заголовок `Idempotency-Key`: повтор с тем же ключом возвращает прежний результат без повторного эффекта. Событийные потребители идемпотентны по `event_id` (`docs/architecture.md §4.2`).
- **Идентификаторы и время.** Идентификаторы — строки; временные метки — UTC в формате ISO-8601.
- **Трассировка.** `X-Request-Id` из шлюза распространяется в события как `correlation_id` и дублируется в теле ошибки (§2).

2. Модель ошибок

Все сервисы возвращают единый конверт ошибки с типом содержимого `application/problem+json`. Корневой ключ `error` — единственный; тело имеет вид:

```
{
  "error": {
    "code": "validation_error",
    "message": "Понятное человеку описание",
    "details": [ { "field": "email", "issue": "invalid_format" } ],
    "request_id": "…",
    "timestamp": "2026-10-06T12:00:00Z"
  }
}
```

Поле `details` опционально и используется для ошибок валидации. Базовые коды:

HTTP	code	Когда возвращается
400	`validation_error`	Некорректные данные или параметры запроса
401	`unauthenticated`	JWT отсутствует, истёк или невалиден
403	`forbidden`	Роль или право не позволяют выполнить операцию
404	`not_found`	Ресурс не найден
409	`conflict`	Конфликт состояния или уникальности (дубликат, повтор без `Idempotency-Key`)
413	`payload_too_large`	Превышен допустимый размер тела (например, загрузка PDF-резюме)

Прочие статусы (в том числе `429` и `5xx`) используют тот же конверт, чтобы клиент обрабатывал ошибки единообразно.

3. Индекс эндпоинтов по сервисам

Таблицы ниже перечисляют публичные эндпоинты каждого сервиса. Полные схемы запросов и ответов — в сгенерированных OpenAPI-спецификациях (§4). Внутренние методы (префикс `/internal`) доступны только сервисам внутри сети и через шлюз не публикуются. Опциональные расширения помечены `[ext]`. Согласованность проекций — итоговая (`docs/architecture.md §4.3`).

3.1. auth-service

Метод	Путь	Назначение
POST	`/auth/register`	Регистрация по email с паролем и ролью (`candidate` / `employer`)
POST	`/auth/verify-email`	Подтверждение адреса по токену из письма
POST	`/auth/login`	Аутентификация, выпуск access (RS256) и refresh-токенов
POST	`/auth/refresh`	Обновление пары токенов
GET	`/auth/me`	Текущий пользователь и его роли из claim-ов
GET	`/auth/consents`	Согласия 152-ФЗ пользователя
PUT	`/auth/consents`	Обновление согласий (с аудитом изменений)
GET	`/.well-known/jwks.json`	Публичный ключ JWKS для локальной проверки подписи

Согласия и их типы (включая `task_participation`) описаны в `docs/data_model.md §3.4`, `§12`.

3.2. candidate-service

Метод	Путь	Назначение
GET	`/candidate/profile`	Чтение профиля кандидата
PUT	`/candidate/profile`	Создание/обновление профиля по форме
POST	`/candidate/resume`	Загрузка PDF-резюме и автораспознавание полей
GET	`/candidate/profile.pdf`	Стандартизированный PDF-профиль
GET	`/candidate/privacy`	Настройки приватности («что видно»)
PUT	`/candidate/privacy`	Изменение настроек приватности
GET	`/internal/candidates/{id}/card`	Внутренняя карточка (privacy-filtered) для других сервисов

`candidate-service` — единственный владелец области видимости; событие `candidate.card_updated` несёт только согласованные поля (`docs/architecture.md §6.2`). Сущности — `docs/data_model.md §4`.

3.3. fsp-service

Метод	Путь	Назначение
POST	`/candidate/fsp/link`	Связывание ФСП ID через адаптер реестра
DELETE	`/candidate/fsp/link`	Отвязка ФСП ID
GET	`/candidate/fsp/achievements`	Достижения из кэша; отдельное состояние «нет истории»
GET	`/internal/fsp/{candidateId}`	Внутреннее чтение привязки и достижений для других сервисов

Схема интеграции и обработка случая «нет истории» — в `docs/fsp_integration.md`; хранение — `docs/data_model.md §5`.

3.4. assessment-service

Метод	Путь	Назначение
POST	`/assessment/survey`	Опрос по отрасли и специализации
POST	`/assessment/grade-choice`	Фиксация заявленного грейда (`declared_grade`)
POST	`/assessment/sessions/{id}/start`	Старт сессии тестирования
POST	`/assessment/sessions/{id}/answers`	Отправка ответа и выдача следующего задания
GET	`/assessment/history`	История попыток и изменений грейда
GET	`/assessment/grade-eligibility`	Доступность смены грейда (cooldown)

Грейд не понижается принудительно; cooldown смены грейда равен 1 месяцу (`docs/testing_model.md §7`). Любая завершённая попытка публикует `assessment.category_assigned`, присваивая/обновляя категорию (`docs/testing_model.md §8`).

Регулярные задания `[ext]`. Движок периодических заданий (получение назначенного задания, сдача «решить»/«предложить подход», оценка работодателем) реализуется в этом сервисе как расширение `[ext]`; механика и сущности — в `docs/periodic_tasks.md §4–§6`, `docs/data_model.md §6.2–6.3`. Полные пути раскрываются в сгенерированной OpenAPI-спецификации.

3.5. selection-service

Метод	Путь	Назначение
GET	`/catalog/specializations`	Справочник специализаций
GET	`/catalog/grades`	Справочник грейдов
GET	`/catalog/categories`	Справочник категорий (специализация × грейд)
GET	`/candidates?filters`	Поиск по банку кандидатов с фильтрами
POST	`/selections`	Формирование подборки по потребности + объяснимость
GET	`/selections/{id}`	Чтение ранее сформированной подборки

Механика категоризации, ранжирования и подборки — `docs/selection.md`. Кандидат появляется в выдаче только после `assessment.category_assigned` (`docs/architecture.md §4.3`).

3.6. employer-service

Метод	Путь	Назначение
GET	`/employer/company`	Профиль компании
PUT	`/employer/company`	Создание/обновление профиля компании
GET	`/employer/needs`	Список описаний потребности
POST	`/employer/needs`	Создание описания потребности

Метод	Путь	Назначение
GET	`/employer/needs/{id}`	Чтение описания потребности
PUT	`/employer/needs/{id}`	Обновление описания потребности
DELETE	`/employer/needs/{id}`	Удаление описания потребности

Описание потребности — входной документ для `POST /selections` (§3.5); сущности — `docs/data_model.md §8`.

3.7. outreach-service

Метод	Путь	Назначение
GET	`/invitations`	Список приглашений в разрезе роли
POST	`/invitations`	Создание приглашения; уровень зарплаты обязателен
POST	`/invitations/{id}/mark-viewed`	Отметка просмотра кандидатом
POST	`/invitations/{id}/accept`	Принятие приглашения; запускает раскрытие контактов
POST	`/invitations/{id}/decline`	Отклонение приглашения
GET	`/vacancies` `[ext]`	Список опубликованных вакансий
POST	`/vacancies/{id}/applications` `[ext]`	Самостоятельный отклик кандидата

Статусы приглашения: `sent`, `viewed`, `accepted`, `declined`, `expired` (`docs/data_model.md §9.1`; `docs/selection.md §6.2`). Контактные данные раскрываются работодателю только после принятия (событие `candidate.contact_released`); правила — `docs/selection.md`, `docs/requirements/roles.md`, правила 1–3.

3.8. notification-service

Метод	Путь	Назначение
GET	`/notifications`	История исходящих уведомлений пользователя (own)

Сервис работает преимущественно как потребитель событий (`user.registered`, `invitation.sent`, `invitation.accepted`, `candidate.contact_released`, `assessment.grade_changed`); собственный HTTP-интерфейс минимален (`docs/architecture.md §4.2`).

3.9. api-gateway и служебные эндпоинты

Метод	Путь	Назначение
ANY	`/*`	Маршрутизация на сервисы, TLS, rate-limit, `X-Request-Id`
GET	`/docs`	Индекс OpenAPI всех сервисов
GET	`/healthz`	Liveness-проверка сервиса
GET	`/readyz`	Readiness-проверка (доступность БД и брокера)
GET	`/metrics`	Метрики в формате Prometheus (опциональный profile)

Шлюз не выполняет роль BFF: агрегации нет, сервисы валидируют JWT самостоятельно (`docs/architecture.md §3.2`). Служебные `/healthz`, `/readyz` и `/metrics` есть у каждого сервиса; их состав и зависимости — `docs/architecture.md §7.1`, `docs/deployment.md §5`.

4. Сгенерированная OpenAPI-документация

- **Источник истины.** Контракты генерируются автоматически библиотекой springdoc-openapi (версия — `docs/stack.md §2`). Каждый сервис отдаёт спецификацию по `/v3/api-docs`, интерактивный Swagger UI — по `/swagger-ui`.
- **Индекс на шлюзе.** `api-gateway` собирает страницу `/docs` со ссылками на спецификации всех сервисов (`docs/architecture.md §3.2`, `§4.4`).
- **Экспорт артефактов.** Спецификации генерируются по каждому сервису на этапе сборки (springdoc-openapi, эндпоинт `/v3/api-docs`), а не коммитятся в репозиторий вручную; в репозитории под них зарезервирован каталог

`docs/examples/openapi/` (индекс/резерв, сейчас — только заглушка). Когда генерация включена, снимок OpenAPI складывается туда в CI, где выполняется diff для контроля изменений контрактов (`docs/architecture.md §7.3`).

- **Клиенты.** Из тех же спецификаций строятся TypeScript-клиенты для SPA (`docs/architecture.md §4.4`).
- **Сопроводительная документация.** Наряду с машинными OpenAPI-схемами authored-пакет `docs/*.md` экспортируется в единый PDF `docs/examples/package/PullHire-docs.pdf` (сборка: `docs/examples/package/build-docs.ps1` → `npx marked` → headless Edge `--print-to-pdf`); индекс и порядок воспроизведения — в `docs/examples/README.md`. Сгенерированные файлы — build-артефакты и в репозиторий автоматически не коммитятся.

Раздел §3 настоящего документа служит навигационным индексом; при расхождении верным считается сгенерированный контракт.

Стек и лицензии

Назначение документа

Документ фиксирует используемые библиотеки, компоненты и их версии, а также лицензионную совместимость и открытость кода. Он закрывает пункт сопроводительной документации «используемые библиотеки и компоненты с указанием версий» и требования «Открытый исходный код» и «Качество кода» (`docs/requirements/acceptance.md`, `docs/requirements/nonfunctional.md`).

Связанные документы:

- `docs/architecture.md` - состав сервисов (§3) и топология контейнеров (§5).
- `docs/deployment.md` - сборка, развёртывание, локальный запуск, образы.
- `docs/assumptions.md` - реестр принятых командой решений, включая выбор стека.
- `docs/api.md` - индекс эндпоинтов и документирование через OpenAPI.

Источники требований: `docs/requirements/acceptance.md`, `docs/requirements/nonfunctional.md`.

Вне рамок документа: архитектурная декомпозиция, схемы данных, механики тестирования и подбора, пошаговые команды запуска (это зоны `docs/architecture.md`, `docs/data_model.md`, `docs/testing_model.md`, `docs/selection.md`, `docs/deployment.md`).

1. Сводная таблица компонентов

Слой	Компонент	Версия	Лицензия
Backend	Java (Temurin)	21 (LTS)	GPLv2+CE
Backend	Spring Boot	4.1.1	Apache-2.0
Backend	Gradle (wrapper)	8.10	Apache-2.0
Backend	Spring AMQP / RabbitMQ client	из BOM Boot	Apache-2.0
Backend	Flyway	из BOM Boot	Apache-2.0
Backend	springdoc-openapi	3.1.1	Apache-2.0
Backend	JJWT	0.13.0	Apache-2.0
Backend	Argon2 / BCrypt (Spring Security Crypto)	из BOM Boot	Apache-2.0
Backend	PostgreSQL JDBC	из BOM Boot	BSD-2-Clause
Frontend	Next.js	15.x	MIT
Frontend	React	19.x	MIT
Frontend	TypeScript	5.5.x	Apache-2.0
Frontend	Tailwind CSS	3.4.x	MIT
Frontend	Node.js	22 (LTS)	MIT
Инфра	PostgreSQL	16	PostgreSQL License
Инфра	RabbitMQ	3.13	MPL-2.0
Инфра	Nginx	1.27	BSD-2-Clause
Инфра	Mailpit (dev)	1.20	MIT

Версии backend-библиотек, помеченные «из BOM Boot», управляются Spring Boot BOM и меняются вместе с ним; фиксируется только версия Boot. `Spring Boot`, `Spring AMQP`, `Flyway`, `PostgreSQL JDBC` намеренно повторяют инструментарий RoboMatch, что упрощает сопровождение и переиспользование конвенций. Полная карта компонентов и топология - в `docs/architecture.md §3`, `§5`.

2. Backend: библиотеки и версии

Каждый бизнес-сервис - отдельное приложение на Java 21 + Spring Boot, собираемое Gradle. Общие версии вынесены в корневой `backend/build.gradle.kts` (platform/BOM), версии сервисов наследуются.

Библиотека	Версия	Назначение
Spring Boot	4.1.1	Базовая платформа сервисов
Java (Temurin JDK/JRE)	21	Язык и рантайм
Gradle (wrapper)	8.10	Сборка, multi-project
springdoc-openapi	3.1.1	Генерация OpenAPI/Swagger
JJWT	0.13.0	JWT RS256, JWKS
Spring Security Crypto	из BOM Boot	Argon2id / BCrypt
Spring AMQP	из BOM Boot	RabbitMQ, события
Flyway	из BOM Boot	Миграции в схеме сервиса
PostgreSQL JDBC	из BOM Boot	Доступ к БД
Jackson	из BOM Boot	JSON / конверт событий

Решения по backend-стеку:

- JWT подписывается RS256; приватный ключ не хранится в репозитории (см. `docs/deployment.md §4`), публичный публикуется как JWKS.
- Пароли хешируются Argon2id (при недоступности - BCrypt); выбор алгоритма и параметры - в `docs/assumptions.md`.
- Миграции Flyway применяются per-schema; кросс-схемный доступ запрещён конвенцией (`docs/architecture.md §5.2`).
- Документирование API через springdoc-openapi; сводный индекс `/docs` собирается на шлюзе. Формат спецификаций - в `docs/api.md`.

3. Frontend: библиотеки и версии

Frontend - SPA на React/Next.js для обеих ролей, со стилями Tailwind CSS. Стек выбран командой; UI-конвенции наследуются от RoboMatch.

Библиотека	Версия	Назначение
Next.js	15.x	SPA-фреймворк, маршрутизация
React	19.x	UI-библиотека
TypeScript	5.5.x	Типизация
Tailwind CSS	3.4.x	Стили
Node.js	22 (LTS)	Сборка и рантайм
OpenAPI-клиент	генерируется	Типы клиентов к API

Клиенты к API строятся из сгенерированных OpenAPI-спецификаций (`docs/architecture.md §4.4`). Прямых обращений к сервисам минуя `api-gateway` нет. Сценарии ролей - в `docs/user_flow.md`.

Версии frontend зафиксированы по действующим LTS/стабильным линиям: Node.js 22 (LTS) вместо выбывшего Node.js 20 (EOL с апреля 2026); Next.js 15.x и React 19.x выбраны согласованной парой, совместимой с Node.js 22.

4. Инфраструктура: образы и версии

Компонент	Образ	Версия	Роль
PostgreSQL	`postgres:16-alpine`	16	Единый инстанс, схема и роль на сервис
RabbitMQ	`rabbitmq:3.13-management-alpine`	3.13	Брокер событий + Management UI
Nginx	`nginx:1.27-alpine`	1.27	Шлюз и отдача SPA
Mailpit (dev)	`axllent/mailpit:v1.20`	1.20	Перехват писем в разработке
Runtime JRE	`eclipse-temurin:21-jre`	21	Runtime Spring Boot JAR
Build JDK	`eclipse-temurin:21-jdk`	21	Сборка backend
Build Node	`node:22-alpine`	22	Сборка frontend

Состав контейнеров, healthcheck и лимиты памяти - в `docs/architecture.md §5`. Команды сборки и запуска, URL и порты - в `docs/deployment.md §2` – `§3`. Целевой суммарный footprint ≈ 2.0–2.2 ГБ RAM (`docs/requirements/nonfunctional.md`,

«Аппаратные требования»).

5. Лицензионная совместимость

Требование: открытое ПО под MIT, Apache-2.0, BSD и аналогичными лицензиями либо ПО из реестра отечественного ПО (`docs/requirements/nonfunctional.md`, «Открытый исходный код»).

Компонент	Лицензия	Совместимость
Spring Boot / Spring/ AMQP / Security	Apache-2.0	Да
Gradle	Apache-2.0	Да
springdoc-openapi, JJWT	Apache-2.0	Да
Flyway (Community)	Apache-2.0	Да
PostgreSQL	PostgreSQL License (BSD-подобная)	Да
PostgreSQL JDBC	BSD-2-Clause	Да
Next.js, React, Tailwind, Node.js	MIT	Да
Nginx	BSD-2-Clause	Да
Mailpit	MIT	Да
RabbitMQ	MPL-2.0	Да (разрешённая)
Java (Temurin)	GPLv2+CE	Да (classpath exception)

Все прямые зависимости распространяются под OSI-совместимыми разрешительными лицензиями и не конфликтуют с закрытым или открытым распространением решения. Использование ПО из реестра отечественного ПО не требуется: требование закрывается открытыми лицензиями. При добавлении библиотеки проверяется лицензия и фиксируется в этом разделе; спорные случаи выносятся в `docs/assumptions.md`.

6. Открытость кода

Решение поставляется открытым и не зашифрованным:

- исходный код всех собственных компонентов (Java, TypeScript, конфигурации) находится в репозитории в виде исходников;
- обфускация не применяется; посторонние изменения исходников запрещены;
- стандартная сборка фреймворков (компиляция Java, бандлинг фронтенда с минификацией статики) не является обфускацией - исходники остаются доступными и воспроизводимо собираются по `docs/deployment.md`;
- сторонние зависимости и образы контейнеров используются как есть; их исходный код публичен, лицензии перечислены в §5;
- секреты и токены в репозиторий не попадают (`docs/deployment.md §4`).

Развёртывание и запуск

Назначение документа

Документ описывает пошаговую сборку, развёртывание и локальный запуск решения, конфигурацию и секреты, проверку работоспособности и демонстрационный сценарий с видео-фолбэком. Он закрывает пункт сопроводительной документации «пошаговая инструкция по сборке, развёртыванию и локальному запуску», требования «Контейнеризация» и «Качество кода», а также условие о демонстрации запуска по запросу жюри (`docs/requirements/acceptance.md`, `docs/requirements/nonfunctional.md`).

Связанные документы:

- `docs/architecture.md` - топология контейнеризации (§5), наблюдаемость и CI (§7).
- `docs/stack.md` - версии библиотек и образов.
- `docs/user_flow.md` - сквозные сценарии ролей (§4 - демонстрация).
- `docs/api.md` - индекс эндпоинтов и модель ошибок (§3.9).
- `docs/assumptions.md` - реестр принятых решений, включая TOPOLOGY A.

Источники требований: `docs/requirements/acceptance.md`, `docs/requirements/nonfunctional.md`, `docs/requirements/user-flow.md`.

Вне рамок документа: архитектурная декомпозиция, схемы данных, механики тестирования и подбора, лицензии (это зоны `docs/architecture.md`, `docs/data_model.md`, `docs/testing_model.md`, `docs/selection.md`, `docs/stack.md`).

1. Требования к окружению

Оборудование (обычная рабочая машина или недорогой виртуальный сервер; специальные требования ТЗ не предъявляет):

Ресурс	Минимум	Рекомендуется
RAM	4 ГБ	6–8 ГБ
CPU	2 ядра	4 ядра
Диск	5 ГБ	10 ГБ
Сеть	порты демо	открытый 80/443

Целевой суммарный footprint контейнеров - ≈ 2.0–2.2 ГБ RAM (`docs/architecture.md §5.1`), что согласуется с требованием «удерживать память в разумных пределах» (`docs/requirements/nonfunctional.md`, «Аппаратные требования»).

Программное обеспечение:

ПО	Версия	Назначение
Docker Engine	24+	Контейнеры
Docker Compose	v2	Оркестрация локального запуска
JDK (Temurin)	21	Локальная сборка backend
Node.js / npm	22 (LTS)	Локальная сборка frontend
Git	любая актуальная	Клонирование

ОС значения не имеет: сборка и запуск воспроизводимы на Windows, macOS и Linux (кросс-платформенность через Docker). Для сборки backend требуется JDK 21, для frontend - Node.js 22; для запуска только Docker достаточно.

2. Сборка

Backend - Gradle multi-project: один корневой `backend/build.gradle.kts` и модуль на сервис.

2.1. Сборка всех сервисов

```
./gradlew build
```

2.2. Сборка backend-сервисов

```
./gradlew :auth-service:bootJar
```

Каждый сервис даёт исполняемый Spring Boot JAR. Docker-образы собираются multi-stage (`eclipse-temurin:21-jdk` → `eclipse-temurin:21-jre`):

```
docker compose build
```

Frontend собирается отдельным модулем:

```
npm ci
npm run build
```

Образ SPA - multi-stage (`node:22-alpine` → `nginx:1.27-alpine`). Инфраструктурные образы (`postgres:16`, `rabbitmq:3.13`, `nginx:1.27`, `mailpit`) скачиваются из реестра; версии зафиксированы в `docs/stack.md §4`.

Конвейер CI (GitHub Actions) собирает образы per-service и проверяет контракты - состав шагов в `docs/architecture.md §7.3`.

3. Локальный запуск (docker-compose)

1. Подготовить конфигурацию:

```
cp .env.example .env
```

На Windows: `Copy-Item .env.example .env`.

2. Собрать и поднять стек:

```
docker compose up -d
```

Порядок старта задан `depends_on: service_healthy` (`docs/architecture.md §5.3`): сначала `postgres` и `rabbitmq`, затем бизнес-сервисы, затем `frontend` и `api-gateway`.

3. Наполнить каталог и демо-данные одноразовым контейнером `seed` (идемпотентно; повторный запуск - no-op):

```
docker compose run --rm seed
```

Порт-карта демонстрации:

Сервис	URL по умолчанию
SPA (frontend)	`http://localhost:3000`
api-gateway	`https://localhost`
RabbitMQ Management	`http://localhost:15672`
Mailpit (dev)	`http://localhost:8025`

Остановка и сброс:

```
docker compose down          # остановить
docker compose down -v       # остановить и удалить тома (сброс БД)
```

4. Конфигурация и секреты

Конфигурация - 12-factor, через переменные окружения. `.env.example` коммитится, `.env` в `.gitignore`; секреты в репозиторий не попадают, в CI встроена проверка (`docs/architecture.md §7.2`).

Переменная	Назначение
`POSTGRES_HOST`, `POSTGRES_PORT`	Адрес единого инстанса PostgreSQL

Переменная	Назначение
`POSTGRES_DB`	Имя базы
`AUTH_DB_ROLE`, `AUTH_DB_PASSWORD`	Роль и пароль схемы `auth`
`CANDIDATE_DB_ROLE`, `CANDIDATE_DB_PASSWORD`	Роль и пароль схемы `candidate`
... (аналогично по сервисам)	Отдельная роль на каждую схему
`RABBITMQ_DEFAULT_USER`, `RABBITMQ_DEFAULT_PASS`	Доступ к брокеру
`JWT_PRIVATE_KEY`, `JWT_PUBLIC_KEY`	Ключи RS256 (в `auth-service`)
`SMTP_HOST`, `SMTP_PORT`	Почта (dev - Mailpit)
`GATEWAY_TLS_CERT`, `GATEWAY_TLS_KEY`	TLS-сертификат шлюза (демо - self-signed)

TOPOLOGY A. Один контейнер PostgreSQL, по одной схеме и роли БД на сервис (`auth`, `candidate`, `fsp`, `assessment`, `selection`, `employer`, `outreach`, `notification`), плюс брокер RabbitMQ. Кросс-схемный доступ запрещён конвенцией (`docs/architecture.md §5`). Миграции Flyway применяются каждой службой в своей схеме.

Миграция A → B. Переход на физический PostgreSQL на сервис - конфигурационное изменение: обновление строк подключения (`POSTGRES_HOST`/порт на сервис) и перенос `PGDATA` схемы; код и контракты не меняются (`docs/architecture.md §5.4`). Решение зафиксировано в `docs/assumptions.md`.

TLS на демо - self-signed сертификат; путь к нему задаётся переменными выше. Для локального запуска без HTTPS предусмотреть включение gateway-маршрутов через `http`.

5. Проверка работоспособности и типовые проблемы

Проверки каждого сервиса (`docs/architecture.md §7.1`):

- `/healthz` - liveness: процесс жив;
- `/readyz` - readiness: доступны БД и брокер;
- `/metrics` - метрики в формате Prometheus (опциональный profile).

Быстрая проверка состояния:

```
docker compose ps          # все сервисы healthy
curl -fsS http://localhost:<port>/healthz
curl -fsS http://localhost:<port>/readyz
```

Smoke-тест compose. Скриптованный порядок: `docker compose up` → зелёные healthcheck → `seed` → скриптованный e2e-сценарий (`docs/architecture.md §7.3`). Тот же прогон закрывает требование «по запросу жюри команда демонстрирует запуск и работу кода» (`docs/requirements/nonfunctional.md`, «Качество кода») и выполняется одной командой на чистом окружении.

Типовые проблемы:

Симптом	Причина	Решение
`port is already allocated`	Занят порт демо	Освободить порт или изменить маппинг в compose
Сервис `unhealthy`	Не готова БД или брокер	Дождаться `postgres`/`rabbitmq`; проверить `/readyz`
Ошибки миграций Flyway	Том от старой версии схемы	`docker compose down -v` и повторный запуск
Письма не видны	Используется не Mailpit	Открыть `http://localhost:8025`
`401` на API	Истёк или невалиден JWT	Перевыпустить токен; проверить ключи RS256
Нехватка памяти	Мало свободной RAM	Отключить demo-профили (Prometheus/Grafana)
Пустой каталог	Не выполнен `seed`	`docker compose run --rm seed`

6. Демонстрационный сценарий и видео-фолбэк

Сквозной сценарий демонстрации повторяет целевой путь ТЗ (`docs/requirements/user-flow.md`) и детализацию шагов по ролям (`docs/user_flow.md §4`):

1. **Регистрация соискателя** - email и подтверждение адреса; письмо перехватывается Mailpit (`http://localhost:8025`).
2. **Опрос и тестирование** - опрос по отрасли и специализации, самостоятельный выбор предполагаемого грейда, тест на заявленный грейд (`docs/testing_model.md`).
3. **Присвоение категории** - по итогам попытки фиксируется категория (специализация × грейд), карточка становится видимой.
4. **Поиск работодателем** - работодатель открывает нужную категорию, видит ранжированную подборку (`docs/selection.md`).
5. **Выход на контакт** - LHH-сценарий: приглашение конкретному кандидату с уровнем заработной платы, статусы приглашения, раскрытие контактов после принятия.
6. **Классический отклик** `[ext]` - просмотр вакансий и отклик, если модуль реализован (границы - `docs/mvp_scope.md`).

Отдельно на демонстрации объясняются механика тестирования и механика подбора с обоснованием (`docs/testing_model.md`, `docs/selection.md`). Работа API и контракты показываются через `/docs` на шлюзе и через `docs/api.md §3.9`.

Видео-фолбэк. Если развернуть публичный прототип невозможно, по условиям ТЗ прикладывается демонстрационное видео работы прототипа и чёткая инструкция по локальному запуску, позволяющая эксперту развернуть решение самостоятельно (`docs/requirements/acceptance.md`, «Презентация и развёртывание»). Этой инструкцией служит настоящий документ; видео повторяет шаги §6 на локально поднятом стеке (§3).

Границы MVP

Назначение документа

Документ фиксирует границу MVP платформы PullHire: что входит в обязательную реализацию, что вынесено в опциональные срезы второй очереди и что сознательно не реализуется. Граница опирается на ТЗ (`docs/requirements/overview.md`, «Границы платформы»; `docs/requirements/acceptance.md`, «Что вне MVP» и «Что будет плюсом»), критерии оценки (`docs/requirements/evaluation.md`) и зафиксированные решения команды (`docs/assumptions.md §9`).

Связанные документы перечислены в §6.

Вне рамок: детальное описание каждой механики (оно в профильных документах), план-график работ и оценка трудоёмкости.

1. Принципы границы MVP

- **Сквозной путь важнее ширины.** Обязательный сценарий соискателя (регистрация → опрос → грейд → тест → категория) и основной сценарий работодателя (потребность → подборка → приглашение) реализуются целиком; вторичные потоки — по остатку.
- **Ядро ценности — LHH-механика.** Инициатива выхода на контакт принадлежит работодателю; это смысловой центр и главный критерий оценки (`docs/requirements/overview.md`, «Постановка задачи»).
- **Опциональные срезы проектируются полностью, реализуются во вторую очередь.** Судье показывается проработанный подход; код — после ядра.
- **Граница привязана к критериям оценки.** Обязательный объём максимизирует баллы ядра, опциональная часть — критерий «Дополнительный функционал» (7%).
- **Расширения помечаются `[ext]`**, а необязательные для MVP срезы — `[opt]`; параметры и решения фиксируются в `docs/assumptions.md`.

2. Входит в MVP

2.1. Роли и сценарии

- Две роли: `candidate` и `employer`; роль фиксируется при регистрации.
- Регистрация по электронной почте с подтверждением адреса (`docs/requirements/roles.md`, правило 4).
- Разграничение доступа по ролям: кандидат видит только свои приглашения, работодатель — банк кандидатов и свои приглашения.
- Кабинет кандидата: профиль, тестирование, категория и грейд, приглашения, настройки приватности.
- Кабинет работодателя: профиль компании, описание потребности, подборка, поиск, выход на контакт.
- Пошаговые пути — `docs/user_flow.md §2–§3`; сущности — `docs/data_model.md §4`, `docs/data_model.md §8`.

2.2. Профиль и резюме кандидата

- Заполнение профиля вручную по форме.
- Загрузка PDF с автораспознаванием полей (ФИО, контакты, стек, грейд, роли, стаж) — правилами; ML/NLP-разбор относится к §3.4.
- Автогенерация стандартизированного PDF-профиля.
- Настройки приватности и согласия на обработку и публикацию данных профиля (`docs/requirements/profiles.md`).
- Владелец «что видно» — `candidate-service` (`docs/architecture.md §3.1`); данные — `docs/data_model.md §4`.

2.3. ФСП-интеграция

- Связывание и отвязывание ФСП ID в настройках профиля.
- Получение достижений через mocked-адаптер реестра (реального API в рамках хакатона нет).
- Отображение достижений работодателю и корректная обработка случая «нет истории».
- Схема интеграции — `docs/fsp_integration.md`; сущности — `docs/data_model.md §5`.

2.4. Грейд-тестирование

- Обязательный конвейер при регистрации: опрос по отрасли и специализации → самостоятельный выбор заявленного грейда → тест на заявленный грейд → автоматическое присвоение категории.
- **MVP-форма грейд-теста состоит только из автопроверяемых форматов**: задачи с кодом, закрытые вопросы, короткий ответ. **Свободные ответы («предложить подход») в MVP-форму грейд-теста не входят** (Решение D, `docs/assumptions.md §9`); они перенесены в регулярные задания (§3.1).
- Грейд не понижается принудительно; повышение и понижение — по инициативе кандидата; cooldown смены грейда = 1 месяц.
- Механика — `docs/testing_model.md`; сущности — `docs/data_model.md §6.1`.

2.5. Категории и подборка

- Категория = специализация × грейд; присваивается по результату теста.
- Внутрикатегорийное ранжирование по силе подтверждённого профиля и достижениям ФСП.
- Подборка формируется на основе описания потребности работодателя; выдача сопровождается краткой объяснимостью.
- Поиск по банку кандидатов с фильтрами: специализация, грейд, категория, стек, наличие подтверждённых достижений ФСП; уточнение запроса без потери ранее полученной подборки.
- Механика — `docs/selection.md`; сущности — `docs/data_model.md §7`.

2.6. Приглашения и выход на контакт (LHH)

- Инициатива у работодателя: просмотр категоризированной базы и направление приглашения конкретному кандидату.
- Уровень заработной платы указывается обязательно; приглашение не требует привязки к опубликованной вакансии.
- Статусы приглашений: отправлено, просмотрено, принято, отклонено.
- Контактные данные раскрываются работодателю только после принятия приглашения (или самостоятельного отклика, если реализован срез §3.2).
- Механика — `docs/selection.md`; сущности — `docs/data_model.md §9.1`; правила — `docs/requirements/roles.md`, правила 1–3.

2.7. Архитектура и инфраструктура

- Микросервисы: `auth`, `candidate`, `fsp`, `assessment`, `selection`, `employer`, `outreach`, `notification`, плюс `api-gateway` и SPA.
- Backend: Java 21 + Spring Boot, Gradle, RabbitMQ, Flyway; frontend: React/Next.js.
- Топология A: одна PostgreSQL, схема и роль на сервис; миграция A → B документируется.
- Контейнеризация (Dockerfile, docker-compose), OpenAPI-спецификация, CI.
- Детали — `docs/architecture.md`, `docs/deployment.md`, `docs/api.md`, `docs/stack.md`.

2.8. Нефункциональные требования

- Стабильная работа при одновременном использовании десятками пользователей; приемлемое время формирования подборки, открытия профиля и поиска.
- Безопасность и 152-ФЗ: согласия на обработку и публикацию, разграничение прав по ролям, безопасное хранение паролей и токенов.
- Открытый, не скомпилированный и не обфусцированный код; лицензии MIT/Apache-2.0/BSD или реестр отечественного ПО.
- Кроссплатформенный веб-сервис для современных десктопных браузеров.
- Источник — `docs/requirements/nonfunctional.md`; развёртывание — `docs/deployment.md`.

3. Опциональные срезы (вторая очередь)

Каждый срез ниже проектируется полностью, но реализуется по остаточному принципу; все они дают вклад в критерий «Дополнительный функционал» (`docs/requirements/evaluation.md`, критерий 6, 7%). Срезы, необязательные для MVP, помечены `[opt]`.

Маркировка. Срез целиком помечается `[opt]`; сущности и эндпоинты внутри среза — `[ext]`.

3.1. Регулярные задания работодателей [opt]

- Полная механика — optional-MVP: каденс `n = 7` дней, промпты работодателя по вакансиям, добровольная подписка кандидата, случайный розыгрыш одного задания на окно, сдача, атрибут свежести профиля.
- **Свободные ответы («решить» / «предложить подход») живут именно здесь**, в периодических заданиях: их оценивает работодатель вручную (статусы `positive` | `negative` | `unreviewed`), без нагрузки 24/7. В MVP-грейд-тест свободный ответ не входит (§2.4).
- Результаты заданий никогда не изменяют грейд, категорию или историю грейда — обновляется только атрибут свежести профиля (display-only).
- Спецификация — `docs/periodic_tasks.md`; сущности — `docs/data_model.md §6.2–6.3`.

3.2. Классический поток вакансий и откликов [opt]

- Публикация вакансий работодателем (создание, редактирование), общий список вакансий, самостоятельный отклик кандидата, список своих откликов и их статусов.
- Сценарий дополнителен к основной LHH-механике: желателен, но не обязателен для MVP.
- Реализуется как опциональный модуль в `outreach-service` с документированным путём выделения (`docs/architecture.md §3.4`); сущности — `docs/data_model.md §9.3`.

3.3. Мобильная адаптивность [opt]

- Адаптивная вёрстка для мобильных устройств и планшетов — опционально для MVP (`docs/requirements/nonfunctional.md`, «Платформенность»).

3.4. ML/NLP-парсинг резюме [opt]

- Применение технологий машинного обучения и обработки естественного языка для разбора резюме, категоризации кандидатов и подбора рассматривается как преимущество при оценке (`docs/requirements/acceptance.md`, «Что будет плюсом»).
- В MVP работает rule-based fallback; ML/NLP-воркер — расширение `[ext]` (`docs/architecture.md §3.4`).

4. Не входит в MVP

- **Защита от фиктивных вакансий и недобросовестных работодателей.** ТЗ ожидает здесь не рабочую реализацию, а концепцию и архитектурный подход, дорабатываемый с ФСП после хакатона (`docs/requirements/overview.md`, «Границы платформы»). Сюда же относятся верификация компании-работодателя и модерация вакансий (`docs/requirements/roles.md`, правило 4).
- **Интеграция с внутренними ATS-системами работодателей.** Ожидается только концепция (`docs/requirements/acceptance.md`, «Что будет плюсом»).
- **Свободные ответы в грейд-тесте.** Исключены из MVP-формы (Решение D, `docs/assumptions.md §9`): обязательны только автопроверяемые форматы; свободный ответ допустим только в регулярных заданиях (§3.1) и как `[ext]`-расширение оценивания.
- **Реальный API реестра ФСП.** Доступ и описание структуры данных не предоставляются — работа на mock/synthetic данных (`docs/requirements/acceptance.md`).
- Публичная витрина чужих вакансий и парсинг сторонних площадок — вне продукта по постановке задачи (`docs/requirements/overview.md`).

4.1. Концепция защиты от фиктивных вакансий и недобросовестных работодателей

> **Post-MVP.** Ниже — только концепция и архитектурный подход, дорабатываемый с ФСП после хакатона; в MVP не реализуется. Ждём от этого пункта именно концепцию, а не код (`docs/requirements/overview.md`, «Границы платформы»; `docs/requirements/roles.md`, правило 4).

- **Уровни доверия работодателя (employer trust/verification tiers).** Компания проходит поэтапную верификацию (подтверждение домена и юрлица, ручная проверка реквизитов, сверка с внешними реестрами, в перспективе — сигналы ФСП); уровень доверия отображается кандидату и управляет доступом к выходу на контакт.
- **Репутация работодателя.** Накапливаемые метрики поведения (доля принятых приглашений, доля «нет ответа», жалобы за период, полнота профиля компании) формируют видимый кандидату репутационный балл без раскрытия персональных данных.
- **Жалобы кандидатов (candidate reports).** Кандидат может пожаловаться на вакансию или поведение работодателя; жалоба попадает в очередь модерации, влияет на уровень доверия и может инициировать карантин.

- **Сигналы фиктивной вакансии (fake-vacancy signals).** Совокупность признаков: некорректные/недостижимые контакты; дублирующиеся или шаблонные объявления; несоответствие заявленной зарплаты и стека; аномально высокая доля приглашений без ответа (`no-response rate`); признаки массового сбора резюме без намерения найма.
- **Поток модерации и карантина (moderation/quarantine flow).** Сигналы агрегируются в скоринг риска; при превышении порога объявление/аккаунт переводятся в карантин (скрытие из выдачи) до ручного разбора модератором; решения аудируются, ложноположительные сценарии предусматривают аппеляцию. Концепция дополняет, но не заменяет верификацию компании и модерацию вакансий (`docs/requirements/roles.md`, правило 4).

4.2. Концепция интеграции с ATS

> **Post-MVP.** Ниже — только концепция и архитектурный подход; в MVP не реализуется. ТЗ ожидает концепцию (`docs/requirements/acceptance.md`, «Что будет плюсом»).

- **Изолированный модуль-адаптер.** Интеграция оформляется как отдельный адаптер (по образцу шва `fsp-service`), а не как встроенная логика ядровых сервисов.
- **Исходящая синхронизация статусов.** Адаптер выгружает в ATS работодателя статусы приглашений и откликов (отправлено/просмотрено/принято/отклонено), при необходимости — событие принятия и раскрытие контакта.
- **Целевые системы.** Greenhouse/Workable/Lever-подобные ATS; различия скрыты за адаптером, ядро о них не знает.
- **Стабильная граница контракта.** Обмен идёт через существующую событийную/контрактную границу (события жизненного цикла приглашения и стабильный интерфейс адаптера); ядровые сервисы не связываются с ATS напрямую — сопряжение отсутствует (`no core coupling`).

5. Обоснование границ

- **Фокус на обязательном пути** максимизирует баллы ядра: LHH-механика — 35%, тестирование и категоризация — 25%, удобство сквозного пути — 10%, полнота реализации — 6%.
- **Опциональные срезы** (§3) — прямой вклад в 7% за дополнительный функционал; они спроектированы целиком, чтобы судья видел проработанный подход, но реализуются после ядра.
- **Не-в-MVP** (§4) ТЗ прямо относит к концепции post-hackathon: реализация не даёт больше баллов, чем документированный архитектурный подход, и расходует ограниченный ресурс команды.
- **Решение D** (только автопроверяемые форматы в грейд-тесте) снимает риск необеспеченной проверки свободных ответов в MVP и переносит их в периодические задания, где оценка обеспечена человеком-работодателем.
- **Отслеживаемость решений.** Параметры и отклонения зафиксированы в `docs/assumptions.md`; состав реализованного против отложенного подводится в `docs/final_summary.md`.

6. Связанные документы

- `docs/architecture.md` — архитектура, сервисы, топология, событийное взаимодействие.
- `docs/data_model.md` — ER-модель, сущности и поля, ссылочные схемы.
- `docs/testing_model.md` — механика грейд-тестирования, банк заданий, скоринг, cooldown.
- `docs/periodic_tasks.md` — регулярные задания работодателей (optional-MVP).
- `docs/selection.md` — категории, ранжирование, подборка, LHH-выход на контакт.
- `docs/validation.md` — процедура валидации решения и результаты.
- `docs/fsp_integration.md` — схема интеграции с реестром ФСП.
- `docs/api.md` — обзор API и конвенции.
- `docs/stack.md` — библиотеки и компоненты с версиями.
- `docs/deployment.md` — сборка, развёртывание, локальный запуск.
- `docs/user_flow.md` — сквозные пользовательские сценарии и демо-путь.
- `docs/glossary.md` — термины.
- `docs/assumptions.md` — реестр решений и параметров; §9 — Решение D о свободных ответах.
- `docs/final_summary.md` — итоговая сводка: реализовано против ТЗ, карта решений.
- `docs/backlog/closed_questions.md` — журнал закрытых вопросов и техдолга.
- Источники: `docs/requirements/overview.md`, `docs/requirements/acceptance.md`, `docs/requirements/evaluation.md`, `docs/requirements/roles.md`, `docs/requirements/testing.md`, `docs/requirements/nonfunctional.md`, `docs/requirements/profiles.md`.

Пользовательские сценарии

Назначение документа

Документ описывает сквозные пользовательские сценарии платформы PullHire в обоих кабинетах — пошагово, на уровне интерфейсных шагов, — а также демонстрационный сценарий для защиты. Описываются шаги и переходы, а не внутренняя логика механик: устройство тестирования — в `docs/testing_model.md`, категории, подборка и выход на контакт — в `docs/selection.md`, регулярные задания — в `docs/periodic_tasks.md`.

Основание: `docs/requirements/user-flow.md`, `docs/requirements/roles.md`, `docs/requirements/acceptance.md` (презентация и развёртывание), `docs/requirements/evaluation.md` (критерии 3 и 5).

Связанные документы: `docs/testing_model.md`, `docs/selection.md`, `docs/periodic_tasks.md` (механики, на которые ссылаются шаги), `docs/mvp_scope.md` (границы MVP), `docs/fsp_integration.md` (связывание ФСП ID), `docs/data_model.md` (сущности), `docs/deployment.md` (демонстрация и запуск), `docs/glossary.md` (термины).

Вне рамок: внутренние алгоритмы механик, дизайн-макеты, контракты API.

1. Общая легенда пути

Платформа связывает напрямую только работодателя и соискателя. Путь двусторонний: кандидат проходит обязательный онбординг и становится видимым, работодатель описывает потребность и по ней получает подборку, после чего сам выходит на контакт. Ключевые инварианты пути:

- **Кандидат невидим работодателям до события `assessment.category_assigned`.** До присвоения категории он не появляется в поиске и подборке (`docs/testing_model.md §8`).
- **Контактные данные закрыты до принятия приглашения.** Работодатель не видит контакты, пока кандидат не принял приглашение (или не откликнулся сам, если реализован классический поток) — `docs/requirements/roles.md`, правило 2.
- **Инициатива выхода на контакт — у работодателя** (LHH-механика) — `docs/requirements/overview.md`, «Постановка задачи».

Схема путей:

```
Кандидат:      регистрация → согласия → профиль → ФСП ID → опрос → выбор грейда → тест
               → assessment.category_assigned → карточка видна работодателю
               → приглашение → принятие → раскрытие контактов

Работодатель:  регистрация → компания → описание потребности → подборка + объяснимость
               → поиск и фильтры → приглашение → (после принятия) контакты
```

Каждый шаг ниже представлен таблицей «Шаг / Действие в интерфейсе / Результат».

2. Сценарий кандидата

2.1. Регистрация и согласия

Шаг	Действие в интерфейсе	Результат
1	Открывает лендинг → «Регистрация»	Форма регистрации
2	Вводит email и пароль	Отправлено письмо-подтверждение
3	Переходит по ссылке из письма	Адрес подтверждён, аккаунт активен
4	Отмечает согласие на обработку и публикацию данных профиля	Записаны согласия (152-ФЗ)
5	Входит в кабинет кандидата	Кабинет `candidate` доступен

Основание: `docs/requirements/roles.md`, правило 4. Запись согласий — `docs/data_model.md §3.4`; аутентификация — `docs/architecture.md §6.1`.

2.2. Профиль и резюме

Шаг	Действие в интерфейсе	Результат
1	Заполняет профиль по форме (ФИО, контакты, стек, роли, стаж)	Черновик профиля
2	Или загружает PDF и запускает автораспознавание	Поля предзаполнены, кандидат правит
3	Настраивает приватность (что видно работодателю)	Сохранены настройки видимости
4	Скачивает/обновляет стандартизированный PDF-профиль	Готов PDF профиля

Основание: `docs/requirements/profiles.md`, «Профиль и резюме кандидата». Сущности — `docs/data_model.md §4`. Автораспознавание — rule-based; ML/NLP-разбор относится к опциональному срезу (`docs/mvp_scope.md §3.4`).

2.3. Связывание ФСП ID

Шаг	Действие в интерфейсе	Результат
1	В настройках указывает ФСП ID → «Связать»	Запрос на связывание
2	Система через mocked-адаптер получает достижения	Достижения показаны в профиле
3	(если истории нет) Видит отметку «нет истории»	Корректная обработка отсутствия данных
4	Может отвязать ID	Связь снята

Основание: `docs/requirements/profiles.md`, «Связь с ID участника ФСП»; схема — `docs/fsp_integration.md`; сущности — `docs/data_model.md §5`.

2.4. Опрос, грейд и тестирование

Шаг	Действие в интерфейсе	Результат
1	Проходит опрос по отрасли и специализации	Заполнен профиль домена
2	Самостоятельно выбирает предполагаемый грейд	Заявленный грейд (`declared_grade`)
3	Запускает тест на заявленный грейд (автопроверяемые задания)	Открыта сессия тестирования
4	Проходит адаптивную сессию	Накоплена оценка `θ` и `SE`
5	Получает результат: полоса грейда и категория (либо «требует перепроверки»)	Присвоена категория либо ручная проверка

Основание: `docs/testing_model.md §2`, `§5`, `§6`, `§8`. Грейд не понижается принудительно; cooldown смены грейда = 1 месяц (`docs/testing_model.md §7`). В MVP-форму входят только автопроверяемые форматы (`docs/mvp_scope.md §2.4`).

2.5. Карточка и видимость

Шаг	Действие/событие	Результат
1	Завершение попытки публикует `assessment.category_assigned`	Обновлена проекция категории
2	Карточка кандидата появляется в банке	Видны категория, специализация, грейд, достижения ФСП
3	(до события шага 1) Кандидат отсутствует в поиске и подборке	**Кандидат невидим работодателям**
4	Контактные данные в карточке скрыты	Раскрытие — только через §2.6

Основание: `docs/testing_model.md §8`; проекция карточки — `docs/data_model.md §7.3` (свежесть в `selection.candidate_cards` не входит). Работодателю доступна только категория, но не результаты попытки и не `θ` (`docs/testing_model.md §8`). Сигнал свежести отображается отдельно, в employer-представлении (`docs/periodic_tasks.md §6.4`).

2.6. Приглашение и раскрытие контактов

Шаг	Действие в интерфейсе	Результат
1	Получает приглашение (предложение, зарплата, компания, способ связи)	Приглашение в списке входящих
2	Видит актуальный статус приглашения	Статус: отправлено / просмотрено
3	Принимает или отклоняет	Статус: принято / отклонено
4	(при принятии) Система публикует `invitation.accepted` → `candidate.contact_released`	Работодателю раскрыты контакты

Основание: `docs/requirements/roles.md`, правила 2–3; механика — `docs/selection.md`; сущности — `docs/data_model.md §9.1–9.2`. Уровень зарплаты указывается обязательно (`docs/requirements/roles.md`, правило 1).

2.7. Подписка на регулярные задания [opt]

Раздел описывает опциональный срез (`[opt]`), необязательный для MVP (`docs/mvp_scope.md §3.1`).

Шаг	Действие в интерфейсе	Результат
1	Включает участие в регулярных заданиях (по умолчанию ВЫКЛЮЧЕНО)	Сохранён opt-in
2	При наличии категории попадает в розыгрыш	Кандидат правомочен
3	Раз в окно `n = 7` дней получает одно случайное задание	Задание доступно
4	Решает или предлагает подход; отправляет ответ	Ответ уходит на оценку работодателю
5	Сдача обновляет свежесть профиля (display-only)	Бейдж «актуальный профиль»

Основание: `docs/periodic_tasks.md §4–§6`. Результаты заданий не меняют грейд/категорию (`docs/periodic_tasks.md §6.3`).

3. Сценарий работодателя

3.1. Регистрация и профиль компании

Шаг	Действие в интерфейсе	Результат
1	Регистрация по email и подтверждение адреса	Аккаунт `employer`
2	Отмечает согласия	Согласия записаны
3	Заполняет профиль компании (описание, направление, контакты)	Профиль компании сохранён

Основание: `docs/requirements/profiles.md`, «Профиль компании»; `docs/requirements/roles.md`, «Работодатель». Сущности — `docs/data_model.md §8.1`.

3.2. Описание потребности

Шаг	Действие в интерфейсе	Результат
1	Создаёт описание потребности: нужные специалисты, стек, специализация, грейд, чем занимается команда	Потребность сохранена
2	Запрашивает подборку	Запущено формирование подборки

Основание: `docs/requirements/profiles.md`, «Описание потребности»; `docs/requirements/roles.md`, «Работодатель». Сущности — `docs/data_model.md §8.2`.

3.3. Подборка и объяснимость

Шаг	Действие в интерфейсе	Результат
1	Открывает подборку	Рекомендованные категории и кандидаты
2	Читает краткую объяснимость по каждому кандидату	Понятно, почему кандидат в выдаче
3	Просматривает карточки кандидатов	Контакты скрыты

Основание: `docs/requirements/selection.md`, «Формирование подборки», «Объяснимость ранжирования и выдачи»; механика — `docs/selection.md`.

3.4. Поиск и фильтры

Шаг	Действие в интерфейсе	Результат
1	Переходит в банк кандидатов	Доступ ко всей базе категоризированных кандидатов
2	Фильтрует по специализации, грейду, категории, стеку, наличию достижений ФСП	Отфильтрованная выдача

Шаг	Действие в интерфейсе	Результат
3	Уточняет запрос	Ранее полученная подборка не теряется

Основание: `docs/requirements/selection.md`, «Поиск и фильтрация банка кандидатов».

3.5. Приглашение и просмотр контактов

Шаг	Действие в интерфейсе	Результат
1	Открывает карточку → «Пригласить»	Форма приглашения
2	Указывает описание предложения и уровень зарплаты (обязательно); вакансия необязательна	Приглашение готово
3	Отправляет приглашение	Статус «отправлено»; отслеживаются смены статусов
4	После принятия кандидатом контакты раскрываются	Доступны контактные данные кандидата

Основание: `docs/requirements/roles.md`, правила 1–3; механика — `docs/selection.md`; сущности — `docs/data_model.md §9.1`. Классический поток вакансий и отклик кандидата — опциональный срез `[opt]` (`docs/mvp_scope.md §3.2`).

3.6. Регулярные задания работодателя [opt]

Раздел описывает опциональный срез (`[opt]`), необязательный для MVP (`docs/mvp_scope.md §3.1`).

Шаг	Действие в интерфейсе	Результат
1	Отвечает на промпт по вакансии `{add	update
2	Просматривает сданные решения по своим заданиям	Видны ответы кандидатов
3	Оценивает: `positive` / `negative` / `unreviewed`	Результат задания записан
4	Видит сигнал свежести профиля	Контакты кандидата остаются закрытыми

Основание: `docs/periodic_tasks.md §3`, `§6`; адресная выдача кандидату запрещена (`docs/periodic_tasks.md §3.4`).

4. Демонстрационный сценарий

Сквозной сценарий для защиты воспроизводит обязательный путь ТЗ (`docs/requirements/acceptance.md`, «Презентация и развёртывание»; `docs/requirements/user-flow.md`, «Демонстрационный сценарий»):

1. **Регистрация соискателя** с подтверждением адреса (§2.1).
2. **Опрос и тестирование**: опрос по отрасли/специализации, выбор грейда, прохождение теста (§2.4).
3. **Присвоение категории**: показ события `assessment.category_assigned` и появления карточки в банке (§2.5).
4. **Поиск работодателем нужной категории**: описание потребности → подборка с объяснимостью → фильтры (§3.2–§3.4).
5. **Выход на контакт**: приглашение с указанием зарплаты → принятие кандидатом → раскрытие контактов (§3.5, §2.6).
6. **Классический отклик на вакансию** — если срез реализован `[opt]` (`docs/mvp_scope.md §3.2`).
7. Отдельно демонстрируются и объясняются **механика тестирования** (`docs/testing_model.md`) и **механика подбора** (`docs/selection.md`).

Демонстрация дополняется показом запуска прототипа; порядок сборки и локального старта — в `docs/deployment.md`. Если развёрнутый прототип недоступен, прилагается демонстрационное видео и инструкция по локальному запуску (`docs/requirements/acceptance.md`).

5. Связанные документы

- `docs/mvp_scope.md` — границы MVP и опциональные срезы, помеченные `[opt]`.
- `docs/testing_model.md` — механика грейд-тестирования и присвоение категории.
- `docs/selection.md` — категории, ранжирование, подборка, LHH-выход на контакт.
- `docs/periodic_tasks.md` — регулярные задания работодателей (optional-MVP).
- `docs/fsp_integration.md` — связывание ФСП ID и отображение достижений.
- `docs/data_model.md` — сущности и поля, к которым отсылают шаги.
- `docs/architecture.md` — сервисы, события, контактный гейтинг.

- `docs/deployment.md` — сборка, развёртывание и демонстрация.
- `docs/glossary.md` — термины.
- `docs/assumptions.md` — реестр решений и параметров.
- Источники: `docs/requirements/user-flow.md`, `docs/requirements/roles.md`, `docs/requirements/acceptance.md`, `docs/requirements/evaluation.md`.

Глоссарий

Назначение документа

Документ собирает в одном месте термины, используемые в пакете документации PullHire, с определениями и ссылками на места употребления. Каждый термин определяется один раз; профильные документы опираются на эти определения, а не переопределяют их.

Термины сгруппированы по уровням: базовые понятия предметной области (§1), понятия платформы (§2), понятия механик (§3), термины ФСП (§4) и архитектурные термины (§5).

Вне рамок: полные описания механик (в профильных документах) и параметры решений (в `docs/assumptions.md`).

1. Базовые термины

Термин	Определение	Где используется
Отрасль (industry)	Крупное направление ИТ-деятельности, к которому относится кандидат; фиксируется опросом (`industry_code`).	`docs/testing_model.md §2.1`; `docs/data_model.md §4.1`
Специализация (specialization)	Конкретное направление внутри отрасли (например, бэкенд-разработка); задаётся опросом и входит в определение категории.	`docs/testing_model.md §2.1`; `docs/data_model.md §7.1`
Грейд (grade)	Уровень квалификации кандидата (джуниор/мидл/сеньор и аналоги); заявляется кандидатом и уточняется результатом теста.	`docs/testing_model.md §2.2`, `§6.2`; `docs/data_model.md §7.1`
Стек (stack)	Набор технологий и инструментов кандидата или требуемый в потребности; используется для описания профиля и фильтрации.	`docs/requirements/profiles.md`; `docs/requirements/selection.md`
Кандидат (candidate)	Соискатель; проходит обязательный онбординг и становится видимым работодателям после присвоения категории.	`docs/requirements/roles.md`, «Кандидат»; `docs/user_flow.md §2`
Работодатель (employer)	Представитель компании, который описывает потребность, работает с подборкой и инициирует выход на контакт.	`docs/requirements/roles.md`, «Работодатель»; `docs/user_flow.md §3`

2. Термины платформы

Термин	Определение	Где используется
Категория	Группа однотипных соискателей, в основе которой связка специализации и грейда (например, бэкенд-разработчики с опытом); присваивается по результату теста и видна работодателю.	`docs/testing_model.md §8`; `docs/selection.md`; `docs/data_model.md §7.2`
Верифицированный профиль	Профиль кандидата, чья категория и позиция подтверждены объективными данными — результатом теста и достижениями ФСП, — а не самоописанным резюме.	`docs/selection.md`; `docs/testing_model.md §8`
Потребность (need)	Описание работодателем нужных специалистов: стек, специализация, грейд, чем занимается команда; вход для формирования подборки.	`docs/requirements/profiles.md`, «Описание потребности»; `docs/data_model.md §8.2`
Подборка	Рекомендованный набор категорий и конкретных кандидатов, сформированный под потребность работодателя и сопровождаемый краткой объяснимостью.	`docs/selection.md`; `docs/user_flow.md §3.3`
LHH-механика	Механика выхода на контакт с обратной инициативой: инициативу проявляет работодатель, который просматривает категоризированную базу и направляет приглашение конкретному кандидату (ориентир — сервис LHH).	`docs/requirements/overview.md`; `docs/selection.md`
Приглашение (invitation)	Предложение работодателя конкретному кандидату: описание предложения, уровень зарплаты, название компании и способ связи; без обязательной привязки к опубликованной вакансии.	`docs/requirements/roles.md`, правила 1–2; `docs/data_model.md §9.1`
Статус приглашения	Текущее состояние взаимодействия по приглашению: отправлено, просмотрено, принято, отклонено; видно обеим сторонам.	`docs/requirements/roles.md`, правило 3; `docs/data_model.md §11`

3. Термины механик

Термин	Определение	Где используется
Грейд-тест	Обязательное тестирование на заявленный грейд в составе конвейера «опрос → заявленный грейд → тест → категория»; в MVP состоит только из автопроверяемых форматов.	`docs/testing_model.md §1`, `§3.1`; `docs/mvp_scope.md §2.4`
θ (theta)	Оценка уровня кандидата на общей шкале IRT; является итоговой метрикой вместо процента правильных ответов.	`docs/testing_model.md §5.1`, `§6.1`, `§9.4`
SE (standard error)	Стандартная ошибка оценки `θ`; используется в правилах остановки и для пометки «требует перепроверки».	`docs/testing_model.md §5.2`, `§6.3`
Якорное задание (anchor item)	Небольшое защищённое задание фиксированной экспертной сложности, включаемое в каждую форму для эквайтинга и обнаружения дрейфа.	`docs/testing_model.md §3.3`, `§9.3`
Cut-score	Документированные границы, по которым непрерывная шкала `θ` разбивается на полосы грейдов и переводится в категории.	`docs/testing_model.md §6.2`
Cooldown	Минимальный интервал между сменами грейда; для платформы принят равным 1 месяцу.	`docs/testing_model.md §7.3`; `docs/data_model.md §6.1`
Регулярные задания (periodic tasks)	Периодически предлагаемые кандидату короткие задачи от работодателя с ответом «решить» или «предложить подход»; поддерживают актуальность профиля.	`docs/periodic_tasks.md §2`; `docs/mvp_scope.md §3.1`
Свежесть профиля (profile freshness)	Атрибут, обновляемый при сдаче регулярного задания; по умолчанию отображается, но не влияет на ранжирование; не меняет грейд/категорию.	`docs/periodic_tasks.md §6.3`; `docs/data_model.md §6.3`
Свободный ответ (free-form)	Ответ в виде текста/подхода, оцениваемый не автоматически; исключён из MVP-грейд-теста и живёт только в регулярных заданиях.	`docs/testing_model.md §3.1`; `docs/mvp_scope.md §2.4`, `§4`

4. Термины ФСП

Термин	Определение	Где используется
ФСП ID	Идентификатор участника Федерации спортивного программирования, который профиль кандидата может связать с собой для получения достижений.	`docs/requirements/profiles.md`, «Связь с ID участника ФСП»; `docs/data_model.md §5.1`
Достижения (ФСП)	Подтверждённые данные о результатах кандидата в соревнованиях и мероприятиях ФСП, обогащающие профиль и учитываемые при ранжировании.	`docs/requirements/profiles.md`, «Достижения ФСП в профиле»; `docs/data_model.md §5.2`
«Нет истории»	Корректно обработанное состояние профиля при отсутствии у кандидата истории ФСП; не является ошибкой и не блокирует путь.	`docs/requirements/profiles.md`; `docs/fsp_integration.md`
Keycloak-совместимость	Ориентир на совместимость аутентификации с Keycloak (формат claims, JWKS), упрощающий последующую интеграцию ФСП; не является обязательным требованием.	`docs/requirements/nonfunctional.md`, «Аутентификация»; `docs/architecture.md §6.1`

5. Архитектурные термины

Термин	Определение	Где используется
Сервис (service)	Самостоятельный backend-компонент микросервисной архитектуры, владеющий своим контекстом и данными (например, `assessment-service`, `selection-service`).	`docs/architecture.md §3`; `docs/data_model.md §2`
Схема БД (schema)	Логически выделенное пространство таблиц PostgreSQL, принадлежащее одному сервису и доступное только через его роль БД.	`docs/architecture.md §5.2`; `docs/data_model.md §1`
Событие (event)	Асинхронное сообщение RabbitMQ в общем конверте, публикуемое сервисом-продюсером и потребляемое подписчиками (например, `assessment.category_assigned`).	`docs/architecture.md §4.2`
Outbox (transactional outbox)	Приём надёжной публикации критичных событий: запись события в ту же транзакцию, что и изменение данных, с последующей отправкой брокеру.	`docs/architecture.md §4.3`; `docs/testing_model.md §8`
Проекция (projection)	Локальная согласованная со временем копия данных другого сервиса, наполняемая событиями и используемая для чтения на горячем пути.	`docs/architecture.md §4.3`; `docs/data_model.md §7.2`

Термин	Определение	Где используется
TOPOLOGY A / TOPOLOGY B	A — одна PostgreSQL с отдельной схемой и ролью на сервис (MVP); B — физически отдельная СУБД на сервис; переход A → B выполняется сменой конфигурации без переписывания кода.	`docs/architecture.md §5.4`; `docs/assumptions.md`

Допущения

Назначение документа

Документ — единый реестр решений команды PullHire, принятых там, где техническое задание оставляет выбор за командой или не задаёт значение. Каждая запись фиксирует суть решения, источник, обоснование и влияние на реализацию. Реестр дополняет журнал `docs/backlog/closed_questions.md` (история закрытия вопросов) и уточняет границы `docs/mvp_scope.md`.

Связанные документы: `docs/testing_model.md` (§3, §7), `docs/periodic_tasks.md` (§2–§7), `docs/fsp_integration.md` (§3), `docs/data_model.md` (§5, §6), `docs/architecture.md` (§5), `docs/mvp_scope.md`, `docs/backlog/closed_questions.md`.

Источники требований: `docs/requirements/acceptance.md` («Элементы, прорабатываемые самостоятельно»), `docs/requirements/overview.md`, `docs/requirements/testing.md`, `docs/requirements/profiles.md`, `docs/requirements/nonfunctional.md`.

Вне рамок: детальные алгоритмы и структуры полей — в профильных документах; здесь только решения и их влияние.

1. Формат записи

Каждое допущение оформлено отдельным разделом `## N.` и состоит из подразделов:

- `### Что` — предмет решения и пробел в требованиях;
- `### Решение` — принятое значение или правило;
- `### Источник` — где ТЗ оставляет выбор и на основании чего решение принято;
- `### Обоснование` — почему выбрано именно это;
- `### Влияние` — на какие документы и сущности влияет;
- `### Связанные вопросы` — ссылки на `docs/backlog/closed_questions.md`, если вопрос поднимался.

Идентификаторы (сервисы, поля, события, значения перечислений) — в английском коде; пояснения — по-русски. Маркеры: `[ext]` — расширение вне MVP, `[opt]` — необязательный срез. Принятые значения не дублируются в профильных документах, а цитируются оттуда.

2. Grade-change cooldown = 1 месяц

Что

ТЗ требует ограничить частоту смены грейда, но конкретный интервал оставляет команде (`docs/requirements/overview.md`, «Постановка задачи»).

Решение

Cooldown смены грейда = **1 месяц**. Контролируется таблицей `cooldowns` схемы `assessment`; проверка права — эндпоинт `/assessment/grade-eligibility`.

Источник

`docs/requirements/testing.md` («Ограничение смены грейда»); `docs/requirements/overview.md` («…не чаще одного раза в три месяца — конкретный интервал команда выбирает сама»).

Обоснование

1 месяц — минимально достаточный интервал, чтобы новая попытка следовала за реальным ростом кандидата, а не за «перебором» сложности, и при этом не завышает входной барьер в демо. Отдельная таблица вместо поля истории развязывает правило и аудит.

Влияние

`docs/testing_model.md §7.3`; `docs/data_model.md §6.1` (`cooldowns`); API `assessment-service`.

Связанные вопросы

`docs/backlog/closed_questions.md` — Q-3.

3. Периодические задания: n = 7, синхронные окна, opt-in

Что

ТЗ задаёт идею регулярных коротких заданий, но не определяет каденс, объём, участие и вес свежести.

Решение

- `n = 7` дней — глобальная константа обоих каденсов (промпты работодателя и выдачи кандидатам); дедлайн `d = n`;
- окна **глобальные синхронные** (не per-account rolling), идемпотентность через `cycle_date`;
- участие — **opt-in, по умолчанию ВЫКЛЮЧЕНО**; согласие `task_participation`;
- ровно одна выдача на кандидата в окно; дедупликация на **уровне задания** (task-level);
- `[ext]` лимит повторов `re-offer cap = 2`;
- пул ограничивается специализацией кандидата (обязательный фильтр); фильтр по грейду — `[ext]`;
- свежесть профиля `ProfileFreshness` — **display-only**, в ранжирование не входит.

Источник

`docs/requirements/testing.md` («Регулярные задания»); `docs/requirements/roles.md` («Кандидат»; «желательно, но не обязательно для MVP»).

Обоснование

Синхронные окна упрощают идемпотентность и допускают повторный запуск планировщика без дублей. Opt-in защищает кандидата и соответствует 152-ФЗ. Display-only свежести исключает «игру с механикой» и ранговый выигрыш за счёт объёма заданий: задания никогда не меняют грейд и категорию (hard rule).

Влияние

`docs/periodic_tasks.md §2–§7`; `docs/data_model.md §6.2–6.3`; `docs/mvp_scope.md §3.1`.

Связанные вопросы

`docs/backlog/closed_questions.md` — Q-2.

4. Содержимое каталога специализаций и грейдов

Что

Справочник специализаций и грейдов команда составляет самостоятельно; на MVP достаточно основных направлений (`docs/requirements/acceptance.md`; `docs/requirements/overview.md`).

Решение

- специализации (MVP): `backend`, `frontend`, `fullstack`, `mobile`, `devops`, `qa`, `data`, `ml`;
- грейды (MVP): `intern`, `junior`, `middle`, `senior`, `lead`.

Справочник — `taxonomy` в схеме `selection`; категория = специализация × грейд.

Источник

`docs/requirements/overview.md` («Справочник специализаций и грейдов команда составляет самостоятельно… достаточно основных направлений»); `docs/requirements/acceptance.md` («Элементы, прорабатываемые самостоятельно»).

Обоснование

Основные ИТ-направления и общепринятая грейдовая шкала достаточны для демонстрации сквозного пути и подбора. Расширение каталога — это данные, а не код, поэтому закладываться на полный отраслевой классификатор на MVP не требуется.

Влияние

`docs/data_model.md §7.1` (`taxonomy`); `docs/selection.md`; `docs/periodic_tasks.md §4` (scope по специализации); `docs/testing_model.md §2.1`.

Связанные вопросы

Вопрос не выделялся; решение команды в рамках ТЗ.

5. Состав полей ФСП

Что

API и структура данных реестра ФСП не предоставляются; состав получаемых и отображаемых сведений о достижениях команда определяет сама (`docs/requirements/acceptance.md`; `docs/requirements/profiles.md`).

Решение

Принята собственная минимальная модель:

- `fsp_links`: `fsp_id`, `link_status` (`pending` | `linked` | `unlinked` | `error`), `history_status` (`unknown` | `present` | `no_history`), timestamps;
- `achievements_cache`: `title`, `event_type`, `achieved_at`, `place`, `payload` (`jsonb`).

Работодателю показываются `title`, `event_type`, `achieved_at`, `place`. Случай «нет истории» (`history_status = no_history`) обрабатывается явно и нейтрально для ранжирования. Keycloak-совместимость авторизации — ориентир, не требование.

Источник

`docs/requirements/profiles.md` («Связь с ID участника ФСП», «Достижения ФСП в профиле»); `docs/requirements/acceptance.md` («Ресурсы организатора»).

Обоснование

Плоская модель устойчива к неизвестному контракту: `payload` хранит произвольную структуру, а стабильные базовые поля образуют публичный контракт. Замена mock-реестра на боевой затрагивает только адаптер и маппинг, не остальные сервисы.

Влияние

`docs/fsp_integration.md §2–§7`; `docs/data_model.md §5`; `docs/architecture.md §5`.

Связанные вопросы

`docs/backlog/closed_questions.md` — Q-1, Q-4.

6. Форматы заданий грейд-теста

Что

Содержание и формат заданий команда определяет сама (`docs/requirements/testing.md`, «Формат заданий»; `docs/requirements/acceptance.md`).

Решение

MVP-форма грейд-теста состоит **только из автопроверяемых форматов**:

- одиночный / множественный выбор;
- короткий ответ (число / строка);
- задача с кодом (автотесты + проверка схожести).

Свободный ответ («предложить подход») в MVP-форму не входит — см. §9. Ключ оценивания всегда определяется шаблоном, а не LLM.

Источник

`docs/requirements/testing.md` («Формат заданий»); `docs/requirements/overview.md` («Содержание и формат заданий команда определяет сама»).

Обоснование

Автопроверка обеспечивает сопоставимость кандидатов, воспроизводимость результата и отсутствие нагрузки человеческой проверки в контуре MVP.

Влияние

`docs/testing_model.md §3.1`, `§3.6`; `docs/mvp_scope.md §2.4`.

Связанные вопросы

`docs/backlog/closed_questions.md` — Q-2.

7. Синтетические данные и seed

Что

Готовых датасетов кандидатов и вакансий, как и эталонной разметки для проверки качества, в рамках хакатона нет (`docs/requirements/acceptance.md`).

Решение

- собственный синтетический seed, **идемпотентный**, запускается одноразовым контейнером `seed`;
- отдельные наборы кандидатов с историей ФСП (`history_status = present`) и без истории (`history_status = no_history`);
- демо-описания вакансий — seed-контекст; задания и тест формируются из калиброванного банка (шаблоны + якоря), а не из текстов вакансий;
- повторный запуск seed не создаёт дублей.

Источник

`docs/requirements/acceptance.md` («Элементы, прорабатываемые самостоятельно»); `docs/requirements/overview.md`.

Обоснование

Синтетика прямо разрешена условиями задачи. Обязательная обработка случая «нет истории» проверяется отдельным набором. Идемпотентность нужна для воспроизводимости демо и прогонов CI.

Влияние

`docs/architecture.md §5.1`; `docs/fsp_integration.md §4`; `docs/deployment.md §5`.

Связанные вопросы

`docs/backlog/closed_questions.md` — Q-4.

8. Топология БД: TOPOLOGY A и путь A → B

Что

Требование удерживать память в разумных пределах и микросервисная декомпозиция конфликтуют при выборе числа экземпляров СУБД.

Решение

TOPOLOGY A: одна PostgreSQL, схема и отдельная роль на сервис; запрет cross-schema join/FK; доступ только через роль владельца. Документирован путь **A → B** (физическая PostgreSQL на сервис) как изменение только connection strings, без переписывания кода.

Источник

`docs/requirements/nonfunctional.md` («Аппаратные требования», «Архитектура»).

Обоснование

Логическая изоляция обеспечивается уже сейчас, единый backup и экономия памяти важны для демо. Жёсткая конвенция «нет cross-schema доступа» делает переход к физическому разделению конфигурационным, а не миграционным.

Влияние

`docs/architecture.md §5.2`, `§5.4`; `docs/deployment.md §1`, `§3`.

Связанные вопросы

Вопрос не выделялся; решение команды.

9. Свободные ответы вне MVP грейд-теста (Decision D)

Что

Свободные ответы («предложить подход») требуют содержательной проверки, обеспеченность которой в контуре MVP под вопросом.

Решение

Decision D. Свободные ответы **исключены из MVP-формы грейд-теста**; MVP-форма содержит только автопроверяемые форматы (§6). Свободный ответ остаётся в продукте **только в регулярных заданиях работодателей**, где оценивается работодателем (статусы `positive` | `negative` | `unreviewed`), без нагрузки 24/7.

Расширение `[ext]` — оценивание свободных ответов российским облачным LLM по **фиксированной человеческой рубрике**:

- авто-вердикт допустим только при `≥ 8/10` или `≤ 3/10`; всё промежуточное и любой прокторинг-флаг → **очередь человеческой проверки**;
- очередь — пакетно, в рабочие часы, SLA **24–48 рабочих часов** (не 24/7);
- **никогда не auto-fail**; ключ оценивания и сложность `b` LLM не определяет;
- пиннинг версии модели и лог хеша промпта на каждый вердикт, `temperature = 0`;
- fallback при недоступности API — задание исключается из скоринга, попытка пересчитывается (безопасно за счёт якорей и эквайтинга);
- локальный хостинг LLM **исключён** требованиями к железу (`docs/requirements/nonfunctional.md`, «Аппаратные требования»).

Источник

`docs/requirements/testing.md` («Формат заданий», «Регулярные задания»); `docs/requirements/acceptance.md` («Элементы, прорабатываемые самостоятельно»); `docs/requirements/nonfunctional.md`.

Обоснование

Автопроверяемые форматы снимают риск необеспеченной проверки в MVP и сохраняют сопоставимость результата. Перенос свободного ответа в периодические задания даёт ему обеспеченную оценку человека и не создаёт зависимости от 24/7-контура.

Влияние

`docs/testing_model.md §3.1`, `§3.6`, `§9.2`; `docs/periodic_tasks.md §6.1`; `docs/mvp_scope.md §2.4`, `§3.1`, `§4`; `docs/validation.md`.

Связанные вопросы

`docs/backlog/closed_questions.md` — Q-2.

10. Политика удаления аккаунта

Что

Порядок удаления аккаунта и связанных данных ТЗ не задаёт.

Решение

- аккаунт удаляется **soft-delete** (пометка `deleted_at`, доступ отзывается);
- персональные ссылки в логах заданий **анонимизируются** (разрываются), агрегаты сохраняются для аудита и целостности статистики;
- при удалении запускаются каскады по владельцу данных (например, отставление заданий и истечение открытых назначений).

Источник

`docs/requirements/nonfunctional.md` («Безопасность и 152-ФЗ»); `docs/requirements/roles.md`, правило 4.

Обоснование

Soft-delete сохраняет ссылочную целостность и историю для аудита 152-ФЗ, а анонимизация логов убирает персональные связи. Полное физическое удаление сопряжено с риском осиротевших ссылок в схемах сервисов.

Влияние

`docs/periodic_tasks.md §8` (граничный случай «аккаунт удалён»); `docs/data_model.md §3.1` (`auth.users.deleted_at`), `§13` (аудит и история).

Связанные вопросы

Вопрос не выделялся; решение команды.

11. Якорение периодических заданий (вакансия/потребность)

Что

Модуль публикации вакансий — `[ext]` (см. `docs/mvp_scope.md`), поэтому на MVP у работодателя может не быть опубликованной вакансии; при этом регулярные задания должны выдаваться и в этом случае. Требуется определить, к чему якорится задание и по какому признаку строится пул выдачи.

Решение

- базовое якорение — на вакансию: `Task.vacancy_id` (логическая ссылка на `outreach.vacancies.id`, `[ext]`); в fallback-режиме промпт также несёт `EmployerTaskPrompt.need_id`;
- если вакансия не опубликована (модуль `[ext]` не построен) — задание якорится на «описание потребности»/компанию работодателя через логическое поле `Task.need_id` (`employer.needs.id`);
- оба поля якоря NULLable; заполняется ровно одно из них;
- фильтр розыгрыша (пул выдачи) поддерживает **оба** якоря: задание попадает в пул, если связано с вакансией либо с потребностью;
- обязательный фильтр по специализации кандидата и опциональный `[ext]` фильтр по грейду сохраняются без изменений.

Источник

`docs/requirements/roles.md` («Кандидат», «Регулярные задания» — желательно, но не обязательно для MVP); `docs/requirements/testing.md` («Регулярные задания»); `docs/mvp_scope.md` (модуль вакансий — `[ext]`).

Обоснование

Без логического якоря `Task.need_id` fallback-розыгрыш невозможен: работодатель без опубликованной вакансии не смог бы получить задание, и механика регулярных заданий зависела бы от `[ext]`-модуля. Два NULLable-якоря позволяют строить пул выдачи независимо от наличия вакансии, не нарушая HARD RULE об отсутствии cross-schema FK.

Влияние

`docs/data_model.md §6.2` (`Task.need_id`, `EmployerTaskPrompt.need_id`, enum `action`), `§2` (ERD-связь); `docs/periodic_tasks.md §3–§5`; `docs/mvp_scope.md`.

Связанные вопросы

`docs/backlog/closed_questions.md` — TD-7.

Итоговая сводка

Назначение документа

Краткая сводка собранного пакета документации PullHire: что поставлено относительно ТЗ, где зафиксированы решения, какие границы и упрощения приняты осознанно, как проверен и экспортирован пакет. Документ опирается на весь authored-пакет `docs/` и **не вводит новых решений**.

Связанные документы: `docs/mvp_scope.md` (границы), `docs/assumptions.md` (реестр решений), `docs/backlog/closed_questions.md` (история закрытия), `docs/examples/README.md` (артефакты), `docs/worklog.md` (журнал работ).

Источники требований: `docs/requirements/acceptance.md`, `docs/requirements/evaluation.md`, `docs/requirements/nonfunctional.md`.

Вне рамок: пересказ содержания профильных документов; здесь только сводка и карта ссылок.

1. Поставлено vs ТЗ

Критерий ТЗ	Что сделано	Где доказательство
Новая механика подбора (35%)	LHH-механика: категория = специализация × грейд, внутрикатегорийное ранжирование, подборка по потребности с объяснимостью, поиск/фильтры, приглашение с обязательной зарплатой, раскрытие контактов после принятия	`docs/selection.md`, `docs/user_flow.md §3`, `docs/data_model.md §7`, `§9`
Механика тестирования, валидация, категоризация (25%)	Обязательный конвейер «опрос → заявленный грейд → тест → категория»; calibrated template pools + IRT-lite (1PL) + якоря + cut-score; анти-утечка; запрет принудительного понижения; cooldown	`docs/testing_model.md`, `docs/validation.md`, `docs/data_model.md §6.1`
Удобство и интуитивность интерфейса (10%)	Сквозной путь в обоих кабинетах, роль-скоупная маршрутизация, демонстрационный сценарий	`docs/user_flow.md`, `docs/deployment.md §6`
Техническая проработка решения (7%)	Микросервисы (8 сервисов + gateway + SPA), REST + RabbitMQ, TOPOLOGY A и путь A→B, security/152-ФЗ, OpenAPI, CI	`docs/architecture.md`, `docs/stack.md`, `docs/deployment.md`
Полнота реализации и сопроводительная документация (6%)	Пакет `docs/` закрывает все пункты сопроводительной документации; обработка «нет истории ФСП» обязательна и нейтральна	`docs/` (весь пакет), `docs/fsp_integration.md §6`, `docs/mvp_scope.md`
Дополнительный функционал (7%)	Регулярные задания и классический поток вакансий/откликов `[opt]` — спроектированы полностью (реализация второй очередью); защита от фиктивных вакансий и интеграция с ATS — только концепция (`docs/mvp_scope.md §4`), без реализации	`docs/periodic_tasks.md`, `docs/mvp_scope.md §3`, `§4`
Презентация и питч-сессия (10%)	Демо-сценарий, инструкция по локальному запуску, видео-фолбэк, экспортируемый пакет doc/pdf	`docs/user_flow.md §4`, `docs/deployment.md §6`, `docs/examples/`
Сопроводительная документация: архитектура	Функциональная и компонентная архитектура, схема сервисов, взаимодействие	`docs/architecture.md`
Сопроводительная документация: механики и схема ФСП	Тестирование, подбор, схема интеграции с ФСП, API, библиотеки с версиями, инструкция запуска	`docs/testing_model.md`, `docs/selection.md`, `docs/fsp_integration.md`, `docs/api.md`, `docs/stack.md`, `docs/deployment.md`
Ссылка на документацию в формате doc/pdf	Экспорт пакета в единый doc/pdf; артефакт — в `docs/examples/package/`	`docs/examples/README.md §3`, `docs/examples/package/`

2. Карта решений

Решение	Где зафиксировано
Cooldown = 1 месяц	`docs/assumptions.md §2`; `docs/backlog/closed_questions.md` Q-3

Решение	Где зафиксировано
Периодические задания: `n = 7`, синхронные окна, opt-in, freshness display-only	`docs/assumptions.md §3`; `docs/backlog/closed_questions.md` Q-2
Каталог специализаций и грейдов	`docs/assumptions.md §4`
Состав полей ФСП, Keycloak-совместимость как ориентир	`docs/assumptions.md §5`; `docs/backlog/closed_questions.md` Q-1, Q-4
Форматы заданий грейд-теста (только автопроверяемые)	`docs/assumptions.md §6`; `docs/backlog/closed_questions.md` Q-2
Синтетические данные и идемпотентный seed	`docs/assumptions.md §7`; `docs/backlog/closed_questions.md` Q-4
Топология БД: TOPOLOGY A и путь A → B	`docs/assumptions.md §8`; `docs/architecture.md §5.2`, `§5.4`
Decision D: свободные ответы вне MVP-грейд-теста	`docs/assumptions.md §9`; `docs/mvp_scope.md §2.4`, `§4`; `docs/backlog/closed_questions.md` Q-2
Политика удаления аккаунта (soft-delete + анонимизация логов)	`docs/assumptions.md §10`
Event-contract: republish `assessment.category_assigned`	`docs/backlog/closed_questions.md` TD-1; `docs/testing_model.md §7.1`, `§8`
Обязательный scope по специализации	`docs/backlog/closed_questions.md` TD-2; `docs/periodic_tasks.md §4`
Хранение опроса и seed-полей	`docs/backlog/closed_questions.md` TD-3; `docs/data_model.md §6.1`
Snapshot `employer_contact` без cross-schema join	`docs/backlog/closed_questions.md` TD-4; `docs/data_model.md §9.1`
Exposure caps на уровне семейства / задания	`docs/backlog/closed_questions.md` TD-5; `docs/testing_model.md §9.1`
Санкционированное отклонение `testing_model §9.5`	`docs/backlog/closed_questions.md` TD-6; `docs/worklog.md` (Phase 5)
Микросервисы, Java 21 + Spring Boot, React/Next.js, RabbitMQ, Flyway, springdoc	`docs/architecture.md §3`, `§5`; `docs/stack.md`

3. Границы и осознанные упрощения

- **Свободный ответ вне MVP-грейд-теста** (Decision D): MVP-форма — только автопроверяемые форматы; «предложить подход» живёт в регулярных заданиях, оценивается работодателем (`docs/assumptions.md §9`).
- **Регулярные задания работодателей** — `[opt]`, optional-MVP: спроектированы полностью, реализуются второй очередью (`docs/periodic_tasks.md`, `docs/mvp_scope.md §3.1`).
- **Классический поток вакансий и откликов** — `[opt]`, свёрнутый модуль в `outreach-service` с документированным путём выделения (`docs/mvp_scope.md §3.2`).
- **ML/NLP-разбор резюме** — rule-based fallback в MVP; ML/NLP-воркер помечен `[ext]` (`docs/mvp_scope.md §3.4`).
- **Мобильная адаптивность** — `[opt]`; целевой минимум — десктопные браузеры (`docs/mvp_scope.md §3.3`).
- **Защита от фиктивных вакансий и ATS** — только концепция и архитектурный подход, без реализации (`docs/mvp_scope.md §4`).
- **Реальный API реестра ФСП** — mock-адаптер и синтетические данные; модель устойчива к неизвестному контракту (`docs/fsp_integration.md §4`, `§7`).
- **Калибровка тестов** — IRT-lite (1PL), сложности провизорны до накопления данных; MST — fallback (`docs/testing_model.md §3.5`, `§5.3`).
- **Топология данных** — одна PostgreSQL (TOPOLOGY A) со схемой и ролью на сервис; переход к физическому разделению — конфигурационный (`docs/architecture.md §5.2`).
- **Инфраструктурные упрощения MVP** — Compose DNS вместо discovery, Nginx вместо BFF, без K8s/Consul/распределённого трейсинга; Prometheus/Grafana опциональны (`docs/architecture.md §4.1`, `§7`).
- **Оценивание LLM** — только облачный LLM `[ext]` по фиксированной рубрике; локальный хостинг исключён требованиями к железу (`docs/assumptions.md §9`).

4. Проверка и экспорт пакета

Проверка пакета. Каждый authored-документ `docs/` проверяется на соблюдение конвенций: один `#` H1, первый раздел `## Назначение документа`, нумерованные `## N.` / `### N.M.` с точкой, inline-ссылки вида `docs/x.md §N`, бэктик-пути без `[text](path)`-ссылок. Источники `docs/requirements/` и `docs/source/` остаются read-only и в пакет не входят.

Экспорт. Пакет `docs/*.md` экспортируется в единый формат doc/pdf; итоговый артефакт размещается в `docs/examples/package/`. Собранный экспорт — `docs/examples/package/PullHire-docs.pdf` (плюс исходные `PullHire-docs.md` и `PullHire-docs.html`), генерируется скриптом `docs/examples/package/build-docs.ps1` (`docs/*.md` → бандл → `npx`

`marked` → headless Edge `--print-to-pdf`). Структура репозитория и требование «ссылка на сопроводительную документацию в формате doc или pdf» закрываются этим артефактом (`docs/requirements/acceptance.md`, «Сопроводительная документация»).

Артефакты. `docs/examples/openapi/` — per-service OpenAPI, генерируется springdoc на этапе сборки (резервируется как build-артефакт, не поддерживается вручную); `docs/examples/package/PullHire-docs.pdf` — doc/pdf-экспорт пакета, создаётся при сборке (сгенерированные файлы в репозиторий автоматически не коммитятся). Индекс артефактов, что их генерирует и как воспроизвести — в `docs/examples/README.md`.

5. Подробнее

Тема	Документ
Архитектура, сервисы, топология, безопасность	`docs/architecture.md`
ER-модель, сущности, поля, перечисления	`docs/data_model.md`
Механика грейд-тестирования и присвоение категории	`docs/testing_model.md`
Регулярные задания работодателей	`docs/periodic_tasks.md`
Категории, ранжирование, подборка, LHH-выход на контакт	`docs/selection.md`
Процедура валидации решения и результаты	`docs/validation.md`
Схема интеграции с реестром ФСП	`docs/fsp_integration.md`
Индекс API и конвенции	`docs/api.md`
Библиотеки, компоненты, лицензии, открытость кода	`docs/stack.md`
Сборка, развёртывание, локальный запуск	`docs/deployment.md`
Границы MVP и опциональные срезы	`docs/mvp_scope.md`
Сквозные пользовательские сценарии и демо-путь	`docs/user_flow.md`
Термины	`docs/glossary.md`
Реестр допущений и решений	`docs/assumptions.md`
Итоговая сводка	`docs/final_summary.md`
Журнал закрытых вопросов и техдолга	`docs/backlog/closed_questions.md`
Примеры артефактов (OpenAPI, экспорт пакета)	`docs/examples/README.md`
Журнал сборки пакета	`docs/worklog.md`

Журнал закрытых вопросов

Назначение документа

Единый журнал истории вопросов, поднимавшихся при сборке пакета PullHire, и их закрытия. Дополняет реестр `../assumptions.md` (зафиксированные решения) и не дублирует его: здесь — вопрос, откуда он возник, требуемое действие и итоговый статус. Нумерация сквозная по двум пространствам: `Q-N` (вопросы и ответы) и `TD-N` (технический долг).

Связанные документы: `../assumptions.md`, `../testing_model.md`, `../periodic_tasks.md`, `../fsp_integration.md`, `../mvp_scope.md`, `../worklog.md`.

Источники требований: `docs/requirements/acceptance.md`, `docs/requirements/testing.md`, `docs/requirements/overview.md`, `docs/requirements/profiles.md`, `docs/requirements/nonfunctional.md`.

Вне рамок: сами принятые решения как таковые (они в `../assumptions.md`); здесь — только динамика закрытия.

1. Формат записи

Каждая запись — подраздел `### Q-N.` или `### TD-N.` со полями:

- `Что` — суть вопроса;
- `Откуда` — документ, где вопрос возник;
- `Действие` — что требовалось или было сделано;
- `Статус` — один из вердиктов: `решён командой`, `закрыт ответом из требований`, `осознанная граница`, `вне объёма`;
- `Резолюция` — итог со ссылкой на `../x.md` или `docs/requirements/x.md`.

Ссылки из `docs/backlog/` на authored-документы — `../x.md`; на источники требований — `docs/requirements/x.md`.

2. Группа Q. Вопросы и ответы (Q-N)

Q-1. Совместимость с Keycloak

- Что: обязательна ли совместимость авторизации с Keycloak (ФСП ID «в дальнейшем» будет работать на базе Keycloak).
- Откуда: `docs/requirements/nonfunctional.md` («Аутентификация»); `docs/requirements/profiles.md` («Связь с ID участника ФСП»).
- Действие: принять решение о способе аутентификации; при выборе Keycloak-совместимого пути обеспечить форму claim-ов и JWKS.
- Статус: закрыт ответом из требований.
- Резолюция: совместимость — **ориентир, не требование**; выбран JWT RS256 с Keycloak-совместимыми claim-ами (`sub`, `realm_access.roles`) и `/.well-known/jwks.json`; см. `../fsp_integration.md §2`, `../architecture.md §6.1`.

Q-2. Формат заданий

- Что: каким должен быть формат заданий грейд-теста.
- Откуда: `docs/requirements/testing.md` («Формат заданий»); `docs/requirements/acceptance.md` («Элементы, прорабатываемые самостоятельно»).
- Действие: выбрать форматы и механику проверки.
- Статус: решён командой.
- Резолюция: форматы выбирает команда — закрытые вопросы, короткий ответ, задачи с кодом; свободный ответ вне MVP-грейд-теста (Decision D). См. `../assumptions.md §6`, `§9`; `../testing_model.md §3.1`.

Q-3. Интервал смены грейда

- Что: с какой частотой разрешать смену грейда.
- Откуда: `docs/requirements/testing.md` («Ограничение смены грейда»); `docs/requirements/overview.md` («Постановка задачи»).
- Действие: задать интервал cooldown.
- Статус: решён командой.

- Резолюция: принят **1 месяц**; см. `../assumptions.md §2`; `../testing_model.md §7.3`.

Q-4. API и структура ФСП

- Что: какие данные реестра ФСП получать и как отображать.
- Откуда: `docs/requirements/acceptance.md` («Ресурсы организатора»); `docs/requirements/profiles.md`.
- Действие: определить модель и источник данных при отсутствии предоставленного API.
- Статус: закрыт ответом из требований.
- Резолюция: API и структура данных **не предоставлены**; принята собственная модель и mock/синтетика; см. `../assumptions.md §5`, `§7`; `../fsp_integration.md §3–§4`.

3. Группа TD. Технический долг (TD-N)

TD-1. Event-contract: republish `category_assigned`

- Что: обновлять категорию только при первичном прохождении или при любой завершённой попытке.
- Откуда: `../testing_model.md §7`; `../architecture.md §4.2–4.3`.
- Действие: закрепить контракт событий между `assessment-service` и `selection-service`.
- Статус: решён командой.
- Резолюция: любая завершённая попытка (включая up/down и повторы) перепубликует `assessment.category_assigned`; `assessment.grade_changed` — audit/notification only. См. `../testing_model.md §7.1`, `§8`.

TD-2. Обязательный scope по специализации

- Что: обязателен ли фильтр по специализации в розыгрыше регулярных заданий.
- Откуда: `../periodic_tasks.md §4`, `§5.1`.
- Действие: закрепить обязательность фильтра и отделить его от опционального фильтра по грейду.
- Статус: решён командой.
- Резолюция: совпадение специализации — **обязательный** фильтр выдачи; фильтр по грейду — `[ext]`. См. `../periodic_tasks.md §4` (C3).

TD-3. Хранение опроса

- Что: где хранить ответы опроса и заявленные измерения.
- Откуда: `../testing_model.md §2.1`; `../data_model.md §6.1`.
- Действие: зафиксировать поля хранения.
- Статус: решён командой.
- Резолюция: `assessment.test_sessions.survey_answers` + `declared_dimensions`; отрасль — `candidate.profiles.industry_code` (`../data_model.md §4.1`); публичная карточка отдаёт industry/специализацию/грейд. Form-level `test_sessions.form_seed`, per-item `attempts.variant_seed`. См. `../data_model.md §4.1` (`industry_code`), `§6.1`.

TD-4. Snapshot `employer_contact`

- Что: как приглашение получает контакт работодателя без cross-schema join.
- Откуда: `../data_model.md §9.1`; `../architecture.md §5.2`.
- Действие: выбрать способ доставки контакта в схему `outreach`.
- Статус: решён командой.
- Резолюция: приглашение несёт snapshot `employer_contact` (способ связи); `applications` несёт `released_contact` / `contact_released_at`; cross-schema join не используется. См. `../data_model.md §9.1`, `§9.3`.

TD-5. Exposure caps на уровне семейства / задания

- Что: на каком уровне ограничивать число показов заданий банка.
- Откуда: `../testing_model.md §3.4`, `§9.1`; `../data_model.md §6.1`.
- Действие: определить уровень лимитов и поведение при исчерпании.
- Статус: решён командой.

- Резолюция: caps на уровне `template_family` (NULLable, у якорей отсутствует, с CHECK) и задания; при исчерпании — вывод из ротации. См. `../testing_model.md §3.4`, `§9.1`; `../data_model.md §6.1`.

TD-6. Отклонение структуры `testing_model §9.5`

- Что: подраздел `### 9.5. Антипаттерны` в `../testing_model.md` не следует шаблону остальных подразделов.
- Откуда: `../worklog.md` (Phase 5).
- Действие: зафиксировать отклонение или привести раздел к шаблону.
- Статус: осознанная граница (sanctioned).
- Резолюция: отклонение **санкционировано** и зафиксировано в `../worklog.md` (Phase 5); раздел оставлен как содержательный перечень антипаттернов.

TD-7. Пакет правок финального ревью

- Что: результаты финального сквозного ревью пакета документации — набор точечных правок, выявленных проверкой согласованности.
- Откуда: `../worklog.md` (финальное ревью).
- Действие: внести правки в затронутые документы, не меняя структуру и границы MVP.
- Статус: решён командой.
- Резолюция: (1) в `../mvp_scope.md §4` зафиксирован **blocker** — концепт «фиктивная вакансия/ATS»; (2) переформулирован event-contract (`../architecture.md §4.2`); (3) исправлена ссылка на deployment-документ; (4) свежесть профиля добавлена в карточку; (5) закреплено fallback-якорение заданий (`../assumptions.md §11`; `../data_model.md §6.2`); (6) soft-delete аккаунта отражён в схеме через `auth.users.deleted_at` (`../data_model.md §3.1`; `../assumptions.md §10`).

TD-8. Артефакт экспорта doc/pdf

- Что: в пакете отсутствует артефакт экспорта документации в формате doc/pdf.
- Откуда: `../worklog.md` (финальное ревью).
- Действие: определить источник и способ генерации экспорта.
- Статус: осознанная граница.
- Резолюция: экспорт `doc/pdf` — артефакт сборки, не authored-документ; зафиксирован как техдолг и не блокирует MVP.

4. Хронология закрытия

Дата	Этап	Закрыто
2026-10-05	docs/requirements	Baseline источников (verbatim-цитаты T3)
2026-10-06	Phase 5: core docs	Q-3 (cooldown = 1 месяц); TD-1 (event-contract), TD-3 (хранение опроса), TD-5 (exposure caps); TD-6 (санкционированное отклонение `testing_model §9.5`)
2026-10-06	Phase 6: structural docs + examples/	Q-1 (Keycloak), Q-2 (форматы заданий), Q-4 (API/структура ФСП); TD-2 (scope по специализации), TD-4 (snapshot `employer_contact`)
2026-10-06	Финальное ревью	TD-3 (ссылка `data_model §4.1`); TD-7 (пакет правок финального ревью), TD-8 (артефакт экспорта doc/pdf)

Worklog

2026-10-05 — docs/requirements/

- Источник: `docs/source/ФСП спец. трек Т3.pdf` (PPTX — дизайн-шаблон, не источник).
- Создано 9 файлов `docs/requirements/` — verbatim-цитаты, структура по эталону RoboMatch.
- Верификация: 142/142 фрагментов совпали с PDF-дампом.
- `context.md` переименован в `overview.md` (коллизия с harness `CONTEXT.md`).
- Открыто: пустые placeholders `## Что следует для реализации` / `## Вопросы разработки`.

2026-10-06 — Phase 5: core docs

- Сделано: созданы `docs/architecture.md`, `docs/data_model.md`, `docs/testing_model.md`, `docs/periodic_tasks.md`.
- Структура — по дизайну Phase 5a (единый H1, `## Назначение документа`, нумерованные `## N.`).
- Architecture: 8 бизнес-сервисов + api-gateway + SPA; TOPOLOGY A и путь A → B; RabbitMQ/outbox.
- Data_model: ER по 8 схемам; сущности регулярных заданий — §6.2–6.3.
- Testing_model: calibrated template pools + IRT-lite (1PL) + якоря + cut-score; cooldown.
- Periodic_tasks: каденс `n = 7`, opt-in, дедупликация task-level, freshness display-only.
- Отклонения от структуры: `docs/testing_model.md §9.5 «Антипаттерны»` — санкционированное отклонение (не следует шаблону подразделов); запись — `docs/backlog/closed_questions.md` TD-6.
- Открыто: финальная сверка перекрёстных ссылок — Phase 6.

2026-10-06 — Phase 6: structural docs + examples/

- Сделано: `docs/selection.md`, `docs/validation.md`, `docs/fsp_integration.md`, `docs/api.md`, `docs/stack.md`, `docs/deployment.md`, `docs/mvp_scope.md`, `docs/user_flow.md`, `docs/glossary.md`; реестр `docs/assumptions.md`; сводка `docs/final_summary.md`; журнал `docs/backlog/closed_questions.md`; каталог `docs/examples/` (README + `openapi/` + `package/`).
- Закреплено: Decision D (свободные ответы вне MVP-грейд-теста) — `docs/assumptions.md §9`, `docs/mvp_scope.md §4`.
- Отклонения от структуры: нет; `docs/testing_model.md §9.5` остаётся санкционированным (TD-6).
- Открыто: сборка per-service OpenAPI в `docs/examples/openapi/` и экспорт doc/pdf в `docs/examples/package/` — порядок воспроизведения в `docs/examples/README.md`.